AIACC 指令集架构

发布版本	0.4.15
发布时间	2025.4.12

删除[SYT]: 2

删除[SYT]: 3

删除[SYT]: 23

注意

本文件所含信息均具有保密性质并仅限于内部使用。不得对本文件、本文件的任何部分或本文件所含的任何信息进行未经授权地使用、披露或复制。

NOTICE

The information contained in this document is confidential and is intended only for internal use. Unauthorized use, disclosure or copying of this document, any part hereof or any information contained herein is strictly prohibited.

修订记录

版本号	修订日期	作者	修订内容
0.1.0	2025.1.1		Initial draft
0.2.7	2025.10.22		第一次正式发布
0.2.8	2025.12.14		对 SMC/INTADD 模块的微指令增加输出符号控制
0.3.0	2025.12.17		增加 SMI_IQ
0.3.1	2025.12.18		增加基于微指令视角的编程指南
0.3.2	2025.12.21		增加 RtoVR 宏定义相关的设计文档（excel 表格和 ppt 文档）
0.3.3	2025.12.31		替换 SMC_IQ 为 SMI_IQ 针对卷积和矩阵乘法改进了 SMI_IQ，增加了嵌套循环功能
0.3.4	2026.1.5		修正 SMI_IQ 中 CODE 的定义部分和编程指南中浮点加法的例子中 INC 的错误
0.3.5	2026.1.5		编程指南中浮点加法的例子中输入编号和 bank 的错误； 根据讨论修改 EMBED 的定义
0.3.6	2026.1.9		对基于 7.1 进行了重写； 增加了 SMI_IQ 中 ready 相关的功能表述和接口定义
0.3.7	2026.1.11		修改了 EMBED 和 JUMP 指令
0.3.8	2026.1.12		根据反馈的问题改进了 EMBED 指令
0.3.9	2026.1.12		补充对于 INC_EMBED
0.3.10	2026.1.14		补充对于 EMBED 指令的表述
0.3.11	2026.1.14		补充对于 INC_EMBED 和 VLD 的表述 修改 RtoVR 指令的 bit 数
0.3.12	2026.1.21		完成 SMI_DS_T 初版，满足矩阵乘法计算的需求，后续有可能根据新需求更新这个部分，
0.3.13	2026.1.21		更新了矩阵存储和 STB、LDB 的部分 1）为 STB、LDB 增加 Line mode 和 outstanding 功能 2）增加张量在外部地址空间存储的表述，完善张量在 UR 中存储的表述
0.3.14	2026.1.22		初始化 SMI 的架构章节 更新 SMC 的架构章节
0.3.15	2026.1.26		改进了张量存储和矩阵乘法计算部分 补充了 line mode 的表述，增加了 burst 第一个 cycle 的 strobe 修改了 SMC 里面计算单元微指令的 LSB INTADD 和 FPADD 增加了减法功能
0.3.16	2026.1.27		对调了 6 和 7 两个部分 改进了向量与矩阵乘法和矩阵与矩阵乘法的部分 修正了 STB、LDB 和 FPMUL16 的一些问题

0.3.17	2026.2.4		修改 STB 和 LDB，将指令部分和设计分为两个部分
0.4.0	2026.2.5		细化目前支持的 LLM 量化方式
0.4.1	2026.2.5		为每个模块添加了 smc_id_en 指令字段 修改了 STB 和 LDB 中读取数据的数据量定义
0.4.2	2026.2.5		将所有的-替换为_，未使用修订模式 highlight 这个改动
0.4.3	2026.2.6		将中文的信号名修改为英文
0.4.4	2026.2.10		补充了对于 get_first_one 和 get_first_zero 的功能说明； 修改了 STB 和 LDB 的信号位宽； 补充了 UR 中的位宽信息； 修改了 SHIFT*模块中 smc_id_en 的信号功能； 部分-是减号，部分-代表至，不能统一替换为_，进行修改； 修改 SMC_ID 的说明和初始化的方式； 知识库中其他位置初始化《AIACC 指令架构 5.4.2.2 SMI 指令寄存器 v0.1.0》。
0.4.5	2026.2.13		格式整理
0.4.6	2026.2.13		修改数据存储和矩阵乘法部分 基于 LLAMA2 7B 对矩阵乘法举例
0.4.7	2026.2.15		为 STB 和 LDB 增加 Transpose 和 reshape 功能，增加 DS 模块
0.4.8	2026.2.25		修订 LDB 和 STB 中与 DS 相关的部分 修改 UR 和 STB/LDB 的接口以及说明
0.4.9	2026.3.4		增加 SMI-IS 模块 SMI 模块接口中增加与 SMC 接口的连接说明 修改 DS 中对于 tid 的功能表述
0.4.10	2026.3.4		考虑加法树在向量计算中的应用，修改了 INTADD、INT8toFP16 、 FP16MUL 、 FP32ADDTREE5to1 、 FP32ADDTREE8to1 的源数据和目的数据的数据格式 针对 LDB、STB 修改 BITS_INC
0.4.11	2026.3.16		修改错误： 1）FP16MUL (5.3.3)一处 Typo，D0 功能公式 Copy-Paste 错误，S0*S0 应为 S0*S4 2）STB (5.3.15)一处 Typo，tid 字段标注 1bit 但编码需要 2bit，同时更新 SMI 指令寄存器 excel 表格 3）MOUT0 (5.3.24)一处问题，微指令缺少 smc_id_en 字段 SMC/SMI 接口 CRU_O/CRD_O 列表缺失 DS 、CRU_FP16MUL_S0123_O、CRU_FP16MUL_S4567_O、等信号 4）UR (5.3.17)中 ur_raddr_0 的位宽 Typo 5）对 INC 定义进行了补充，明确 INC 即 INC_i，i 来着 EMBED 字段

			6）对于 SMI-IQ 的写入和读取进行了进一步的说明
0.4.12	2026.3.23		1）FP32ADDTREE5TO1 的功能说明不够准确，进行改进 2）对 INTADD 支持的数据精度组合进行补充说明 3）VIN0、VIN1、VOUT0、MIN0、MIN1、MOUT0 的微指令表标题均写“下行微指令表”改为“上行微指令表” 4）针对 SMI 和 SMC 寄存器复位的默认值进行说明 5）对 cfg_en 和 cfg_rst 进行补充说明 6）对 UR 复位之后的默认值进行补充说明 7）对 SMI 控制寄存器中的中断进行补充说明 8）为 STB、LDB 增加 AXI 出错的 err 信息 9）修改 STB、LDB 的 line_byte_cnt 的位数 10）修改上一版本中 SMI_IQ 错误删除 WRD_8，在 V1.0 中也需要这些信号支持带有广播数据的算子 11）对 COND 的使用增加说明 12）改进对于间插展开的说明
0.4.13	2026.3.27		补充 SMI 寄存器的偏移地址
0.4.14	2026.4.4		修改 SMU 的 Typo 为 SMC FP32ADDTREE8TO1 功能说明中修改了一处 Typo 删除 FPtoFP 中 FP32toFP32 和 FP16toFP16 的说明，实现中不需要支持这两种情况 SHIFTUP 功能说明中修改一处 Typo SHIFTDN 的功能表述不正确，进行修改，增加 CRU
0.4.15	2026.4.7		修改了对于 COND 的表述和 SMI_IR 的输出 IR_INST_RDY 的表述。 修改 DS 的设计参数和功能说明中关于 outstanding 的表述。 增加 LDB 的功能表述支持填充一整个行为常数。
			● V2 ■ 卷积支持 ■ 更多量化支持（优先卷积、KVcache） ■ STB 和 LDB 支持特殊功能（Scatter、Gather、SQZ、UNSQZ、None_Zero、Masked Select） ■ LSU 一般功能和特殊功能（Scatter、Gather、SQZ、UNSQZ、None_Zero、Masked Select） ■ LDB 和 STB 的 AXI ID 考虑多个 参考： https://docs.pytorch.org/docs/stable/generated/torch.nonzero.html#torch-nonzero https://docs.pytorch.org/docs/stable/generated/torch.masked_select.html

设置格式[SYT]: 无项目符号或编号

目 录

1 目的 10

2 适用范围 10

3 引用文件 10

4 术语和定义 10

5 系统架构 11

5.1 架构概览 11

5.1.1 MID（宏指令译码单元）（待完善） 13

5.1.2 SMI（流多处理单元的指令单元） 13

5.1.3 SMC（流多处理单元的计算单元） 14

5.1.4 LSU（存取单元）（待完善） 15

5.2 SMC 架构 16

5.2.1 SMC 接口 16

5.2.2 SMC 寄存器 19

5.2.3 SMC 相关的设计参数 20

5.3 SMC 微架构和微指令 20

5.3.1 INTADD（Integer Adder） 21

5.3.2 INT8toFP16（Integer To Float Convert） 24

5.3.3 FP16MUL（Float16 Multiply） 25

5.3.4 FP32ADDTREE5TO1（Float32 Adder Tree 5to1） 30

5.3.5 FP32ADDTREE8TO1（Float32 Adder Tree 8to1） 33

5.3.6 FPMUL（Float Multiply）.....35
5.3.7 FPADD（Float Adder）.....37
5.3.8 FPtoFP（Float To Float Convert）.....39
5.3.9 INTtoINT（Integer To Integer Convert）.....41
5.3.10 FPtoINT（Float To Integer Convert）.....43
5.3.11 INTtoFP（Integer To Float Convert）.....45
5.3.12 LOGIC.....48
5.3.13 SHIFTUP（Shift Up）.....51
5.3.14 SHIFTDN（Shift Down）.....53
5.3.15 STB（Burst Store）.....55
5.3.16 LDB（Burst Load）.....59
5.3.17 UR（User Register）.....64
5.3.18 RtoVR（DR/UR to DVR/UVR datapath）.....69
5.3.19 VIN0（Vector Input Pipeline Register 0）.....73
5.3.20 VIN1（Vector Input Pipeline Register 1）.....74
5.3.21 VOUT0（Vector Output Pipeline Register 0）.....75
~~5.3.22 MIN0（Matrix put Pipeline Register 0）~~.....76
~~5.3.23 MIN1（Matrix Input Pipeline Register 1）~~.....77
~~5.3.24 MOUT0（Matrix Output Pipeline Register 0）~~.....78
5.3.25 DS（Data Shuffle）.....80
5.4 SMI 架构.....83
5.4.1 SMI 接口.....83
5.4.2 SMI 寄存器.....85

5.4.3 SMI 相关的设计参数 87
5.4.4 SMI 功能说明 87
5.5 SMI 微架构和微指令 88
5.5.1 SMI_IQ（Instruction Queue） 88
5.5.2 SMI_IR（Instruction Recycle） 95
5.5.3 SMI_DS_T（Data Shuffle_Transpose） 97
5.5.4 SMI_DS_SG（Data Shuffle_Scatter Gather）（待完善）（V1.0 不支持）. 100
5.5.5 SMI_DS_RD （ Data Shuffle_Reduce ） （ SQZ 、 UNSQZ 、 None Zero 、 Masked Select）（待完善）（V1.0 不支持） 100
5.6 LSU 架构（V1.0 不支持） 100
5.7 LSU 微架构和微指令（V1.0 不支持） 100
5.8 MID 架构（V1.0 不支持） 100
5.9 MID 微架构和微指令（V1.0 不支持） 100
6 基于宏指令视角的编程模型 100
6.1 Tensor Storage Model 100
6.1.1 User Register 100
6.1.2 张量存储于 SMC 外部地址空间 104
6.2 支持的数据类型 105
7 基于微指令视角的编程指南和样例 106
7.1 基于微指令视角的编程指南 106
7.2 基于微指令视角的编程样例 106
7.2.1 向量浮点加法 106

7.2.2 基于加法树的向量与矩阵乘法 109
7.2.3 基于加法树的矩阵与矩阵乘法 115
7.2.4 基于乘累加器的矩阵乘法（V1.0 不支持） 122
7.2.5 卷积（V1.0 不支持） 122
7.2.6 池化和深度分离卷积（V1.0 不支持） 124
8 宏指令集（后端接口 IR） 124
9 支持的量化方案（待完善） 125
9.1 Transformer 量化方案 125
9.1.1 W4IA16F 量化方案 125
9.1.2 W16FA16F 量化方案 125
9.1.3 W8IA8I 量化方案 125
9.1.4 W8FA8F 量化方案 125

1 目的

本文档编写目的是为了详细描述 AIACC V1 指令集、硬件架构，以便 IC 设计、IC 验证、建模仿真、工具链开发等人员能够充分了解和进行开发。

2 适用范围

本文档适用于 AIACC V1 的研发。

3 相关文件

表 3- 1：引用文件列表

序号	引用文件名称	说明
1.	《AIACC 指令集架构 附录 A 数值计算规定》	约定了所有数值计算的规则，需要注意的是即使计算产生溢出/NaN/Inf 也不会导致本模块异常或者报错。
2.		

4 术语和定义

表 4- 1：术语和定义表

序号	术语	定义说明
1.	ISA	Instruction Set Architecture，指令集
2.	MID	
3.	SMI	
4.	SMC	
5.	LSU	
6.	Stream	数据流
7.	UR	User Register
8.	DR	Data Path Register
9.	Macro Instruction	
10.	Micro Instruction	

5 系统架构

5.1 架构概览

AIACC 由一个宏指令译码单元（Macro Instruction Decode Unit，简称 MID）、一个流多处理单元的指令单元（Stream Multiprocessor Micro Instruction Unit，简称 SMI）和多个流多处理单元的计算单元（Stream Multiprocessor Compute Unit，简称 SMC）和存取单元（Load Store Unit，简称 LSU）构成。

宏指令译码器的功能是收取或者获得宏指令，进行译码产生微指令。宏指令可以由指令译码器主动从片外或者片上存储空间读取，也可以由主控处理器发射到 MID。

微指令发射单元的功能是向多个流多处理器发射微指令。

流多处理单元的功能是基于微指令进行数据的计算。

存取单元的功能是将 AIACC 内部数据存储到外部和将外部数据读取到 AIACC 内部。

架构概略图下图所示：

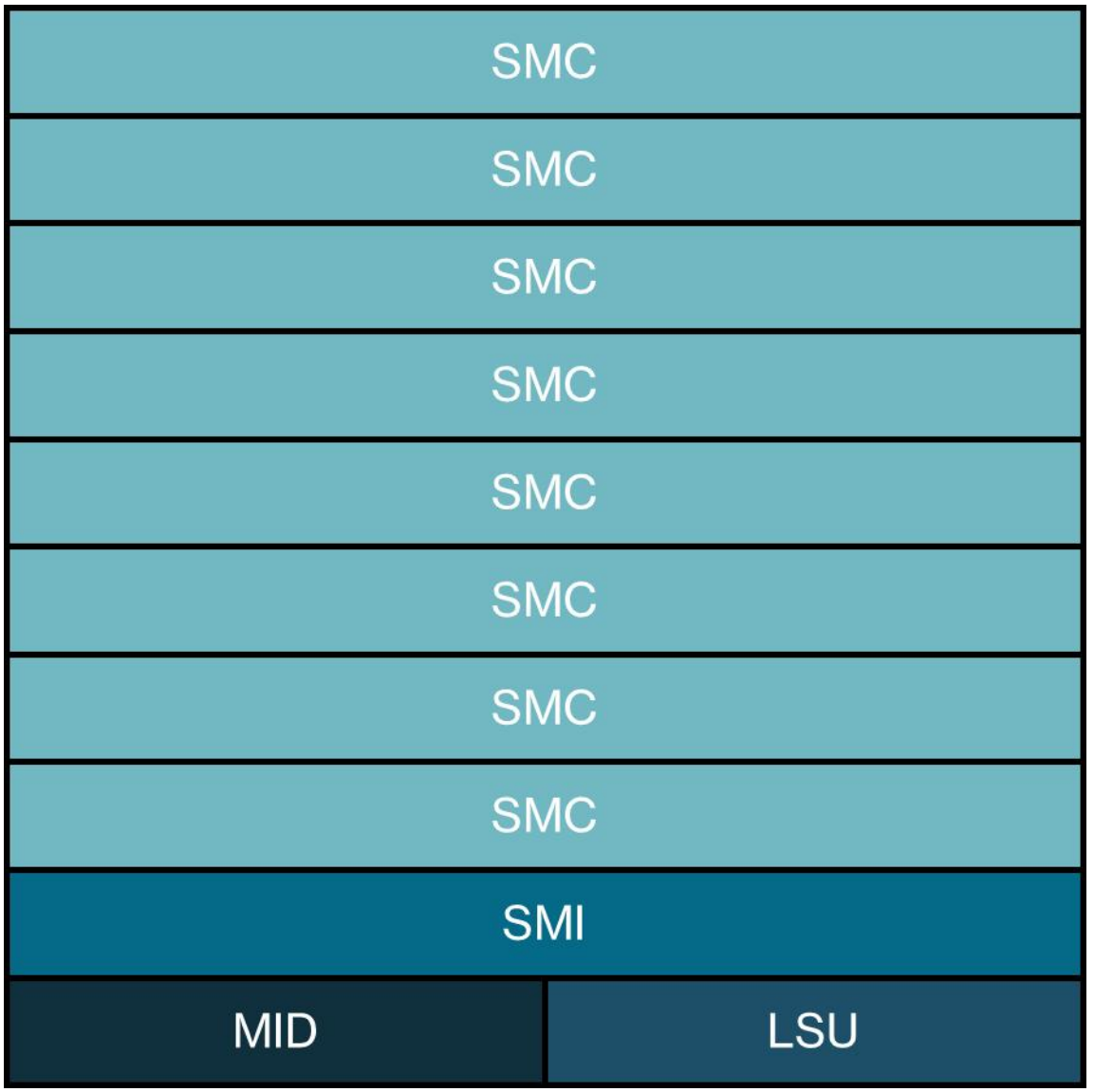


图 1 AIACC 架构示意图

AIACC 对外呈现的设计参数有：

- 各种精度和各种计算算子（向量计算和矩阵乘法计算）的计算单元数量
- 核内存储容量
- 核外数据访问接口的位宽
- 运行频率

AIACC 内部的设计参数有：

- SMC 的数量

删除[SYT]: U

对于 AIACC V1.0，上述参数的典型值为

表 5- 1：RISC-V CPU 功能与特性列表

参数	**典型值**

FP16 向量计算单元数	32*8=256
FP32 向量计算单元数	32*4=128
FP16 矩阵乘法计算单元数	32*32=1024
核内存储容量	32*8*256*16=1MB
SMC 核外数据访问接口的位宽	32*128=4Kb=512B
运行频率	FPGA 上 100MHz，N12 工艺下 1GHz
SMC 的数量	32

5.1.1 MID（宏指令译码单元）（待完善）

宏指令译码器的功能是收取或者获得宏指令，进行译码产生微指令和 LSU 指令。宏指令可以由指令译码器主动从片外或者片上存储空间读取，也可以由主控处理器发射到 MID。

宏指令可以是自定义的指令集，也可以是基于 RISC-V 指令集架构进行扩展的指令集。对于 AIACC V1，暂时不定义宏指令，保留对指令集生态的选择权。

一条宏指令启动一次数据流计算，一次数据流计算是指多个 SMC 中进行的批量数据从用户寄存器（User Register）读出，经过若干数据通道寄存器（Datapath Register）和若干计算、数据交换后写回到用户寄存器的过程。

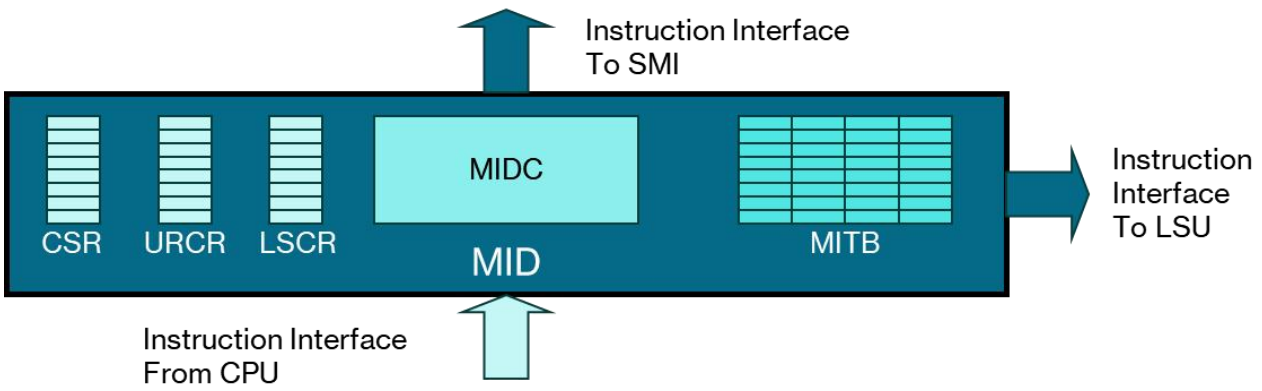


图 2 MID 架构示意图

5.1.2 SMI（流多处理单元的指令单元）

一条宏指令（不包含用于配置 CSR、URCR、LSCR、MITB 的宏指令）经过译码得到多列并行执行的微指令，这些多列指令存于多个指令队列（Instruction Queue，简称 IQ）中。

每一列存于一个指令队列的微指令，用于对该微指令队列对应 DR 或 URG（用户寄存器组，User Register Group，简称 URG，是指共用相同的控制逻辑的多个 UR 的集合）的相关的数据通道进行连续的控制，从而实现对对应 DR 和 UR 的更新；每个 IQ 具有一个对应的专属指令发射控制

器（Instruction Issue Controller，简称 IIC），IIC 读取 IQ 中的指令进行解析之后发射。不同的 IQ 由于指令格式未必完全相同，因此对应的 IIC 也是独特的。

需要注意的是，IQ 具有一定的深度，编译器需要将该深度作为编译的基础参数；指令中具有跳转指令以便实现循环，指令中具有参数自加功能，这两个特性是为了提升指令密度。IIC 进行指令译码的时候需要对这些功能进行支持。

另外需要注意的是，对于 UR 数据存取的指令与其他指令（称为计算指令）是相互异步的，如果架构设计中允许出现存取指令和计算指令对 UR 的竞争，则存取指令在数据可用的情况下与计算指令进行竞争仲裁，且存取指令的优先级低；如果架构设计中不允许出现存取指令和计算指令对 UR 的竞争，则存取指令在数据可用的情况下就可以发射。

下图是 SMI 架构的示意图。

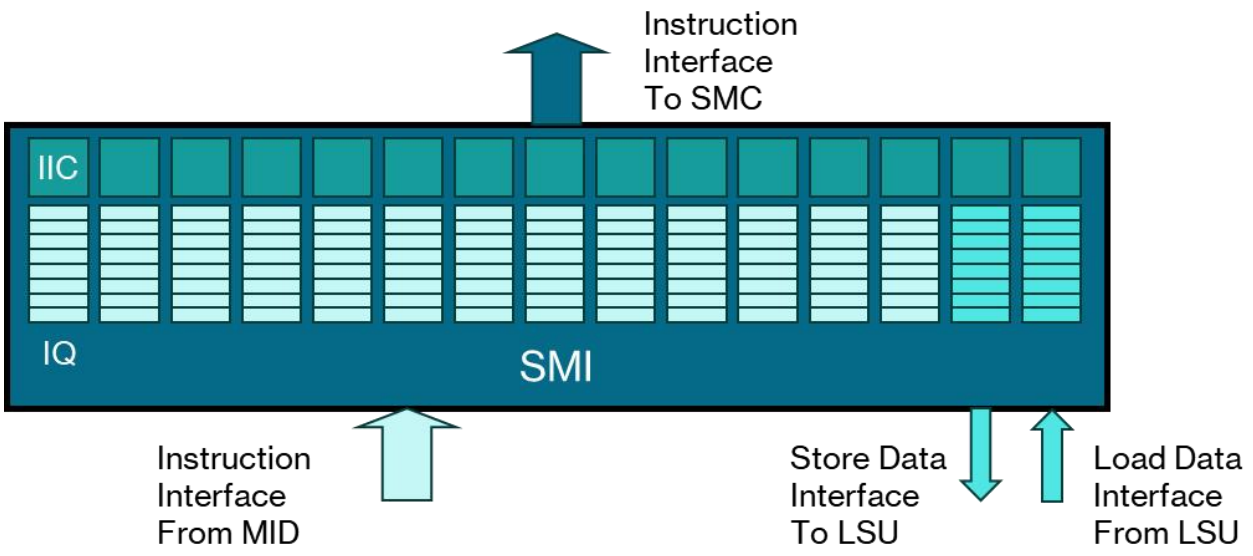


图 3 SMI 架构示意图

5.1.3 SMC（流多处理单元的计算单元）

一个流处理器单元包含若干个 UR（User Register）、若干个 DR（Datapath Register）和若干个 CR（Command Register），以及用于计算和数据交换的组合逻辑。更新每个 DR 或者 UR 的逻辑受到对应 CR 中的微指令控制；UR 和 DR 的数量与指令队列的数量一致。

下图是 SMC 设计思想的示意图。每个 UR 或者 DR 具有一个专属的寄存器更新逻辑单元（Register Update Unit，简称 RUU），RUU 的输入可以是本 SMC 中的任何一个 UR、任何一个

DR、与该 DR 和 RUU 紧密联系的 CR、南方相邻 SMC 或北方相邻 SMC 到本 SMC 的输入。如果一个 DR 对应的 RUU 的输入信号有来自南方相邻的 SMC，称之为 Upstream DR；如果一个 DR 对应的 RUU 的输入信号有来自北方相邻的 SMC，称之为 Downstream DR；如果没有来自南方相邻 SMC 和北方相邻 SMC 到本 SMC 的输入，称之为 Localstream DR；所有的 UR 都是 LocalStream UR。CR 分为 CRU（Command Register Upstream）和 CRD（Command Register Downstream），由南向北称之为 Upstream，由北向南称之为 Downstream，CRU 由南方相邻 SMC 的对应 CRU 和本 SMC 中的对应 CRUU（Command Register Update Unit）驱动更新；CRD 由北方相邻 SMC 的对应 CRD 和本 SMC 的 CRUU 驱动更新。

特殊的是最南侧的 SMC 中的 CRU 由 SMI 中的 command queue 和本 SMC 中的对应 CRUU 驱动更新；而 SMI 亦会接收来自最南侧的 SMC 中的 Downstream DR 和 CRD 所驱动的信号。

SMC 具体的架构参考本文“SMC 微架构和微指令”一节。

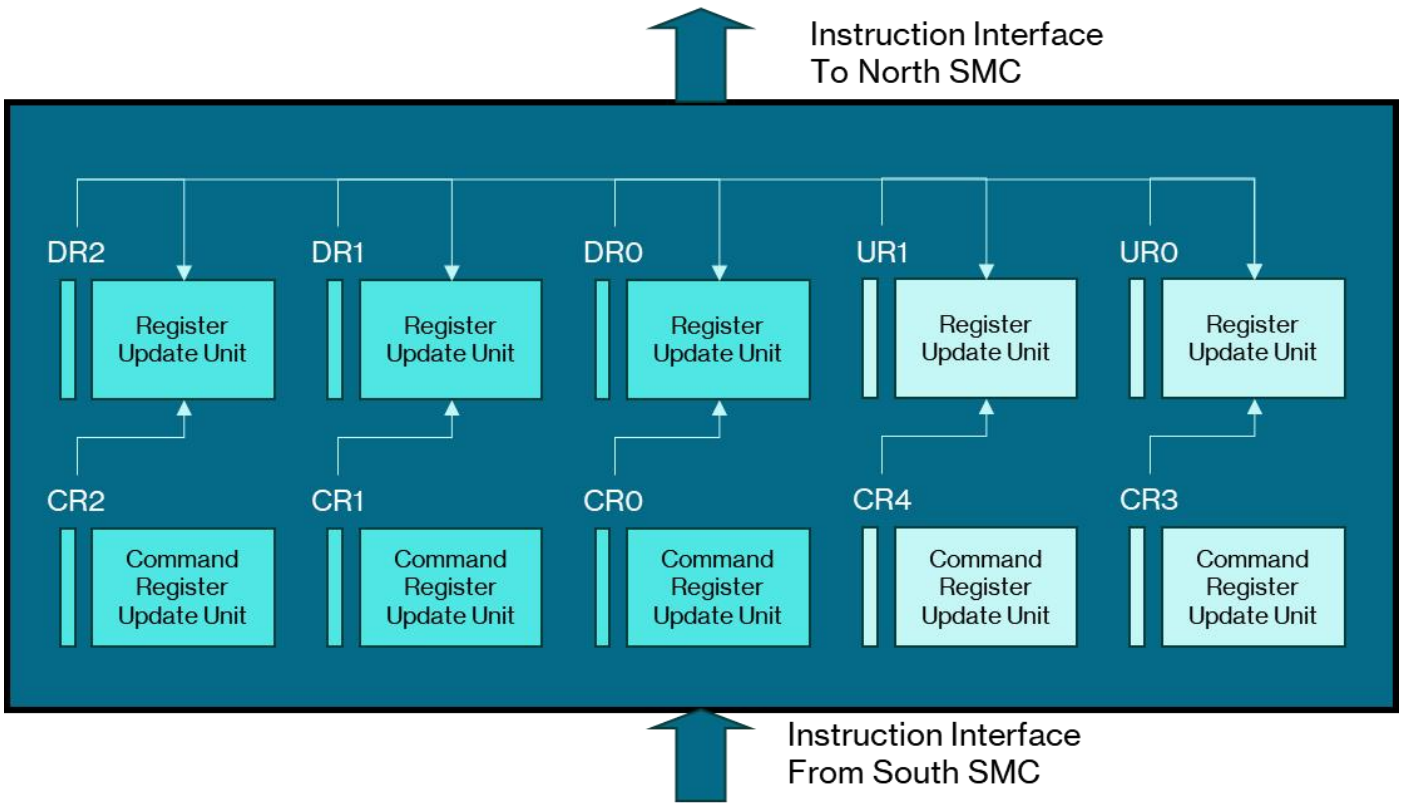


图 4 SMC 设计思想示意图

5.1.4 LSU（存取单元）（待完善）

LSU 的功能是将 UR 内部数据存储到 AIACC 外部和将 AIACC 外部数据读取到 UR 内部功能的

一部分。LSU 与 SMI 的分工为：SMI 处理的指令对应内部的 URG，SMI 与 LSU 的接口也是对应的 URG 的；而 LSU 处理的指令对应外部总线的接口（不同的接口一般对应不同的 interleave 地址空间），LSU 负责处理两个接口之间的数据映射。

它负责接收来自 MID 的指令，根据指令是存储还是读取以及端口编号送至相应的指令队列（读取指令队列，Load Instruction Queue，简称 LIQ；或者存储指令队列，Store Instruction Queue，简称 SIQ），之后由对应的控制器（读取控制器，Load Controller，简称 LC；或者存储控制器，Store Controller，简称 SC）执行指令队列中的指令，相关的数据缓存于数据队列中（读取数据队列，Load Data Queue，简称 LDQ；或者存储数据队列，Store Data Queue，简称 SDQ）。将一个 UR 和外部存储之间的数据存储或者读取称为一个存取数据流（Load & Store Data Stream，简称 LSDS），来自 MID 的指令中包含了这个 LSDS 是从哪一个 URG 到哪一个外部总线接口的映射关系，LC 或者 SC 负责解析指令并完成映射。

需要特殊注意的是，对于读取指令，外部总线返回数据存在乱序可能。对此有两种实现策略，一种是在 LSU 中进行数据重排，保证顺序正确；另外一种是设计 SMI 指令的时候就将一个 URG 指令队列，拆成多个对应不同 Interleave 地址空间的多个指令队列。

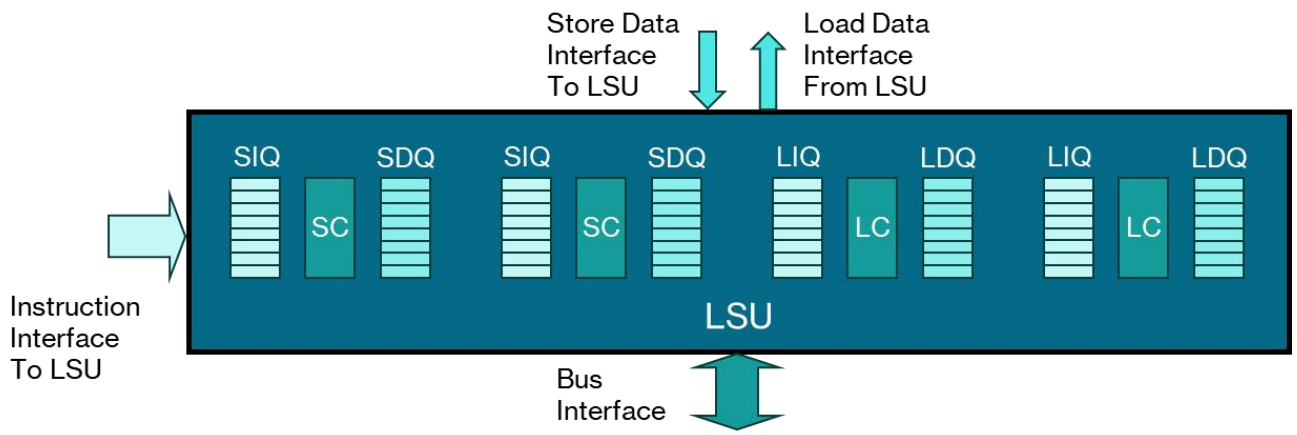


图 5 LSU 设计思想示意图

5.2 SMC 架构

5.2.1 SMC 接口

表 5- 2：SMC 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义

CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号

CRU_O	*	输出（寄存器）	为
			{
			CRU_INTADD_O,
			CRU_INT8toFP16_O,
			CRU_FP16MUL_S0123_O,
			CRU_FP16MUL_S4567_O,
			CRU_FP16MUL_O,
			CRU_FP32ADDTREE5TO1_O,
			CRU_FP32ADDTREE8TO1_O,
			CRU_FPMUL_O,
			CRU_FPADD_O,
			CRU_FPtoFP_O,
			CRU_INTtoINT_O,
			CRU_FPtoINT_O,
			CRU_INTtoFP_O,
			CRU_LOGICAL_O,
			CRU_SHIFTUP_O,
			CRU_SHIFTDN_O,
			CRU_STB_O,
			CRU_STBAXI_O,
			CRU_LDB_O,
			CRU_LDBAXI_O,
			CRU_UR_O,
			CRU_RtoVR_O,
			CRU_VIN0_O,
			CRU_VIN1_O,
			CRU_VOUT0_O,
			CRU_DS_O
			}

CRU	*	输入	为 { CRU_INTADD, CRU_INT8toFP16, CRU_FP16MUL_S0123, CRU_FP16MUL_S4567, CRU_FP16MUL, CRU_FP32ADDTREE5TO1, CRU_FP32ADDTREE8TO1, CRU_FPMUL, CRU_FPADD,

			CRU_FPtoFP, CRU_INTtoINT, CRU_FPtoINT, CRU_INTtoFP, CRU_LOGICAL, CRU_SHIFTUP, CRU_SHIFTDN, CRU_STB, CRU_STBAXI, CRU_LDB, CRU_LDBAXI, CRU_UR, CRU_RtoVR, CRU_VIN0, CRU_VIN1, CRU_VOUT0, CRU_DS }

CRD_O	*	输出（寄存器）	为
			{
			CRD_SHIFTDN_O,
			CRD_STB_O,
			CRD_STBAXI_O,
			CRD_LDB_O,
			CRD_LDBAXI_O,
			CRD_DS_O
			}

CRD	*	输入	为
			{
			CRD_SHIFTDN,
			CRD_STB,
			CRD_STBAXI,
			CRD_LDB,
			CRD_LDBAXI,
			CRD_DS
			}

AXI*	*		AXI 的 AW、W、B、AR、R 通道信号 数据位宽为 128bit、地址位宽 40bit
SMC_ID	5	输入	各个 SMC 的 SMC_ID 初始化输入端口
SMC_ID_WR	1	输入	各个 SMC 的 SMC_ID 初始化输入端口
SMC_ID_O	5	输出	各个 SMC 的 SMC_ID 初始化输出端口

SMC_ID_WR_O	1	输出	各个 SMC 的 SMC_ID 初始化输出端口

5.2.2 SMC 寄存器

5.2.2.1 DR

DR：Data Path Register

5.2.2.2 CRU

CRU：Control Register Upstream

所有的 CRU 的功能的共性部分

```
if SMC_ID==0 :
    SMC[SMC_ID]/CRU_* = func_pop_cmd_queue(*)
elif SMC_ID<=smc_id_en:
    SMC[SMC_ID]/CRU_* = SMC[SMC_ID-1]/CRU_*
```

5.2.2.3 CRD

CRD：Control Register Downstream

所有的 CRD 的功能的共性部分

```
if  SMC_ID==smc_id_en:
    SMC[SMC_ID]/CRD_* = SMC[SMC_ID]/CRU_*
if  SMC_ID<smc_id_en:
    SMC[SMC_ID-1]/CRD_* = SMC[SMC_ID]/CRD_*
```

5.2.2.4 UR

UR：User Register，物理实现为 SRAM

5.2.2.5 SMC_ID

每个 SMC 模块具有一个 SMC_ID 寄存器，保存本 SMC 的 ID。

if SMC_ID_WR==1 :

SMC[当前]/SMC_ID（寄存器） = SMC_ID（输入端口）

SMC[当前]/SMC_ID_O（输出端口） = SMC[当前]/SMC_ID（寄存器）

if SMC_ID_WR==1 :

SMC[当前]/SMC_ID_WR_O（输出端口）=1

else

SMC[当前]/SMC_ID_WR_O（输出端口）=0

5.2.3 SMC 相关的设计参数

表 5- 3：SMC 相关设计参数表

名称	含义	V1.0 支持
PARAM_SMC_NUM	SMC 的数量	32
PARAM_SMC_AXI_EN	控制 SMC 是否具有 AXI Master Interface ‘b0：无，SMC 不能直接访问外部地址空间 ‘b1：有，SMC 能直接访问外部地址空间	1

5.2.4 SMC 相关的功能说明

除非特殊说明，否则所有 SMC 寄存器复位后的默认值为 0。

5.3 SMC 微架构和微指令

5.3.1 INTADD（Integer Adder）

5.3.1.1 INTADD 的接口信号

表 5- 4：INTADD 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_INTADD_D0	128	输出（寄存器）	见寄存器说明
DR_INTADD_D1	128	输出（寄存器）	见寄存器说明
DR_INTADD_ST	128	输出（寄存器）	见寄存器说明
CRU_INTADD_O	11	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_INTADD_S0	128	输入	代表 INTADD 的计算输入数据 0
DVR_INTADD_S1	128	输入	代表 INTADD 的计算输入数据 1
DVR_INTADD_S2	128	输入	代表 INTADD 的计算输入数据 2
CRU_INTADD	11	输入	代表 INTADD 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

带格式表格[孙浩南]

5.3.1.2 INTADD 相关的寄存器

1) SMC[*]/DR_INTADD_D0：代表编号为*的 SMC 中的 INTADD 输出寄存器 0，128bit。

2) SMC[*]/DR_INTADD_D1：代表编号为*的 SMC 中的 INTADD 输出寄存器 1，128bit。

3) SMC[*]/DR_INTADD_ST：代表编号为*的 SMC 中的 INTADD 的 Status 寄存器，4 组*3bit=12bit，但是占位 128bit。这 12 个 bit 所处的位置和含义为：

   (1) Bit2、1、0 分别为第一组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (2) Bit34、33、32 分别为第二组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (3) Bit66、65、64 分别为第三组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (4) Bit98、97、96 分别为第四组 GT（Great than）、EQ（Equal）、LS（Less than）。

5.3.1.3 INTADD 相关的设计参数

表 5- 5：INTADD 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_INTADD_CNT	INTADD 加法器的数量	4 个 32bit 加法器或 32 个 8bit 加法器
PARAM_DR_INTADD_PRECISION	INTADD 的数据精度	整数 8bit 整数 32bit
PARAM_SUBWORD_PARALLELISM	是否支持子字并行	是

5.3.1.4 INTADD 相关的微指令

表 5- 6：INTADD 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_0_prec	2	按顺序排列	源寄存器 0 的精度 ‘b00：4bit ‘b01：8bit（目前不支持） ‘b10：16bit（目前不支持） ‘b11：32bit
src_1_prec	2	按顺序排列	源寄存器 1 的精度 ‘b00：4bit ‘b01：8bit（目前不支持） ‘b10：16bit（目前不支持） ‘b11：32bit
src_2_prec	2	按顺序排列	源寄存器 2 的精度 ‘b00：4bit ‘b01：8bit（目前不支持） ‘b10：16bit（目前不支持） ‘b11：32bit
src_0_sign	1	按顺序排列	源寄存器 0 符号 ‘b0：无符号数 ‘b1：有符号数 当该寄存器为 4bit 的时候，这个 field 只可以是’ b0
src_1_sign	1	按顺序排列	源寄存器 1 符号 ‘b0：无符号数 ‘b1：有符号数
src_2_sign	1	按顺序排列	源寄存器 2 符号 ‘b0：无符号数 ‘b1：有符号数
dst_sign	1	按顺序排列	目的寄存器符号 ‘b0：无符号数 ‘b1：有符号数

up_stat_en	1	按顺序排列	是否更新状态寄存器 'b0：不更新 'b1：更新
add_or_sub	1	按顺序排列	加减法 'b0：加法 'b1：减法
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.1.5 INTADD 功能说明

1）当做 4bit+8bit 的时候，支持 32 个 8bit 加法器

源操作数是一个 8bit 数（S1 和 S0 中各包含 16 个 8bit 数据）和一个 4bit 数（S2，符号扩展至 8bit），结果存为 8bit。

源寄存器 0，分为 16 组，每组 8bit，记为 DVR_INTADD_S0[y]，y=0,1,...15

源寄存器 1，分为 16 组，每组 8bit，记为 DVR_INTADD_S1[y]，y=0,1,...15

源寄存器 2，分为 32 组，每组 4bit，记为 DVR_INTADD_S2[y]，y=0,1,...31

目的寄存器 0，分为 16 组，每组 8bit，记为 DR_INTADD_D0[x]，x=0,1,...15

目的寄存器 1，分为 16 组，每组 8bit，记为 DR_INTADD_D1[x]，x=0,1,...15

SMC[*]/DR_INTADD_D1[x] = SMC[*]/DVR_INTADD_S2[x+16].astype(numpy.int8) + SMC[*]/DVR_INTADD_S1[x]

SMC[*]/DR_INTADD_D0[x] = SMC[*]/DVR_INTADD_S2[x].astype(numpy.uint8) + SMC[*]/DVR_INTADD_S0[x]

不更新 SMC[*]/DR_INTADD_ST

2）当做 32bit+32bit 的时候，支持 4 个 32bit 加法器

源寄存器 0，分为 4 组，每组 32bit，记为 DVR_INTADD_S0[y]，y=0,1,...3

源寄存器 1，分为 4 组，每组 32bit，记为 DVR_INTADD_S1[y]，y=0,1,...3

目的寄存器 0，分为 4 组，每组 32bit，记为 DR_INTADD_D0[x]，x=0,1,...3

SMC[*]/DR_INTADD_D0[x] = SMC[*]/DVR_INTADD_S0[x] + SMC[*]/DVR_INTADD_S1[x]

同时更新 SMC[*]/DR_INTADD_ST

3）其他的组合在 V1.0 版本不支持

5.3.2 INT8toFP16（Integer To Float Convert）

5.3.2.1 INT8toFP16 的接口信号

表 5- 7：INT8toFP16 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_INT8toFP16_D0	128	输出（寄存器）	见寄存器说明
DR_INT8toFP16_D1	128	输出（寄存器）	见寄存器说明
DR_INT8toFP16_D2	128	输出（寄存器）	见寄存器说明
DR_INT8toFP16_D3	128	输出（寄存器）	见寄存器说明
CRU_INT8toFP16_O	2	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_INT8toFP16_S0	128	输入	代表 INT8toFP16 的计算输入数据 0
DVR_INT8toFP16_S1	128	输入	代表 INT8toFP16 的计算输入数据 1
CRU_INT8toFP16	2	输入	代表 INT8toFP16 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.2.2 INT8toFP16 相关的寄存器

1）SMC[*]/DR_INT8toFP16_D0：代表编号为*的 SMC 中的 INT8toFP16 输出寄存器 0，128bit。

2）SMC[*]/DR_INT8toFP16_D1：代表编号为*的 SMC 中的 INT8toFP16 输出寄存器 1，128bit。

3）SMC[*]/DR_INT8toFP16_D2：代表编号为*的 SMC 中的 INT8toFP16 输出寄存器 2，128bit。

4）SMC[*]/DR_INT8toFP16_D3：代表编号为*的 SMC 中的 INT8toFP16 输出寄存器 3，128bit。

5.3.2.3 INT8toFP16 相关的设计参数

表 5- 8：INT8toFP16 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_INT8toFP16_CNT	INT8toFP16 的数量，基于最大数据精度	32 个 INT8toFP16

5.3.2.4 INT8toFP16 相关的微指令

表 5- 9：INT8toFP16 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_sign	1	按顺序排列	源寄存器符号 ‘b0：无符号数 ‘b1：有符号数
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.2.5 INT8toFP16 功能说明

支持 32 个 INT8toFP16 的时候，

源寄存器 0，分为 16 组，每组 8bit，记为 SMC[*]/DVR_INT8toFP16_S0[y]，y=0,1,...15

源寄存器 1，分为 16 组，每组 8bit，记为 SMC[*]/DVR_INT8toFP16_S1[y]，y=0,1,...15

目的寄存器 0，分为 8 组，每组 16bit，记为 SMC[*]/DR_INT8toFP16_D0[x]，x=0,1,...7

目的寄存器 1，分为 8 组，每组 16bit，记为 SMC[*]/DR_INT8toFP16_D1[x]，x=0,1,...7

目的寄存器 2，分为 8 组，每组 16bit，记为 SMC[*]/DR_INT8toFP16_D2[x]，x=0,1,...7

目的寄存器 3，分为 8 组，每组 16bit，记为 SMC[*]/DR_INT8toFP16_D3[x]，x=0,1,...7

SMC[*]/DR_INT8toFP16_D3[x]=SMC[*]/DVR_INT8toFP16_S1[x+8].astype(numpy.float16)

SMC[*]/DR_INT8toFP16_D2[x]=SMC[*]/DVR_INT8toFP16_S1[x].astype(numpy.float16)

SMC[*]/DR_INT8toFP16_D1[x]=SMC[*]/DVR_INT8toFP16_S0[x+8].astype(numpy.float16)

SMC[*]/DR_INT8toFP16_D0[x]=SMC[*]/DVR_INT8toFP16_S0[x].astype(numpy.float16)

5.3.3 FP16MUL（Float16 Multiply)

5.3.3.1 FP16MUL 的接口信号

表 5- 10：FP16MUL 的接口信号表

名称	Bit	输入 / 输出（寄存器）	含义

	数		
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_FP16MUL_D0	128	输出（寄存器）	见寄存器说明
DR_FP16MUL_D1	128	输出（寄存器）	见寄存器说明
DR_FP16MUL_D2	128	输出（寄存器）	见寄存器说明
DR_FP16MUL_D3	128	输出（寄存器）	见寄存器说明
DR_FP16MUL_D4	128	输出（寄存器）	见寄存器说明
DR_FP16MUL_D5	128	输出（寄存器）	见寄存器说明
DR_FP16MUL_D6	128	输出（寄存器）	见寄存器说明
DR_FP16MUL_D7	128	输出（寄存器）	见寄存器说明
CRU_FP16MUL_O	3	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_FP16MUL_S0	128	输入	将写入 DVR_FP16MUL_S0 的输入数据
DVR_FP16MUL_S1	128	输入	将写入 DVR_FP16MUL_S1 的输入数据
DVR_FP16MUL_S2	128	输入	将写入 DVR_FP16MUL_S2 的输入数据
DVR_FP16MUL_S3	128	输入	将写入 DVR_FP16MUL_S3 的输入数据
DVR_FP16MUL_S4567	128	输入	将写入 DVR_FP16MUL_S4567 的输入数据
CRU_FP16MUL_S0123	2	输入	代表编号为*的 SMC 中的 FP16MUL S0123 中控制数据写入的上行指令寄存器，存储内容为下面介绍的微指令。
CRU_FP16MUL_S4567	4	输入	代表编号为*的 SMC 中的 FP16MUL S0123 中控制数据写入的上行指令寄存器，存储内容为下面介绍的微指令。
CRU_FP16MUL	3	输入	代表编号为*的 SMC 中的 FP16MUL 中控制乘法计算的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.3.2 FP16MUL 相关的寄存器

1）SMC[*]/DR_FP16MUL_D0：代表编号为*的 SMC 中的 FP16MUL 计算的输出寄存器 0，128bit。

2）SMC[*]/DR_FP16MUL_D1：代表编号为*的 SMC 中的 FP16MUL 计算的输出寄存器 1，128bit。

3）SMC[*]/DR_FP16MUL_D2：代表编号为*的 SMC 中的 FP16MUL 计算的输出寄存器 2，128bit。

4）SMC[*]/DR_FP16MUL_D3：代表编号为*的 SMC 中的 FP16MUL 计算的输出寄存器 3，128bit。

5）SMC[*]/DR_FP16MUL_D4：代表编号为*的 SMC 中的 FP16MUL 计算的输出寄存器 4，128bit。

6）SMC[*]/DR_FP16MUL_D5：代表编号为*的 SMC 中的 FP16MUL 计算的输出寄存器 5，128bit。

7）SMC[*]/DR_FP16MUL_D6：代表编号为*的 SMC 中的 FP16MUL 计算的输出寄存器 6，128bit。

8）SMC[*]/DR_FP16MUL_D7：代表编号为*的 SMC 中的 FP16MUL 计算的输出寄存器 7，128bit。

9）SMC[*]/DR_FP16MUL_S0_*：代表编号为*的 SMC 中的 FP16MUL 计算输入的一个源数据，它来自广播；*代表数字，可以支持多组寄存器，128bit。

10）SMC[*]/DR_FP16MUL_S1_*：代表编号为*的 SMC 中的 FP16MUL 计算输入的一个源数据，它来自广播；*代表数字，可以支持多组寄存器，128bit。

11）SMC[*]/DR_FP16MUL_S2_*：代表编号为*的 SMC 中的 FP16MUL 计算输入的一个源数据，它来自广播；*代表数字，可以支持多组寄存器，128bit。

12）SMC[*]/DR_FP16MUL_S3_*：代表编号为*的 SMC 中的 FP16MUL 计算输入的一个源数据，它来自广播；*代表数字，可以支持多组寄存器，128bit。

13）SMC[*]/DR_FP16MUL_S4_*：代表编号为*的 SMC 中的 FP16MUL 计算输入的另一个源数据，它来自本地；*代表数字，可以支持多组寄存器，128bit。

14）SMC[*]/DR_FP16MUL_S5_*：代表编号为*的 SMC 中的 FP16MUL 计算输入的另一个源数据，它来自本地；*代表数字，可以支持多组寄存器，128bit。

15）SMC[*]/DR_FP16MUL_S6_*：代表编号为*的 SMC 中的 FP16MUL 计算输入的另一个源数据，它来自本地；*代表数字，可以支持多组寄存器，128bit。

16）SMC[*]/DR_FP16MUL_S7_*：代表编号为*的 SMC 中的 FP16MUL 计算输入的另一个源数据，它来自本地；*代表数字，可以支持多组寄存器，128bit。

5.3.3.3 FP16MUL 相关的设计参数

表 5- 11：FP16MUL 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_FP16MUL_CNT	FP16MUL 乘法器的数量	32
PARAM_DR_FP16MUL_S0_CNT	DR_FP16MUL_S0、S1、S2、S3 支持的寄存器数量	2
PARAM_DR_FP16MUL_S1_CNT	DR_FP16MUL_S4、S5、S6、S7 支持的寄存器数量	2

5.3.3.4 FP16MUL 相关的微指令

表 5- 12：FP16MUL 相关上行微指令 CRU_FP16MUL 表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_0_id	1	按顺序排列	0 源寄存器编码 最多支持 2 个 0 源寄存器
src_1_id	1	按顺序排列	1 源寄存器编码 最多支持 2 个 1 源寄存器
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

表 5- 13：FP16MUL 相关上行微指令 CRU_FP16MUL_S0123 表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src3210_high_id	1	按顺序排列	寄存器编码高位 {S3,S2,S1,S0}最多支持 2 个寄存器为{S3,S2,S1,S0}_0 和 {S3,S2,S1,S0}_1
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

表 5- 14：FP16MUL 相关上行微指令 CRU_FP16MUL_S4567 表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src7654_high_id	1	按顺序排列	寄存器编码高位 S4567 每个又具有 2 个寄存器
src7654_low_id	2	按顺序排列	寄存器编码低位 S4567 具有 4 个寄存器 2 ‘b00 为 4 2 ‘b 01 为 5 2 ‘b 10 为 6 2 ‘b 11 为 7

smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.3.5 FP16MUL 功能说明

1） SMC[*]/DR_FP16MUL_S0123 更新逻辑

如果 CRU_FP16MUL_S0123 有效

SMC[*]/DR_FP16MUL_S0_[寄存器编码高位] = DVR_FP16MUL_S0

SMC[*]/DR_FP16MUL_S1_[寄存器编码高位] = DVR_FP16MUL_S1

SMC[*]/DR_FP16MUL_S2_[寄存器编码高位] = DVR_FP16MUL_S2

SMC[*]/DR_FP16MUL_S3_[寄存器编码高位] = DVR_FP16MUL_S3

2） SMC[*]/DR_FP16MUL_S4567 更新逻辑

如果 CRU_FP16MUL_S4567 有效

SMC[*]/DR_FP16MUL_S[寄存器编码低位]_[寄存器编码高位] = DVR_FP16MUL_S4567

3） SMC[*]/DR_FP16MUL_D*更新逻辑

每个源寄存器都分为 8 组，每组 16bit，记为

SMC[*]/DR_FP16MUL_S*_*[y]，y=0,1,..7

每个目的寄存器都分为 4 组，每组 32bit，记为

SMC[*]/DR_FP16MUL_D*[x]，x=0,1,...3

SMC[*]/DR_FP16MUL_D7[x]= numpy.multiply(

SMC[*]/DR_FP16MUL_S3_*[x+4], SMC[*]/DR_FP16MUL_S7_*[x+4])

SMC[*]/DR_FP16MUL_D6[x]= numpy.multiply(

SMC[*]/DR_FP16MUL_S3_*[x], SMC[*]/DR_FP16MUL_S7_*[x])

SMC[*]/DR_FP16MUL_D5[x]= numpy.multiply(

SMC[*]/DR_FP16MUL_S2_*[x+4], SMC[*]/DR_FP16MUL_S6_*[x+4])

SMC[*]/DR_FP16MUL_D4[x]= numpy.multiply(

SMC[*]/DR_FP16MUL_S2_*[x], SMC[*]/DR_FP16MUL_S6_*[x])

SMC[*]/DR_FP16MUL_D3[x]= numpy.multiply(

SMC[*]/DR_FP16MUL_S1_*[x+4], SMC[*]/DR_FP16MUL_S5_*[x+4])

SMC[*]/DR_FP16MUL_D2[x]= numpy.multiply(

SMC[*]/DR_FP16MUL_S1_*[x], SMC[*]/DR_FP16MUL_S5_*[x])

SMC[*]/DR_FP16MUL_D1[x]= numpy.multiply(

SMC[*]/DR_FP16MUL_S0_*[x+4], SMC[*]/DR_FP16MUL_S4_*[x+4])

SMC[*]/DR_FP16MUL_D0[x]= numpy.multiply(

SMC[*]/DR_FP16MUL_S0_*[x], SMC[*]/DR_FP16MUL_S4_*[x])

5.3.4 FP32ADDTREE5TO1（Float32 Adder Tree 5to1）

5.3.4.1 FP32ADDTREE5TO1 的接口信号

表 5- 15：FP32ADDRTREE5TO1 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_FP32ADDTREE5TO1_D0	128	输出（寄存器）	见寄存器说明
DR_FP32ADDTREE5TO1_D1	128	输出（寄存器）	见寄存器说明
CRU_FP32ADDTREE5TO1_O	1	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_FP32ADDTREE5TO1_S0	128	输入	代表 FP32ADDTREE5TO1 的计算输入数据 0
DVR_FP32ADDTREE5TO1_S1	128	输入	代表 FP32ADDTREE5TO1 的计算输入数据 1
DVR_FP32ADDTREE5TO1_S2	128	输入	代表 FP32ADDTREE5TO1 的计算输入数据 2
DVR_FP32ADDTREE5TO1_S3	128	输入	代表 FP32ADDTREE5TO1 的计算输入数据 3

DVR_FP32ADDTREE5TO1_S4	128	输入	代表 FP32ADDTREE5TO1 的计算输入数据 4
DVR_FP32ADDTREE5TO1_S5	128	输入	代表 FP32ADDTREE5TO1 的计算输入数据 5
DVR_FP32ADDTREE5TO1_S6	128	输入	代表 FP32ADDTREE5TO1 的计算输入数据 6
DVR_FP32ADDTREE5TO1_S7	128	输入	代表 FP32ADDTREE5TO1 的计算输入数据 7
CRU_FP32ADDTREE5to1	1	输入	代表 FP32ADDTREE5TO1 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.4.2 FP32ADDTREE5TO1 相关的寄存器

1）SMC[*]/DR_FP32ADDTREE5TO1_D0：代表 FP32ADDTREE5TO1 计算的输出寄存器 0，128bit。

2）SMC[*]/DR_FP32ADDTREE5TO1_D1：代表 FP32ADDTREE5TO1 计算的输出寄存器 1，128bit。

5.3.4.3 FP32ADDTREE5TO1 相关的设计参数

表 5- 16：FP32ADDTREE5TO1 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_FP32ADDTREE5TO1_CNT	FP32ADDTREE5TO1 的数量	8
PARAM_DR_FP32ADDTREE5TO1_SCNT	FP32ADDTREE5TO1 的输入寄存器数量	5

5.3.4.4 FP32ADDTREE5TO1 相关的微指令

表 5- 17：FP32ADDTREE5TO1 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	vld ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
smc_id_en	5		smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.4.5 FP32ADDTREE5TO1 功能说明

每个源寄存器都分为 4 组，每组 32bit，记为

SMC[*]/DVR_FP32ADDTREE5TO1_S*[y]，y=0,1,...3

共有 32 个 32bit 源寄存器

每个目的寄存器都分为 4 组，每组 32bit，记为

SMC[*]/DR_FP32ADDTREE5TO1_D*[x]，x=0,1,...3

共有 8 个 32bit 目的寄存器

本模块的功能是将 32 个 32bit 源寄存器分为 8 个，每组 4 个。每组内的 4 个 32bit 寄存器与一个目的 32bit 寄存器做加法树计算，得到一个目的 32bit 寄存器。

SMC[*]/DR_FP32ADDTREE5TO1_D0[0] = numpy.sum(

SMC[*]/DR_FP32ADDTREE5TO1_D0[0], DVR_FP32ADDTREE5TO1_S0[31:0],

DVR_FP32ADDTREE5TO1_S0[63:32], DVR_FP32ADDTREE5TO1_S0[95:64],

DVR_FP32ADDTREE5TO1_S0[127:96])

SMC[*]/DR_FP32ADDTREE5TO1_D0[1] = numpy.sum(

SMC[*]/DR_FP32ADDTREE5TO1_D0[1], DVR_FP32ADDTREE5TO1_S1[31:0],

DVR_FP32ADDTREE5TO1_S1[63:32], DVR_FP32ADDTREE5TO1_S1[95:64],

DVR_FP32ADDTREE5TO1_S1[127:96])

SMC[*]/DR_FP32ADDTREE5TO1_D0[2] = numpy.sum(

SMC[*]/DR_FP32ADDTREE5TO1_D0[2], DVR_FP32ADDTREE5TO1_S2[31:0],

DVR_FP32ADDTREE5TO1_S2[63:32], DVR_FP32ADDTREE5TO1_S2[95:64],

DVR_FP32ADDTREE5TO1_S2[127:96])

SMC[*]/DR_FP32ADDTREE5TO1_D0[3] = numpy.sum(

SMC[*]/DR_FP32ADDTREE5TO1_D0[3], DVR_FP32ADDTREE5TO1_S3[31:0],

DVR_FP32ADDTREE5TO1_S3[63:32], DVR_FP32ADDTREE5TO1_S3[95:64],

DVR_FP32ADDTREE5TO1_S3[127:96])

SMC[*]/DR_FP32ADDTREE5TO1_D1[0] = numpy.sum(

SMC[*]/DR_FP32ADDTREE5TO1_D1[0], DVR_FP32ADDTREE5TO1_S4[31:0],

DVR_FP32ADDTREE5TO1_S4[63:32], DVR_FP32ADDTREE5TO1_S4[95:64],

DVR_FP32ADDTREE5TO1_S4[127:96])

SMC[*]/DR_FP32ADDTREE5TO1_D1[1] = numpy.sum(

SMC[*]/DR_FP32ADDTREE5TO1_D1[1], DVR_FP32ADDTREE5TO1_S5[31:0],

DVR_FP32ADDTREE5TO1_S5[63:32], DVR_FP32ADDTREE5TO1_S5[95:64],

DVR_FP32ADDTREE5TO1_S5[127:96])

SMC[*]/DR_FP32ADDTREE5TO1_D1[2] = numpy.sum(

SMC[*]/DR_FP32ADDTREE5TO1_D1[2], DVR_FP32ADDTREE5TO1_S6[31:0],

DVR_FP32ADDTREE5TO1_S6[63:32], DVR_FP32ADDTREE5TO1_S6[95:64],

DVR_FP32ADDTREE5TO1_S6[127:96])

SMC[*]/DR_FP32ADDTREE5TO1_D1[3] = numpy.sum(

SMC[*]/DR_FP32ADDTREE5TO1_D1[3], DVR_FP32ADDTREE5TO1_S7[31:0],

DVR_FP32ADDTREE5TO1_S7[63:32], DVR_FP32ADDTREE5TO1_S7[95:64],

DVR_FP32ADDTREE5TO1_S7[127:96])

5.3.5 FP32ADDTREE8TO1（Float32 Adder Tree 8to1）

5.3.5.1 FP32ADDTREE8TO1 的接口信号

表 5- 18：FP32ADDTREE8TO1 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_FP32ADDTREE8TO1_D	128	输出（寄存器）	见寄存器说明
CRU_FP32ADDTREE8TO1_O	3	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_FP32ADDTREE8TO1_S0	128	输入	代表 FP32ADDTREE8TO1 的计算输入数据 0
DVR_FP32ADDTREE8TO1_S1	128	输入	代表 FP32ADDTREE8TO1 的计算输入数据 1
CRU_FP32ADDTREE8to1	3	输入	代表 FP32ADDTREE8TO1 的上行指令寄存器，存储内容为下面介绍的微指

			令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.5.2 FP32ADDTREE8TO1 相关的寄存器

1） SMC[*]/DR_FP32ADDTREE8TO1_D：代表编号为*的 SMC 中的 FP32ADDTREE8TO1 计算的输出寄存器，128bit。

5.3.5.3 FP32ADDTREE8TO1 相关的设计参数

表 5- 19：FP32ADDTREE8TO1 相关设计参数表

名称	**含义**	**V1.0 支持**
PARAM_DR_FP32ADDTREE8TO1_CNT	FP32ADDTREE8TO1 的数量	1
PARAM_DR_FP32ADDTREE8TO1_SCNT	FP32ADDTREE8TO1 的输入寄存器数量	8

5.3.5.4 FP32ADDTREE8TO1 相关的微指令

表 5- 20：FP32ADDTREE 相关上行微指令表

名称	**Bit 数**	**Bit 位**	**含义**
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
dst_id	2	按顺序排列	目的寄存器编码 每次加法结果存于 128bit 中的 32bit，最多存 4 个
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.5.5 FP32ADDTREE8TO1 功能说明

每个源寄存器都分为 4 组，每组 32bit，记为

SMC[*]/DVR_FP32ADDTREE8TO1_S*[y]，y=0,1,...3

目的寄存器都分为 4 组，每组 32bit，记为

SMC[*]/DR_FP32ADDTREE8TO1_D[x]，x=0,1,...7

SMC[*]/DR_FP32ADDTREE8TO1_D[x] = numpy.sum(

SMC[*]/DVR_FP32ADDTREE8TO1_S0[0],

SMC[*]/DVR_FP32ADDTREE8TO1_S0[1],

SMC[*]/DVR_FP32ADDTREE8TO1_S0[2],

SMC[*]/DVR_FP32ADDTREE8TO1_S0[3]

SMC[*]/DVR_FP32ADDTREE8TO1_S1[0],

SMC[*]/DVR_FP32ADDTREE8TO1_S1[1],

SMC[*]/DVR_FP32ADDTREE8TO1_S1[2],

SMC[*]/DVR_FP32ADDTREE8TO1_S1[3])

其中 x 为目的寄存器编码

删除[SYT]: 0

5.3.6 FPMUL（Float Multiply）

5.3.6.1 FPMUL 的接口信号

表 5- 21：FPMUL 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_FPMUL_D	128	输出（寄存器）	见寄存器说明
CRU_FPMUL_O	3	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_FPMUL_S0	128	输入	代表 FPMUL 的计算输入数据 0
DVR_FPMUL_S1	128	输入	代表 FPMUL 的计算输入数据 1
CRU_FPMUL	3	输入	代表 FPMUL 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.6.2 FPMUL 相关的寄存器

1) SMC[*]/DR_FPMUL_D：代表编号为*的 SMC 中的 FPMUL 计算的输出寄存器，128bit。

5.3.6.3 FPMUL 相关的设计参数

表 5- 22：FPMUL 相关设计参数表

名称	含义	V1.0 支持

PARAM_DR_FPMUL_CNT	FPMUL 乘法器的数量，基于最高数据精度	4
PARAM_DR_FPMUL_PRECISION	FPMUL 的数据精度	16bit 32bit
PARAM_SUBWORD_PARALLELISM	是否支持子字并行	是
PARAM_DIFF_PRECISION_SS	是否支持源源之间是否数据精度不同	不支持
PARAM_DIFF_PRECISION_SD	是否支持源目的之间是否数据精度不同	不支持

5.3.6.4 FPMUL 相关的微指令

表 5- 23：FPMUL 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_prec	1	按顺序排列	源寄存器的精度 ‘b0：16bit ‘b1：32bit
dst_prec	1	按顺序排列	目的寄存器的精度 ‘b0：16bit ‘b1：32bit
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.6.5 FPMUL 功能说明

1）当目的寄存为 32bit、 源寄存器为 32bit 的时候，支持 4 个 FPMUL

源寄存器 0，分为 4 组，每组 32bit，记为 SMC[*]/DVR_FPMUL_S0[y]，y=0,1,...3

源寄存器 1，分为 4 组，每组 32bit，记为 SMC[*]/DVR_FPMUL_S1[y]，y=0,1,...3

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_FPMUL_D[x]，x=0,1,...3

SMC[*]/DR_FPMUL_D[x] = numpy.multiply(SMC[*]/DVR_FPMUL_S0[x], SMC[*]/DVR_FPMUL_S1[x])

2）当目的寄存器为 16bit、源寄存为 16bit 的时候，支持 8 个 FPMUL

源寄存器 0，分为 8 组，每组 16bit，记为 SMC[*]/DVR_FPMUL_S0[y]，y=0,1,...7

源寄存器 1，分为 8 组，每组 16bit，记为 SMC[*]/DVR_FPMUL_S1[y]，y=0,1,...7

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_FPMUL_D[x]，x=0,1,...7

SMC[*]/DR_FPMUL_D[x] = numpy.multiply(SMC[*]/DVR_FPMUL_S0[x], SMC[*]/DVR_FPMUL_S1[x])

5.3.7 FPADD（Float Adder）

5.3.7.1 FPADD 的接口信号

表 5- 24：FPADD 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_FPADD_D	128	输出（寄存器）	见寄存器说明
DR_FPADD_ST	128	输出（寄存器）	见寄存器说明
CRU_FPADD_O	4	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_FPADD_S0	128	输入	代表 FPADD 的计算输入数据 0
DVR_FPADD_S1	128	输入	代表 FPADD 的计算输入数据 1
CRU_FPADD	4	输入	代表 FPADD 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.7.2 FPADD 相关的寄存器

1) SMC[*]/DR_FPADD_D：代表编号为*的 SMC 中的 FPADD 计算的输出寄存器，128bit。

2) SMC[*]/DR_FPADD_ST：代表编号为*的 SMC 中的 FPADD 的 Status 寄存器，8 组*3bit=24bit，但是占位 128bit。这 12 个 bit 所处的位置和含义为：

   (1) Bit2、1、0 分别为第一组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (2) Bit18、17、16 分别为第二组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (3) Bit34、33、32 分别为第三组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (4) Bit50、49、48 分别为第四组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (5) Bit66、65、64 分别为第五组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (6) Bit82、81、80 分别为第六组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (7) Bit98、97、96 分别为第七组 GT（Great than）、EQ（Equal）、LS（Less than）。

   (8) Bit114、113、112 分别为第八组 GT（Great than）、EQ（Equal）、LS（Less than）。

5.3.7.3 FPADD 相关的设计参数

表 5- 25：FPADD 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_FPADD_CNT	FPADD 加法器的数量，基于最高数据精度	4
PARAM_DR_FPADD_PRECISION	FPMUL 的数据精度	16bit 32bit
PARAM_SUBWORD_PARALLELISM	是否支持子字并行	是
PARAM_DIFF_PRECISION_SS	是否支持源源之间是否数据精度不同	不支持
PARAM_DIFF_PRECISION_SD	是否支持源目的之间是否数据精度不同	不支持

5.3.7.4 FPADD 相关的微指令

表 5- 26：FPADD 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_prec	1	按顺序排列	源寄存器的精度 ‘b0：16bit ‘b1：32bit
dst_prec	1	按顺序排列	目的寄存器的精度 ‘b0：16bit ‘b1：32bit
up_stat_en	1	按顺序排列	是否更新状态寄存器 ‘b0：不更新 ‘b1：更新
add_or_sub	1	按顺序排列	加减法 ‘b0：加法 ‘b1：减法
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.7.5 FPADD 功能说明

1）当目的寄存为 32bit、 源寄存器为 32bit 的时候，支持 4 个 FPADD

源寄存器 0，分为 4 组，每组 32bit，记为 SMC[*]/DVR_FPADD_S0[y]，y=0,1,...3

源寄存器 1，分为 4 组，每组 32bit，记为 SMC[*]/DVR_FPADD_S1[y]，y=0,1,...3

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_FPADD_D[x]，x=0,1,...3

SMC[*]/DR_FPADD_D[x] = numpy.add(SMC[*]/DVR_FPADD_S0[x], SMC[*]/DVR_FPADD_S1[x])

同时更新 DR_INTADD_ST 中的第一三五七组

2）当目的寄存器为 16bit、源寄存为 16bit 的时候，支持 8 个 FPADD

源寄存器 0，分为 8 组，每组 16bit，记为 SMC[*]/DVR_FPADD_S0[y]，y=0,1,...7

源寄存器 1，分为 8 组，每组 16bit，记为 SMC[*]/DVR_FPADD_S1[y]，y=0,1,...7

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_FPADD_D[x]，x=0,1,...7

SMC[*]/DR_FPADD_D[x] = numpy.add(SMC[*]/DVR_FPADD_S0[x], SMC[*]/DVR_FPADD_S1[x])

同时更新 SMC[*]/DR_FPADD_ST 中的所有组信号

5.3.8 FPtoFP（Float To Float Convert)

5.3.8.1 FPtoFP 的接口信号

表 5- 27：FPtoFP 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_FPtoFP_D	128	输出（寄存器）	见寄存器说明
CRU_FPtoFP_O	5	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_FPtoFP_S	128	输入	代表 FPtoFP 的计算输入数据
CRU_FPtoFP	5	输入	代表 FPtoFP 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.8.2 FPtoFP 相关的寄存器

1) SMC[*]/DR_FPtoFP_D：代表编号为*的 SMC 中的 FPtoFP 计算的输出寄存器，128bit。

5.3.8.3 FPtoFP 相关的设计参数

表 5- 28：FPtoFP 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_FPtoFP_CNT	FPtoFP 的数量	4

PARAM_DR_FPtoFP_PRECISION	FPtoFP 的数据精度	16bit 32bit
PARAM_SUBWORD_PARALLELISM	是否支持子字并行	是
PARAM_DIFF_PRECISION_SD	是否支持源目的之间是否数据精度不同	支持

5.3.8.4 FPtoFP 相关的微指令

表 5- 29：FPtoFP 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 'b1：指令有效 'b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_prec	1	按顺序排列	源寄存器的精度 'b0：16bit 'b1：32bit
dst_prec	1	按顺序排列	目的寄存器的精度 'b0：16bit 'b1：32bit
src_high_or_low	1	按顺序排列	源数据位置 如果源寄存器精度非全精度，标志数据所在位置： 'b0：低位 'b1：高位
dst_high_or_low	1	按顺序排列	目的数据位置 如果源寄存器精度非全精度，标志数据所在位置： 'b0：低位 'b1：高位
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.8.5 FPtoFP 功能说明

1）当目的寄存为 32bit、 源寄存器为 16bit 的时候，支持 4 个 FPtoFP

源寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DVR_FPtoFP_S[y]，y=0,1,...7

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_FPtoFP_D[x]，x=0,1,...3

根据源数据位置

SMC[*]/DR_FPtoFP_D[x] = SMC[*]/DVR_FPtoFP_S[2x].astype(numpy.float32) 或

SMC[*]/DR_FPtoFP_D[x] = SMC[*]/DVR_FPtoFP_S[2x+1].astype(numpy.float32)

删除[SYT]: 当目的寄存为 32bit、 源寄存器为 32bit 的时候，支持 4 个 FPtoFP
源寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_FPtoFP_S[x]，x=0,1,...3
目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_FPtoFP_D[x]，x=0,1,...3
SMC[*]/DR_FPtoFP_D[x] = SMC[*]/DVR_FPtoFP_S[x].astype(numpy.float32)
当目的寄存器为 16bit、源寄存为 16bit 的时候，支持 8 个 FPtoFP
源寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DVR_FPtoFP_S[x]，x=0,1,...7
目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_FPtoFP_D[x]，x=0,1,...7
SMC[*]/DR_FPtoFP_D[x] = SMC[*]/DVR_FPtoFP_S[x].astype(numpy.float16)

2）当目的寄存器为 16bit、源寄存为 32bit 的时候，支持 4 个 FPtoFP

源寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_FPtoFP_S[y]，y=0,1,...3

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_FPtoFP_D[x]，x=0,1,...7

根据目的数据位置

SMC[*]/DR_FPtoFP_D[2x] = SMC[*]/DVR_FPtoFP_S[x].astype(numpy.float16) 或

SMC[*]/DR_FPtoFP_D[2x+1] = SMC[*]/DVR_FPtoFP_S[x].astype(numpy.float16)

5.3.9 INTtoINT（Integer To Integer Convert)

5.3.9.1 INTtoINT 的接口信号

表 5- 30：INTtoINT 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_INTtoINT_D	128	输出（寄存器）	见寄存器说明
CRU_INTtoINT_O	7	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_INTtoINT_S	128	输入	代表 INTtoINT 的计算输入数据
CRU_INTtoINT	7	输入	代表 INTtoINT 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.9.2 INTtoINT 相关的寄存器

1) SMC[*]/DR_INTtoINT_D：代表编号为*的 SMC 中的 INTtoINT 计算的输出寄存器，128bit。

5.3.9.3 INTtoINT 相关的设计参数

表 5- 31：INTtoINT 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_INTtoINT_CNT	INTtoINT 的数量，基于最大数据精度	4
PARAM_DR_INTtoINT_PRECISION	INTtoINT 的数据精度	16bit 32bit
PARAM_SUBWORD_PARALLELISM	是否支持子字并行	是
PARAM_DIFF_PRECISION_SD	是否支持源目的之间是否数据精度不同	支持

5.3.9.4 INTtoINT 相关的微指令

表 5- 32：INTtoINT 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_prec	1	按顺序排列	源寄存器的精度 ‘b0：16bit ‘b1：32bit
dst_prec	1	按顺序排列	目的寄存器的精度 ‘b0：16bit ‘b1：32bit
src_sign	1	按顺序排列	源寄存器符号 ‘b0：无符号数 ‘b1：有符号数
dst_sign	1	按顺序排列	目的寄存器符号 ‘b0：无符号数 ‘b1：有符号数
src_high_or_low	1	按顺序排列	源数据位置 如果源寄存器精度非全精度，标志数据所在位置： ‘b0：低位 ‘b1：高位
dst_high_or_low	1	按顺序排列	目的数据位置 如果目的寄存器精度非全精度，标志数据所在位置： ‘b0：低位 ‘b1：高位
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.9.5 INTtoINT 功能说明

1）当目的寄存为 32bit、 源寄存器为 32bit 的时候，支持 4 个 INTtoINT

源寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_INTtoINT_S[x]，x=0,1,...3

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_INTtoINT_D[x]，x=0,1,...3

SMC[*]/DR_INTtoINT_D[x] = SMC[*]/DVR_INTtoINT_S[x].astype(numpy.int32)

2）当目的寄存器为 16bit、源寄存为 16bit 的时候，支持 8 个 INTtoINT

源寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DVR_INTtoINT_S[x]，x=0,1,...7

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_INTtoINT_D[x]，x=0,1,...7

SMC[*]/DR_INTtoINT_D[x] = SMC[*]/DVR_INTtoINT_S[x].astype(numpy.int16)

3）当目的寄存为 32bit、 源寄存器为 16bit 的时候，支持 4 个 INTtoINT

源寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DVR_INTtoINT_S[y]，y=0,1,...7

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_INTtoINT_D[x]，x=0,1,...3

根据源数据位置

SMC[*]/DR_INTtoINT_D[x] = SMC[*]/DVR_INTtoINT_S[2x].astype(numpy.int32) 或

SMC[*]/DR_INTtoINT_D[x] = SMC[*]/DVR_INTtoINT_S[2x+1].astype(numpy.int32)

4）当目的寄存器为 16bit、源寄存为 32bit 的时候，支持 4 个 INTtoINT

源寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_INTtoINT_S[y]，y=0,1,...3

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_INTtoINT_D[x]，x=0,1,...7

根据目的数据位置

SMC[*]/DR_INTtoINT_D[2x] = SMC[*]/DVR_INTtoINT_S[x].astype(numpy.int16) 或

SMC[*]/DR_INTtoINT_D[2x+1] = SMC[*]/DVR_INTtoINT_S[x].astype(numpy.int16)

5.3.10 FPtoINT（Float To Integer Convert）

5.3.10.1 FPtoINT 的接口信号

表 5- 33：FPtoINT 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_FPtoINT_D	128	输出（寄存器）	见寄存器说明
CRU_FPtoINT_O	5	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_FPtoINT_S	128	输入	代表 FPtoINT 的计算输入数据
CRU_FPtoINT	5	输入	代表 FPtoINT 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.10.2 FPtoINT 相关的寄存器

1) SMC[*]/DR_FPtoINT_D：代表编号为*的 SMC 中的 FPtoINT 输出寄存器，128bit。

5.3.10.3 FPtoINT 相关的设计参数

表 5- 34：FPtoINT 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_FPtoINT_CNT	FPtoINT 的数量，基于最大数据精度	4
PARAM_DR_FPtoINT_PRECISION	FPtoINT 的数据精度	整数 16bit 整数 32bit
PARAM_SUBWORD_PARALLELISM	是否支持子字并行	是
PARAM_DIFF_PRECISION_SD	是否支持源目的之间是否数据精度不同	支持

5.3.10.4 FPtoINT 相关的微指令

表 5- 35：FPtoINT 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_prec	1	按顺序排列	源寄存器的精度 ‘b0：16bit ‘b1：32bit
dst_prec	1	按顺序排列	目的寄存器的精度 ‘b0：16bit ‘b1：32bit
src_high_or_low	1	按顺序排列	源数据位置 如果源寄存器精度非全精度，标志数据所在位置： ‘b0：低位 ‘b1：高位
dst_high_or_low	1	按顺序排列	目的数据位置 如果目的寄存器精度非全精度，标志数据所在位置： ‘b0：低位 ‘b1：高位
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.10.5 FPtoINT 功能说明

1）当目的寄存为 32bit、 源寄存器为 32bit 的时候，支持 4 个 FPtoINT

源寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_FPtoINT_S[x]，x=0,1,...3

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_FPtoINT_D[x]，x=0,1,...3

SMC[*]/DR_FPtoINT_D[x] = SMC[*]/DVR_FPtoINT_S[x].astype(numpy.int32)

2）当目的寄存器为 16bit、源寄存为 16bit 的时候，支持 8 个 FPtoINT

源寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DVR_FPtoINT_S[x]，x=0,1,...7

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_FPtoINT_D[x]，x=0,1,...7

SMC[*]/DR_FPtoINT_D[x] = SMC[*]/DVR_FPtoINT_S[x].astype(numpy.int16)

3）当目的寄存为 32bit、 源寄存器为 16bit 的时候，支持 4 个 FPtoINT

源寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DVR_FPtoINT_S[y]，y=0,1,...7

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_FPtoINT_D[x]，x=0,1,...3

根据源数据位置

SMC[*]/DR_FPtoINT_D[x] = SMC[*]/DVR_FPtoINT_S[2x].astype(numpy.int32) 或

SMC[*]/DR_FPtoINT_D[x] = SMC[*]/DVR_FPtoINT_S[2x+1].astype(numpy.int32)

4）当目的寄存器为 16bit、源寄存为 32bit 的时候，支持 4 个 FPtoINT

源寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_FPtoINT_S[y]，y=0,1,...3

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_FPtoINT_D[x]，x=0,1,...7

根据目的数据位置

SMC[*]/DR_FPtoINT_D[2x] = SMC[*]/DVR_FPtoINT_S[x].astype(numpy.int16) 或

SMC[*]/DR_FPtoINT_D[2x+1] = SMC[*]/DVR_FPtoINT_S[x].astype(numpy.int16)

5.3.11 INTtoFP（Integer To Float Convert)

5.3.11.1 INTtoFP 的接口信号

表 5- 36：INTtoFP 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号

DR_INTtoFP_D	128	输出（寄存器）	见寄存器说明
CRU_INTtoFP_O	6	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_INTtoFP_S	128	输入	代表 INTtoFP 的计算输入数据
CRU_INTtoFP	6	输入	代表 INTtoFP 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.11.2 INTtoFP 相关的寄存器

1) SMC[*]/DR_INTtoFP_D：代表编号为*的 SMC 中的 INTtoFP 输出寄存器，128bit。

5.3.11.3 INTtoFP 相关的设计参数

表 5- 37：INTtoFP 相关设计参数表

名称	**含义**	**V1.0 支持**
PARAM_DR_INTtoFP_CNT	INTtoFP 的数量，基于最大数据精度	4 个 32bit INTtoFP 或 8 个 16bit INTtoFP
PARAM_DR_INTtoFP_PRECISION	INTtoFP 的数据精度	整数 16bit 整数 32bit
PARAM_SUBWORD_PARALLELISM	是否支持子字并行	是

5.3.11.4 INTtoFP 相关的微指令

表 5- 38：INTtoFP 相关上行微指令表

名称	**Bit 数**	**Bit 位**	**含义**
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
src_prec	1	按顺序排列	源寄存器的精度 ‘b0：16bit ‘b1：32bit
dst_prec	1	按顺序排列	目的寄存器的精度 ‘b0：16bit ‘b1：32bit
src_sign	1	按顺序排列	源寄存器符号 ‘b0：无符号数 ‘b1：有符号数
src_high_or_low	1	按顺序排列	源数据位置 如果源寄存器精度非全精度，标志数据所在位置： ‘b0：低位 ‘b1：高位

dst_high_or_low	1	按顺序排列	目的数据位置 如果目的寄存器精度非全精度，标志数据所在位置： ‘b0：低位 ‘b1：高位
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.11.5 INTtoFP 功能说明

1）当目的寄存为 32bit、 源寄存器为 32bit 的时候，支持 4 个 INTtoFP

源寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_INTtoFP_S[x]，x=0,1,...3

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_INTtoFP_D[x]，x=0,1,...3

SMC[*]/DR_INTtoFP_D[x] = SMC[*]/DVR_INTtoFP_S[x].astype(numpy.float32)

2）当目的寄存器为 16bit、源寄存为 16bit 的时候，支持 8 个 INTtoFP

源寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DVR_INTtoFP_S[x]，x=0,1,...7

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_INTtoFP_D[x]，x=0,1,...7

SMC[*]/DR_INTtoFP_D[x] = SMC[*]/DVR_INTtoFP_S[x].astype(numpy.float16)

3）当目的寄存为 32bit、 源寄存器为 16bit 的时候，支持 4 个 INTtoFP

源寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DVR_INTtoFP_S[y]，y=0,1,...7

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_INTtoFP_D[x]，x=0,1,...3

根据源数据位置

SMC[*]/DR_INTtoFP_D[x] = SMC[*]/DVR_INTtoFP_S[2x].astype(numpy.float32) 或

SMC[*]/DR_INTtoFP_D[x] = SMC[*]/DVR_INTtoFP_S[2x+1].astype(numpy.float32)

4）当目的寄存器为 16bit、源寄存为 32bit 的时候，支持 4 个 INTtoFP

源寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_INTtoFP_S[y]，y=0,1,...3

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_INTtoFP_D[x]，x=0,1,...7

根据目的数据位置

SMC[*]/DR_INTtoFP_D[2x] = SMC[*]/DVR_INTtoFP_S[x].astype(numpy.float16) 或

SMC[*]/DR_INTtoFP_D[2x+1] = SMC[*]/DVR_INTtoFP_S[x].astype(numpy.float16)

5.3.12 LOGIC

5.3.12.1 LOGIC 的接口信号

表 5- 39：LOGICAL 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_LOGIC_D	128	输出（寄存器）	见寄存器说明
CRU_LOGIC_O	6	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_LOGIC_S0	128	输入	代表 LOGICAL 的计算输入数据 0
DVR_LOGIC_S1	128	输入	代表 LOGICAL 的计算输入数据 1
CRU_LOGIC	6	输入	代表 LOGICAL 的上行指令寄存器，存储内容为下面介绍的微指令。
DVR_LOGIC_ST	128	输入	代表 LOGICAL 的源 Status 寄存器 (1) Bit2、1、0 分别为第一组 GT（Great than）、EQ（Equal）、LS（Less than）。 (2) Bit18、17、16 分别为第二组 GT（Great than）、EQ（Equal）、LS（Less than）。 (3) Bit34、33、32 分别为第三组 GT（Great than）、EQ（Equal）、LS（Less than）。 (4) Bit50、49、48 分别为第四组 GT（Great than）、EQ（Equal）、LS（Less than）。 (5) Bit66、65、64 分别为第五组 GT（Great than）、EQ（Equal）、LS（Less than）。 (6) Bit82、81、80 分别为第六组 GT（Great than）、EQ（Equal）、LS（Less than）。 (7) Bit98、97、96 分别为第七组 GT（Great than）、EQ（Equal）、LS（Less than）。 (8) Bit114、113、112 分别为第八组 GT（Great than）、EQ（Equal）、LS（Less than）。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.12.2 LOGIC 相关的寄存器

1) SMC[*]/DR_LOGIC_D：代表编号为*的 SMC 中的 LOGICAL 的输出寄存器，128bit。

5.3.12.3 LOGIC 相关的设计参数

表 5- 40：LOGICAL 相关设计参数表

名称	含义	V1.0 支持
PARAM_DR_LOGICAL_CNT	LOGICAL 的数量，基于最大数据精度	4
PARAM_DR_LOGICAL_PRECISION	LOGICAL 的数据精度	32bit 和 16bit
PARAM_SUBWORD_PARALLELISM	是否支持子字并行	是
PARAM_DIFF_PRECISION_SS	是否支持源源之间是否数据精度不同	不支持
PARAM_DIFF_PRECISION_SD	是否支持源目的之间是否数据精度不同	不支持

5.3.12.4 LOGIC 相关的微指令

表 5- 41：LOGICAL 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	指令有效 ‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
inst_typ	4	按顺序排列	指令类型 ‘b0000：and ‘b0001：or ‘b0010：xor ‘b0011：not ‘b0100：copy ‘b0101：select_great ‘b0110：select_equal ‘b0111：select_less ‘b1000：logic_left_shift ‘b1001：arith_left_shift ‘b1010：rotate_left_shift ‘b1011：logic_right_shift ‘b1100：arith_right_shift ‘b1101：rotate_right_shift ‘b1110：get_first_one ‘b1111：get_first_zero
prec	1	按顺序排列	数据精度 ‘b0：16bit ‘b1：32bit
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.12.5 LOGIC 功能说明

指令操作类型支持 copy, not, and, or, xor, select_great, select_equal, select_less, logic_shift, arith_shift, rotate_shift, get_first_one, get_first_zero

1）当目的和源寄存器为 32bit 的时候，支持 4 个 32bit 计算

目的寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DR_LOGIC_D[x]，x=0,1,...3

源寄存器 0，分为 4 组，每组 32bit，记为 SMC[*]/DVR_LOGIC_S0[x]，x=0,1,...3

源寄存器 1，分为 4 组，每组 32bit，记为 SMC[*]/DVR_LOGIC_S1[x]，x=0,1,...3

源状态寄存器，分为 4 组，每组 32bit，记为 SMC[*]/DVR_LOGIC_ST[x]，x=0,1,...3，其中

SMC[*]/DVR_LOGIC_ST[x][0]记作 SMC[*]/DVR_LOGIC_ST[x]_LS

SMC[*]/DVR_LOGIC_ST[x][1]记作 SMC[*]/DVR_LOGIC_ST[x]_EQ

SMC[*]/DVR_LOGIC_ST[x][2]记作 SMC[*]/DVR_LOGIC_ST[x]_GT

2）当目的和源寄存器为 16bit 的时候，支持 8 个 16bit 计算

目的寄存器，分为 8 组，每组 16bit，记为 SMC[*]/DR_LOGIC_D[x]，x=0,1,...7

源寄存器 0，分为 8 组，每组 16bit，记为 SMC[*]/DVR_LOGIC_S0[x]，x=0,1,...7

源寄存器 1，分为 8 组，每组 16bit，记为 SMC[*]/DVR_LOGIC_S1[x]，x=0,1,...7

源状态寄存器，分为 8 组，每组 16bit，记为 SMC[*]DVR_LOGIC_ST[x]，x=0,1,...7，其中

SMC[*]/DVR_LOGIC_ST[x][0]记作 SMC[*]/DVR_LOGIC_ST[x]_LS

SMC[*]/DVR_LOGIC_ST[x][1]记作 SMC[*]/DVR_LOGIC_ST[x]_EQ

SMC[*]/DVR_LOGIC_ST[x][2]记作 SMC[*]/DVR_LOGIC_ST[x]_GT

1）Copy：SMC[*]/DR_LOGIC_D[x] = SMC[*]/DVR_LOGIC_S0[x]；

2）Not：SMC[*]/DR_LOGIC_D[x] = ~ SMC[*]/DVR_LOGIC_S0[x]；

3）And：SMC[*]/DR_LOGIC_D[x] = SMC[*]/DVR_LOGIC_S0[x] & SMC[*]/DVR_LOGIC_S1[x]；

4）Or：SMC[*]/DR_LOGIC_D[x] = SMC[*]/DVR_LOGIC_S0[x] | SMC[*]/DVR_LOGIC_S1[x]；

5）Xor：SMC[*]/DR_LOGIC_D[x] = SMC[*]/DVR_LOGIC_S0[x] ^ SMC[*]/DVR_LOGIC_S1[x]；

6）Select_great： SMC[*]/DR_LOGIC_D[x] = SMC[*]/DVR_LOGIC_ST_GT ? SMC[*]/DVR_LOGIC_S0[x] : SMC[*]/DVR_LOGIC_S1[x]；

7）Select_equal： SMC[*]/DR_LOGIC_D[x] = SMC[*]/DVR_LOGIC_ST_EQ ? SMC[*]/DVR_LOGIC_S0[x] : SMC[*]/DVR_LOGIC_S1[x]；

8）Select_less： SMC[*]/DR_LOGIC_D[x] = SMC[*]/DVR_LOGIC_ST_LS ? SMC[*]/DVR_LOGIC_S0[x] : SMC[*]/DVR_LOGIC_S1[x]；

9）Logic_left_shift

10）Arith_left_shift

11）Rotate_left_shift

12）Logic_right_shift

13）Arith_right_shift

14）Rotate_right_shift

15）Get_first_one：对于 4 个 32bit 或者 8 个 16bit 的 SMC[*]/DVR_LOGIC_S0[x]分别计算，从低位向高位查找 bit 1，返回第一个 bit 1 所在位置，如果没有 bit 1 则返回全“1”的数，返回值 SMC[*]/DR_LOGIC_D[x]与 SMC[*]/DVR_LOGIC_S0[x]具有相同的数据精度。

16）Get_first_zero：对于 4 个 32bit 或者 8 个 16bit 的 SMC[*]/DVR_LOGIC_S0[x]分别计算，从低位向高位查找 bit 0，返回第一个 bit 0 所在位置，如果没有 bit 0 则返回全“1”的数，返回值 SMC[*]/DR_LOGIC_D[x]与 SMC[*]/DVR_LOGIC_S0[x]具有相同的数据精度。

5.3.13 SHIFTUP（Shift Up)

5.3.13.1 SHIFTUP 的接口信号

表 5- 42：SHIFTUP 的接口信号表

名称	Bit	输入 / 输出（寄存器）	含义

	数		
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_SHIFTUP_D	128	输出（寄存器）	代表 SHIFTUP 的计算输出数据
CRU_SHIFTUP_O	135	输出（寄存器）	代表 SHIFTUP 的上行指令寄存器，去往下游的输出信号。
CRU_SHIFTUP_I	135	输入	代表 SHIFTUP 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.13.2 SHIFTUP 相关的寄存器

1）SMC[*]/CRU_SHIFTUP_O：SMC*中用于 Shift Up 的指令寄存器。

5.3.13.3 SHIFTUP 相关的设计参数

表 5- 43：SHIFTUP 相关设计参数表

名称	含义	V1.0 支持
PARAM_UR_WORD_CNT	User register 宽度的字数，每个字为 32bit	4

5.3.13.4 SHIFTUP 相关的微指令

表 5- 44：SHIFTUP 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
data	128	按顺序排列	写入数据
broadcast	1	按顺序排列	‘b0：不广播 ‘b1：广播
smc_id_en	5	按顺序排列	当广播时，smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能； 当不广播时，smc_id_en 是要写入的目的用户寄存器的 SMC ID。 注意：此处定义与一般计算模块的定义有差别。

5.3.13.5 SHIFTUP 功能说明

if SMC_ID<=smc_id_en:

SMC[SMC_ID]/CRU_SHIFTUP_O = SMC[SMC_ID]/CRU_SHIFTUP_I

if broadcast==0 && SMC_ID==smc_id_en

SMC[SMC_ID]/DR_SHIFTUP_D = SMC[SMC_ID]/CRU_SHIFTUP_O[data]

if broadcast==1 && SMC_ID<=smc_id_en

SMC[SMC_ID]/DR_SHIFTUP_D = SMC[SMC_ID]/CRU_SHIFTUP_O[data]

删除[SYT]: /

删除[SYT]: /

删除[SYT]: 0

删除[SYT]:

删除[SYT]: /

删除[SYT]: /

5.3.14 SHIFTDN（Shift Down）

5.3.14.1 SHIFTDN 的接口信号

表 5- 45：SHIFTDN 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
CRU_SHIFTDN_O	7	输出（寄存器）	代表 SHIFTDN 的上行指令寄存器，去往下游的输出信号。
CRU_SHIFTDN_I	7	输入	代表 SHIFTDN 的上行指令寄存器，存储内容为下面介绍的微指令。
DVR_SHIFTDN_S	128	输入	代表 SHIFTDN 的计算输入数据
CRD_SHIFTDN_O	129	输出（寄存器）	代表 SHIFTDN 的下行指令寄存器，去往下游的输出信号。
CRD_SHIFTDN_I	129	输入	代表 SHIFTDN 的下行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.14.2 SHIFTDN 相关的寄存器

1）SMC[*]/CRD_SHIFTDN_O：SMC*中用于 Single Load 的下行指令寄存器。

5.3.14.3 SHIFTDN 相关的设计参数

表 5- 46：SHIFTDN 相关设计参数表

名称	含义	V1.0 支持
PARAM_UR_WORD_CNT	User register 宽度的字数，每个字为 32bit	4

5.3.14.4 SHIFTDN 相关的微指令

表 5- 47：SHIFTDN 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
broadcast	1	按顺序排列	‘b0：不广播 ‘b1：广播
smc_id_en	5	按顺序排列	当广播时，smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能； 当不广播时，smc_id_en 是要读取的目的用户寄存器的 SMC ID。 注意：此处定义与一般计算模块的定义有差别。

表 5- 47：SHIFTDN 相关下行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
data	128	按顺序排列	读出数据

5.3.14.5 SHIFTDN 功能说明

if broadcast==0 && SMC_ID==smc_id_en

SMC[SMC_ID]/CRD_SHIFTDN_O[128:1] = SMC[SMC_ID]/DVR_SHIFTDN_S

SMC[SMC_ID]/CRD_SHIFTDN_O[0] = 1

if broadcast==1 && SMC_ID<=smc_id_en

SMC[SMC_ID]/CRD_SHIFTDN_O[128:1] = SMC[SMC_ID]/DVR_SHIFTDN_S

SMC[SMC_ID]/CRD_SHIFTDN_O[0] = 1

if CRD_SHIFTDN_I[0]==1

删除[SYT]: if SMC_ID==smc_id_en:

删除[SYT]: U

删除[SYT]:
if SMC_ID<smc_id_en:
SMC[SMC_ID]/CRU_SHIFTDN_O = SMC[SMC_ID]/CRU_SHIFTDN_I

SMC[SMC_ID]/CRD_SHIFTDN_O = SMC[SMC_ID]/CRD_SHIFTDN_I

5.3.15 STB（Burst Store）

删除[SYT]:

5.3.15.1 STB 的接口信号

表 5- 48：STB 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
AXI*			AXI 的 AW、W、B 通道信号
AXISM_M*			AXI Stream Master 接口信号
AXISM_S*			AXI Stream Slave 接口信号
UR_RE	1	输出（寄存器）	UR 的读使能
UR_RADDR	15	输出（寄存器）	UR 的读地址
UR_RDATA	128	输入	UR 的输出数据
CRU_STB_I		输入	
CRU_STB_O		输出（寄存器）	
CRD_STB_I		输入	
CRD_STB_O		输出（寄存器）	
CRU_STBAXI_I		输入	
CRU_STBAXI_O		输出（寄存器）	
CRD_STBAXI_I		输入	
CRD_STBAXI_O		输出（寄存器）	
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.15.2 STB 相关的寄存器

1）SMC[*]/UR_RE：SMC*中的 UR 读使能寄存器。

2）SMC[*]/UR_ADDR：SMC*中的 UR 读地址寄存器。

3）SMC[*]/CRU_STB_O：SMC*中的上行指令的输出寄存器。

4）SMC[*]/CRD_STB_O：SMC*中的下行指令的输出寄存器。

5）SMC[*]/CRU_STBAXI_O：SMC*中的上行指令的输出寄存器。

6）SMC[*]/CRD_STBAXI_O：SMC*中的下行指令的输出寄存器。

5.3.15.3 STB 相关的设计参数

表 5- 49：STB 相关设计参数表

名称	含义	V1.0 支持
PARAM_UR_BYTE_CNT	User register 宽度的字节数	16
PARAM_OUTSTANDING_INST_NUM	SMI 最多可以发出的在执行中的指令	4
PARAM_AXI_EN	是否具有 AXI 接口，当没有 AXI 接口的时候，STB 与 LDB 模块联合起来亦能实现从 UR 到 UR 的数据搬运和整理	1

5.3.15.4 STB 相关的微指令

表 5- 50：STB 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
ur_addr	15	按顺序排列	要读取的目的用户寄存器的地址，该地址为字节地址。
ur_rd_line_cnt	8	按顺序排列	从 UR 读取的行数。
line_byte_cnt	9	按顺序排列	每一行的数据量，最大值为 511
line_stride	6	按顺序排列	行地址间距，相邻行对应元素的地址差为 16*line_stride
inst_id	4	按顺序排列	指令编号 SMI 最多可以发出 PARAM_OUTSTANDING_INST_NUM 个在执行指令，之后 SMI 只有从下行的微指令中获得指令完成的指示，才能发出新的指令。指令编号是循环使用的。
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能
tid	2	按顺序排列	AXI stream 的信号 ‘b00：直接去往 LDB（通过 DS，但是只做 pipeline） ‘b01：通过 DS 去往 LDB ‘b10：直接去往 STBAXI（通过 DS，但是只做 pipeline） ‘b11：通过 DS 去往 STBAXI

表 5- 51：STB 相关下行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
done	1	按顺序排列	指令完成
inst_id	4	按顺序排列	完成的指令编号

表 5- 52：STBAXI 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
first_byte_strb	4	按顺序排列	第一个 AXI burst cycle 的按 byte 的写使能信号编码，除了最后一个 cycle 其他的 byte_strobe 都需要是全 1 ‘h0：strobe=1111_1111_1111_1111 ‘h1：strobe=0000_0000_0000_0001 ‘h2：strobe=0000_0000_0000_0011 ‘h3：strobe=0000_0000_0000_0111 ‘h4：strobe=0000_0000_0000_1111 ‘h5：strobe=0000_0000_0001_1111 ‘h6：strobe=0000_0000_0011_1111 ‘h7：strobe=0000_0000_0111_1111 ‘h8：strobe=0000_0000_1111_1111 ‘h9：strobe=0000_0001_1111_1111 ‘ha：strobe=0000_0011_1111_1111 ‘hb：strobe=0000_0111_1111_1111 ‘hc：strobe=0000_1111_1111_1111 ‘hd：strobe=0001_1111_1111_1111 ‘he：strobe=0011_1111_1111_1111 ‘hf：strobe=0111_1111_1111_1111
last_byte_strb	4	按顺序排列	最后一个 AXI burst cycle 的按 byte 的写使能信号编码，除了最后一个 cycle 其他的 byte_strobe 都需要是全 1 ‘h0：strobe=1111_1111_1111_1111 ‘h1：strobe=0000_0000_0000_0001 ‘h2：strobe=0000_0000_0000_0011 ‘h3：strobe=0000_0000_0000_0111 ‘h4：strobe=0000_0000_0000_1111 ‘h5：strobe=0000_0000_0001_1111 ‘h6：strobe=0000_0000_0011_1111 ‘h7：strobe=0000_0000_0111_1111 ‘h8：strobe=0000_0000_1111_1111 ‘h9：strobe=0000_0001_1111_1111

			‘ha：strobe=0000_0011_1111_1111 ‘hb：strobe=0000_0111_1111_1111 ‘hc：strobe=0000_1111_1111_1111 ‘hd：strobe=0001_1111_1111_1111 ‘he：strobe=0011_1111_1111_1111 ‘hf：strobe=0111_1111_1111_1111
brst	4	按顺序排列	对 AXI burst length 的自然编码，至少支持 burst length 16。 一个 burst 的第一个和最后一个 cycle 才会有 strobe 不全为 1 的情况，其他 cycle 的 strobe 应该都为 1。
gr_base_addr	40	按顺序排列	数据所在外部地址
gr_intlv_addr	40	按顺序排列	各个 SMC 访问外部地址空间的间差地址
inst_id	4	按顺序排列	指令编号 SMI 最多可以发出 PARAM_OUTSTANDING_INST_NUM 个在执行指令，之后 SMI 只有从下行的微指令中获得指令完成的指示，才能发出新的指令。指令编号是循环使用的。
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

表 5- 53：STBAXI 相关下行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
done	1	按顺序排列	指令完成
inst_id	4	按顺序排列	完成的指令编号
err	1	按顺序排列	AXI 总线收到错误的反馈信息

5.3.15.5 STB 功能说明

1）从每一个 smc 读取 burst 长度的数据，这些数据写到外部地址 gr_base_addr，各个 smc 具有 gr_intlv_addr，即第 n 个 smc 向 gr_base_addr + gr_intlv_addr * n (n=0~N_1) 地址开始写入。

2）读取 UR 不会发生竞争。

3）STB 设计中采用 STB 和 STBAXI 两个指令来控制，这两个部分的设计相对解耦，允许反压。中间接口采用 AXI Stream。

4）当不需要特殊的数据整理的时候，从 UR 读取的数据直接传送到 AXI 接口写出，无需 AXI stream 接口；以下情况会用到 AXI stream 接口：

- 从 UR 读再写回 UR，此时不管数据是否需要进行整理，数据流为 STB 读取 UR 数据，发送给 DS 模块，DS 模块经过所需的数据整理，发送给 LDB。
- 从 UR 读，最终从 AXI 接口写出，但是需要数据整理， 数据流为 STB 读取 UR 数据，发送给 DS 模块，DS 模块经过所需的数据整理，发送给 STB。

5）AXI 模块同时接收 STB UR 读取部分的数据和来自 DS 的数据，实际使用中同一条指令，只会有一个数据通道有数据。

6）对于下行微指令

- 每一个 SMC 只有接到下游的 SMC 发来指令，并且下游指令对应的指令编号在本 SMC 也已经完成的情况下才会向上游发出指令。特例是最后一个 SMC（基于 SMC_ID 为全 1 判断）不需要考虑下游。
- 在发送指令的时候，发向上游的指令中的 err 是本地的 err 与来自下游的指令中的 err 的或。

5.3.16 LDB（Burst Load)

5.3.16.1 LDB 的接口信号

表 5- 54：LDB 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
AXI*			AXI 的 AR、R 通道信号
AXISM_M*			AXI Stream Master 接口信号
AXISM_S*			AXI Stream Slave 接口信号
UR_WE	16	输出（寄存器）	UR 的写使能
UR_WADDR	15	输出（寄存器）	UR 的写地址
UR_WDATA	128	输出（寄存器）	UR 的输入数据
CRU_LDB_I		输入	
CRU_LDB_O		输出（寄存器）	
CRD_LDB_I		输入	
CRD_LDB_O		输出（寄存器）	
CRU_LDBAXI_I		输入	

CRU_LDBAXI_O		输出（寄存器）	
CRD_LDBAXI_I		输入	
CRD_LDBAXI_O		输出（寄存器）	
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.16.2 LDB 相关的寄存器

1）SMC[*]/UR_WE：SMC*中的 UR 的写使能寄存器。

2）SMC[*]/UR_ADDR：SMC*中的 UR 的写地址寄存器。

3）SMC[*]/UR_WDATA：SMC*中的 UR 的写数据寄存器。

4）SMC[*]/CRU_LDB_U：SMC*中的上行指令的输出寄存器。

5）SMC[*]/CRD_LDB_O：SMC*中的下行指令的输出寄存器。

6）SMC[*]/CRU_LDBAXI_U：SMC*中的上行指令的输出寄存器。

7）SMC[*]/CRD_LDBAXI_O：SMC*中的下行指令的输出寄存器。

5.3.16.3 LDB 相关的设计参数

表 5- 55：LDB 相关设计参数表

名称	**含义**	**V1.0 支持**
PARAM_UR_BYTE_CNT	User register 宽度的字节数	16
PARAM_OUTSTANDING_INST_NUM	SMI 最多可以发出的在执行中的指令	4
PARAM_AXI_EN	是否具有 AXI 接口，当没有 AXI 接口的时候，STB 与 LDB 模块联合起来亦能实现从 UR 到 UR 的数据搬运和整理	1

5.3.16.4 LDB 相关的微指令

表 5- 56：LDB 相关上行微指令表

名称	**Bit 数**	**Bit 位**	**含义**
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
ur_addr	15	按顺序排列	要写入的目的用户寄存器的地址，该地址为字节地址。
ur_wr_line_cnt	8	按顺序排列	向 UR 写入的行数。

line_byte_cnt	9	按顺序排列	每一行的数据量，最大值为 511
line_stride	6	按顺序排列	行地址间距，相邻行对应元素的地址差为 16*line_stride
const	6	按顺序排列	const[1:0]：为数据精度 ● 2‘b00：4bit ● 2‘b01：8bit ● 2‘b10：16bit ● 2‘b11：32bit const[3:2]：为数据类型 ● 2‘b00：uint ● 2‘b01：int ● 2’ b10：fp 规格化数 ● 2’ b11：fp 非规格化数 const[5:4]：填充数值 ● 2’ b00：零 ● 2’ b01：无效 ● 2’ b10：最小值 ● 2’ b11：最大值
pre_fill	1	按顺序排列	如果每一行写入起始地址不对齐到 16 字节（物理上 UR 的一行）边界时候，每一行开始的没有被来自 AXI 的数据写入的部分是否填充常数。
post_fill	1	按顺序排列	如果每一行写入结束地址不对齐到 16 字节（物理上 UR 的一行）边界时候，每一行结束的没有被来自 AXI 的数据写入的部分是否填充常数。
inst_id	4	按顺序排列	指令编号 SMI 最多可以发出 PARAM_OUTSTANDING_INST_NUM 个在执行指令，之后 SMI 只有从下行的微指令中获得指令完成的指示，才能发出新的指令。指令编号是循环使用的。
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

表 5- 57：LDB 相关下行微指令表

名称	**Bit 数**	**Bit 位**	**含义**
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
done	1	按顺序排列	指令完成
inst_id	4	按顺序排列	完成的指令编号

表 5- 58：LDBAXI 相关上行微指令表

名称	**Bit 数**	**Bit 位**	**含义**
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新

first_byte_strb	4	按顺序排列	第一个 AXI burst cycle 的按 byte 的写使能信号编码，除了最后一个 cycle 其他的 byte_strobe 都需要是全 1 'h0：strobe=1111_1111_1111_1111 'h1：strobe=0000_0000_0000_0001 'h2：strobe=0000_0000_0000_0011 'h3：strobe=0000_0000_0000_0111 'h4：strobe=0000_0000_0000_1111 'h5：strobe=0000_0000_0001_1111 'h6：strobe=0000_0000_0011_1111 'h7：strobe=0000_0000_0111_1111 'h8：strobe=0000_0000_1111_1111 'h9：strobe=0000_0001_1111_1111 'ha：strobe=0000_0011_1111_1111 'hb：strobe=0000_0111_1111_1111 'hc：strobe=0000_1111_1111_1111 'hd：strobe=0001_1111_1111_1111 'he：strobe=0011_1111_1111_1111 'hf：strobe=0111_1111_1111_1111
last_byte_strb	4	按顺序排列	最后一个 AXI burst cycle 的按 byte 的写使能信号编码，除了最后一个 cycle 其他的 byte_strobe 都需要是全 1 'h0：strobe=1111_1111_1111_1111 'h1：strobe=0000_0000_0000_0001 'h2：strobe=0000_0000_0000_0011 'h3：strobe=0000_0000_0000_0111 'h4：strobe=0000_0000_0000_1111 'h5：strobe=0000_0000_0001_1111 'h6：strobe=0000_0000_0011_1111 'h7：strobe=0000_0000_0111_1111 'h8：strobe=0000_0000_1111_1111 'h9：strobe=0000_0001_1111_1111 'ha：strobe=0000_0011_1111_1111 'hb：strobe=0000_0111_1111_1111 'hc：strobe=0000_1111_1111_1111 'hd：strobe=0001_1111_1111_1111 'he：strobe=0011_1111_1111_1111 'hf：strobe=0111_1111_1111_1111
brst	4	按顺序排列	对 AXI burst length 的自然编码，至少支持 burst length 16。 一个 burst 的第一个和最后一个 cycle 才会有 strobe 不全为 1 的情况，其他 cycle 的 strobe 应该都为 1。
gr_base_addr	40	按顺序排列	数据所在外部地址
gr_intlv_addr	40	按顺序排列	各个 SMC 访问外部地址空间的间差地址

inst_id	4	按顺序排列	指令编号 SMI 最多可以发出 PARAM_OUTSTANDING_INST_NUM 个在执行指令，之后 SMI 只有从下行的微指令中获得指令完成的指示，才能发出新的指令。指令编号是循环使用的。
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能
tid	2	按顺序排列	AXI stream 的信号 'b00：直接去往 LDB（通过 DS，但是只做 pipeline） 'b01：通过 DS 去往 LDB 'b10：直接去往 STBAXI（通过 DS，但是只做 pipeline） 'b11：通过 DS 去往 STBAXI

表 5- 59：LDBAXI 相关下行微指令表

名称	**Bit 数**	**Bit 位**	**含义**
vld	1	0	'b1：指令有效 'b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
done	1	按顺序排列	指令完成
inst_id	4	按顺序排列	完成的指令编号
err	1	按顺序排列	AXI 总线收到错误的反馈信息。

5.3.16.5 LDB 功能说明

1）对每一个 smc 读取 burst 长度的数据，这些数据来自外部地址 gr_base_addr，各个 smc 具有 gr_intlv_addr，即第 n 个 smc 从 gr_base_addr + gr_intlv_addr * n (n=0~N_1) 地址开始读取。

2）写入 UR 不会发生竞争。

3）LDB 设计中采用 LDB 和 LDBAXI 两个指令来控制，这两个部分的设计相对解耦，允许反压。中间接口采用 AXI Stream。

4）当不需要特殊的数据整理的时候，从 AXI 读取的数据直接传送到 UR 接口写出，无需 AXI stream 接口；以下情况会用到 AXI stream 接口：

- 从 UR 读再写回 UR，此时不管数据是否需要进行整理，数据流为 STB 读取 UR 数据，发送给 DS 模块，DS 模块经过所需的数据整理，发送给 LDB。
- 从 AXI 读，最终从 UR 接口写出，但是需要数据整理， 数据流为 LDB 读取 AXI 数据，发送给 DS 模块，DS 模块经过所需的数据整理，发送给 LDB。

5）UR write 模块同时接收 AXI 读取部分的数据和来自 DS 的数据，实际使用中同一条指令，只会有一个数据通道有数据。

6）对于下行微指令：

- 每一个 SMC 只有接到下游的 SMC 发来指令，并且下游指令对应的指令编号在本 SMC 也已经完成的情况下才会向上游发出指令。特例是最后一个 SMC（基于 SMC_ID 为全 1 判断）不需要考虑下游。
- 在发送指令的时候，发向上游的指令中的 err 是本地的 err 与来自下游的指令中的 err 的或。

7）允许通过配置 pre_fill 的方式，实现填充一整个行为常数。

设置格式[SYT]: 编号 + 级别: 1 + 编号样式: 1, 2, 3, ... + 起始编号: 1 + 对齐方式: 左侧 + 对齐位置: 0 毫米 + 缩进位置: 0 毫米

5.3.17 UR（User Register）

5.3.17.1 UR 的接口信号

表 5- 60：UR 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
UR_RDATA_0	128	输出（非寄存器）	读出数据 0，连接 RtoVR
UR_RDATA_1	128	输出（非寄存器）	读出数据 1，连接 RtoVR
UR_RDATA_2	128	输出（非寄存器）	读出数据 2，连接 RtoVR
UR_RDATA_3	128	输出（非寄存器）	读出数据 3，连接 RtoVR
URA_RE	1	输入	UR 的异步读使能，连接 LDB
URA_RADDR	15	输入	UR 的异步读地址，连接 LDB
URA_RDATA	128	输出（非寄存器）	UR 的异步输出数据，连接 LDB
CRU_UR_O	11	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
UVR_WDATA_0	128	输入	写入数据 0，连接 RtoVR
UVR_WDATA_1	128	输入	写入数据 1，连接 RtoVR
UVR_WDATA_2	128	输入	写入数据 2，连接 RtoVR
UVR_WDATA_3	128	输入	写入数据 3，连接 RtoVR
URA_WE	16	输入	UR 的异步写使能，连接 LDB
URA_WADDR	15	输入	UR 的异步写地址，连接 LDB
URA_WDATA	128	输入	UR 的异步输入数据，连接 LDB
CRU_UR		输入	代表 UR 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.17.2 UR 相关的寄存器

1）SMC[*]/UR_BANK*_WDATA：SMC*中用于 UR Bank*的写数据 Pipeline 寄存器，128bit。

2）SMC[*]/UR_BANK*_WE：SMC*中用于 UR Bank*的写使能 Pipeline 寄存器，1bit。

3）SMC[*]/UR_BANK*_WBE：SMC*中用于 UR Bank*的写字节使能 Pipeline 寄存器，PARAM_UR_BYTE_CNT bit。

4）SMC[*]/UR_BANK*_WADDR：SMC*中用于 UR Bank*的写地址 Pipeline 寄存器，PARAM_UR_BANK_BIT_NUM+ PARAM_UR_BANK_ADDR_BIT_NUM bit。

5）SMC[*]/UR_BANK*_RADDR：SMC*中用于 UR Bank*的读地址 Pipeline 寄存器，PARAM_UR_BANK_BIT_NUM+ PARAM_UR_BANK_ADDR_BIT_NUM bit。

6）SMC[*]/UR_BANK*_RE：SMC*中用于 UR Bank*的读使能 Pipeline 寄存器，1bit。

7）SMC[*]/UR_BANK*_RDATA：SMC*中用于 UR Bank*的读数据 Pipeline 寄存器，128bit。

5.3.17.3 UR 相关的设计参数

表 5- 61：UR 相关设计参数表

名称	含义	V1.0 支持
PARAM_UR_BYTE_CNT	User register 宽度的字节数	16
PARAM_UR_WR_PORT_CNT	User register 写端口数量	4
PARAM_UR_RD_PORT_CNT	User register 读端口数量	4
PARAM_UR_BANK_BIT_NUM	User register 的 bank 数量	3，即分为 8 个 bank
PARAM_UR_BANK_ADDR_BIT_NUM	User register 每个 bank 的 SRAM 的地址 bit 数量	8，即有 256 个地址
PARAM_UR_WRI_PIPE_CNT	写入 data path 的 pipeline 数量	0 或 1
PARAM_UR_RDI_PIPE_CNT	读取控制信号 data path 的 pipeline 数量	0 或 1
PARAM_UR_RDO_PIPE_CNT	读取输出信号的 pipeline 数量	0 或 1

5.3.17.4 UR 相关的微指令

表 5- 62：UR 相关上行微指令表

名称	Bit 数	Bit 位	含义
ur_we_0	1	0	写入使能 0
ur_wbank_0	PARAM_UR_BANK_BIT_N	按顺序排列	写入 BANK 的编码

	UM		
ur_waddr_0	PARAM_UR_BANK_ADDR_BIT_NUM	按顺序排列	写入地址 0
ur_wbe_0	PARAM_UR_BYTE_CNT	按顺序排列	写入字节使能 0
ur_smc_id_en_w0	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能
ur_we_1	1	按顺序排列	写入使能 1
ur_wbank_1	PARAM_UR_BANK_BIT_NUM	按顺序排列	写入 BANK 的编码
ur_waddr_1	PARAM_UR_BANK_ADDR_BIT_NUM	按顺序排列	写入地址 1
ur_wbe_1	PARAM_UR_BYTE_CNT	按顺序排列	写入字节使能 1
ur_smc_id_en_w1	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能
ur_we_2	1	按顺序排列	写入使能 2
ur_wbank_2	PARAM_UR_BANK_BIT_NUM	按顺序排列	写入 BANK 的编码
ur_waddr_2	PARAM_UR_BANK_ADDR_BIT_NUM	按顺序排列	写入地址 2
ur_wbe_2	PARAM_UR_BYTE_CNT	按顺序排列	写入字节使能 2
ur_smc_id_en_w2	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能
ur_we_3	1	按顺序排列	写入使能 3
ur_wbank_3	PARAM_UR_BANK_BIT_NUM	按顺序排列	写入 BANK 的编码
ur_waddr_3	PARAM_UR_BANK_ADDR_BIT_NUM	按顺序排列	写入地址 3
ur_wbe_3	PARAM_UR_BYTE_CNT	按顺序排列	写入字节使能 3
ur_smc_id_en_w3	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能
ur_re_0	1	按顺序排列	读取使能 0
ur_rbank_0	PARAM_UR_BANK_BIT_NUM	按顺序排列	读取 BANK 的编码
ur_raddr_0	PARAM_UR_BANK_ADDR_BIT_NUM	按顺序排列	读取地址 0
ur_smc_id_en_r0	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令

			使能
ur_re_1	1	按顺序排列	读取使能 1
ur_rbank_1	PARAM_UR_BANK_BIT_NUM	按顺序排列	读取 BANK 的编码
ur_raddr_1	PARAM_UR_BANK_ADDR_BIT_NUM	按顺序排列	读取地址 1
ur_smc_id_en_r1	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能
ur_re_2	1	按顺序排列	读取使能 2
ur_rbank_2	PARAM_UR_BANK_BIT_NUM	按顺序排列	读取 BANK 的编码
ur_raddr_2	PARAM_UR_BANK_ADDR_BIT_NUM	按顺序排列	读取地址 2
ur_smc_id_en_r2	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能
ur_re_3	1	按顺序排列	读取使能 3
ur_rbank_3	PARAM_UR_BANK_BIT_NUM	按顺序排列	读取 BANK 的编码
ur_raddr_3	PARAM_UR_BANK_ADDR_BIT_NUM	按顺序排列	读取地址 3
ur_smc_id_en_r3	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.17.5 UR 功能说明

1）UR 访问说明

- UR 里面每个 SRAM Bank 的读和写使能都有两个驱动源，一个是来自 CRU 指令，另一个是来自 STB/LDB 模块
- 这两个驱动源不会产生任何访问冲突，这是由编译器保证的，所以整体也无反压逻辑

2）当 PARAM_UR_WRI_PIPE_CNT=0 时

```
if SMC_ID==smc_id_en and UR_ID==ur_wbank_* and UR_ADDR==ur_waddr_* and ur_we_*=1 :
   SMC[SMC_ID]/ur_wbite_* = {CRU_UR[ur_wbe_*][15] ? 8’ hff : 8’ h0,
   ……
```

CRU_UR[ur_wbe_*][0] ? 8’ hff : 8’ h0}

SMC[SMC_ID]/UR_[UR_ID][UR_ADDR] = (SMC[SMC_ID]/UR_[UR_ID][UR_ADDR] &

~SMC[SMC_ID]/ur_wbite_*) | (SMC[SMC_ID]/ UVR_WDATA_* & SMC[SMC_ID]/ur_wbite_*)

*=0、1、2、3

当 PARAM_UR_WRI_PIPE_CNT=1 时

针对每个 BANK 的信号进行 Pipeline，涉及寄存器

- SMC[*]/UR_BANK*_WDATA
- SMC[*]/UR_BANK*_WE
- SMC[*]/UR_BANK*_WBE
- SMC[*]/UR_BANK*_WADDR

当 PARAM_UR_RDI_PIPE_CNT=0 时

if SMC_ID=smc_id_en and UR_ID=ur_rbank_* and UR_ADDR=ur_raddr_* and ur_re_*=1 :

SMC[SMC_ID]/ UVR_RDATA_* = SMC[SMC_ID]/UR_[UR_ID][UR_ADDR]

*=0、1、2、3

当 PARAM_UR_RDI_PIPE_CNT=1 时

针对每个 BANK 的信号进行 Pipeline，涉及寄存器

- SMC[*]/UR_BANK*_RADDR
- SMC[*]/UR_BANK*_RE

当 PARAM_UR_RDO_PIPE_CNT=1 时

针对每个 BANK 的信号进行 Pipeline，涉及寄存器

- SMC[*]/UR_BANK*_RDATA

3）UR 的内容

- 由于 UR 基于 SRAM 实现，因此上电之后其内容为随机值。
- 电路充分放电之后其内容为零，但是软件的正常工作不应该依赖于这一个前提。

5.3.18 RtoVR（DR/UR to DVR/UVR datapath）

5.3.18.1 RtoVR 的接口信号

表 5- 63：RtoVR 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DVR_INTADD_S0	128	输出	编号 0，具体解释见本部分功能说明
DVR_INTADD_S1	128	输出	编号 1，具体解释见本部分功能说明
DVR_INTADD_S2	128	输出	编号 2，具体解释见本部分功能说明
DVR_FPMUL_S0	128	输出	编号 3，具体解释见本部分功能说明
DVR_FPMUL_S1	128	输出	编号 4，具体解释见本部分功能说明
DVR_FPADD_S0	128	输出	编号 5，具体解释见本部分功能说明
DVR_FPADD_S1	128	输出	编号 6，具体解释见本部分功能说明
DVR_FPtoFP_S	128	输出	编号 7，具体解释见本部分功能说明
DVR_INTtoINT_S	128	输出	编号 8，具体解释见本部分功能说明
DVR_FPtoINT_S	128	输出	编号 9，具体解释见本部分功能说明
DVR_INTtoFP_S	128	输出	编号 10，具体解释见本部分功能说明
DVR_LOGIC_S0	128	输出	编号 11，具体解释见本部分功能说明
DVR_LOGIC_S1	128	输出	编号 12，具体解释见本部分功能说明
DVR_LOGIC_ST	128	输出	编号 13，具体解释见本部分功能说明
DVR_VIN0_S	128	输出	编号 14，具体解释见本部分功能说明
DVR_VIN1_S	128	输出	编号 15，具体解释见本部分功能说明
DVR_VOUT0_S	128	输出	编号 16，具体解释见本部分功能说明
DVR_INT8toFP16_S0	128	输出	编号 17，具体解释见本部分功能说明
DVR_INT8toFP16_S1	128	输出	编号 18，具体解释见本部分功能说明
DVR_FP16MUL_S0	128	输出	编号 19，具体解释见本部分功能说明
DVR_FP16MUL_S1	128	输出	编号 20，具体解释见本部分功能说明
DVR_FP16MUL_S2	128	输出	编号 21，具体解释见本部分功能说明
DVR_FP16MUL_S3	128	输出	编号 22，具体解释见本部分功能说明
DVR_FP16MUL_S4567	128	输出	编号 23，具体解释见本部分功能说明

DVR_FP32ADDTREE5TO1_S0	128	输出	编号 24，具体解释见本部分功能说明
DVR_FP32ADDTREE5TO1_S1	128	输出	编号 25，具体解释见本部分功能说明
DVR_FP32ADDTREE5TO1_S2	128	输出	编号 26，具体解释见本部分功能说明
DVR_FP32ADDTREE5TO1_S3	128	输出	编号 27，具体解释见本部分功能说明
DVR_FP32ADDTREE5TO1_S4	128	输出	编号 28，具体解释见本部分功能说明
DVR_FP32ADDTREE5TO1_S5	128	输出	编号 29，具体解释见本部分功能说明
DVR_FP32ADDTREE5TO1_S6	128	输出	编号 30，具体解释见本部分功能说明
DVR_FP32ADDTREE5TO1_S7	128	输出	编号 31，具体解释见本部分功能说明
DVR_FP32ADDTREE8TO1_S0	128	输出	编号 32，具体解释见本部分功能说明
DVR_FP32ADDTREE8TO1_S1	128	输出	编号 33，具体解释见本部分功能说明
DVR_SHIFTDN_S	128	输出	编号 34，具体解释见本部分功能说明
UVR_WDATA_0	128	输出	编号 35，具体解释见本部分功能说明
UVR_WDATA_1	128	输出	编号 36，具体解释见本部分功能说明
UVR_WDATA_2	128	输出	编号 37，具体解释见本部分功能说明
UVR_WDATA_3	128	输出	编号 38，具体解释见本部分功能说明
CRU_RTOVR_O		输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DR_INTADD_D0	128	输入	编号 0，具体解释见本部分功能说明
DR_INTADD_D1	128	输入	编号 1，具体解释见本部分功能说明
DR_INTADD_ST	128	输入	编号 2，具体解释见本部分功能说明
DR_FPMUL_D	128	输入	编号 3，具体解释见本部分功能说明
DR_FPADD_D	128	输入	编号 4，具体解释见本部分功能说明
DR_FPADD_ST	128	输入	编号 5，具体解释见本部分功能说明
DR_FPtoFP_D	128	输入	编号 6，具体解释见本部分功能说明
DR_INTtoINT_D	128	输入	编号 7，具体解释见本部分功能说明
DR_FPtoINT_D	128	输入	编号 8，具体解释见本部分功能说明
DR_INTtoFP_D	128	输入	编号 9，具体解释见本部分功能说明
DR_LOGIC_D	128	输入	编号 10，具体解释见本部分功能说明
DR_VIN0_D	128	输入	编号 11，具体解释见本部分功能说明
DR_VIN1_D	128	输入	编号 12，具体解释见本部分功能说明
DR_VOUT0_D	128	输入	编号 13，具体解释见本部分功能说明
DR_INT8toFP16_D0	128	输入	编号 14，具体解释见本部分功能说明
DR_INT8toFP16_D1	128	输入	编号 15，具体解释见本部分功能说明
DR_INT8toFP16_D2	128	输入	编号 16，具体解释见本部分功能说明
DR_INT8toFP16_D3	128	输入	编号 17，具体解释见本部分功能说明
DR_FP16MUL_D0	128	输入	编号 18，具体解释见本部分功能说明
DR_FP16MUL_D1	128	输入	编号 19，具体解释见本部分功能说明
DR_FP16MUL_D2	128	输入	编号 20，具体解释见本部分功能说明
DR_FP16MUL_D3	128	输入	编号 21，具体解释见本部分功能说明
DR_FP16MUL_D4	128	输入	编号 22，具体解释见本部分功能说明
DR_FP16MUL_D5	128	输入	编号 23，具体解释见本部分功能说明

DR_FP16MUL_D6	128	输入	编号 24，具体解释见本部分功能说明
DR_FP16MUL_D7	128	输入	编号 25，具体解释见本部分功能说明
DR_FP32ADDTREE5TO1_D0	128	输入	编号 26，具体解释见本部分功能说明
DR_FP32ADDTREE5TO1_D1	128	输入	编号 27，具体解释见本部分功能说明
DR_FP32ADDTREE8TO1_D	128	输入	编号 28，具体解释见本部分功能说明
DR_SHIFTUP_D	128	输入	编号 29，具体解释见本部分功能说明
UR_RDATA_0	128	输入	编号 30，具体解释见本部分功能说明
UR_RDATA_1	128	输入	编号 31，具体解释见本部分功能说明
UR_RDATA_2	128	输入	编号 32，具体解释见本部分功能说明
UR_RDATA_3	128	输入	编号 33，具体解释见本部分功能说明
CRU_RTOVR		输入	代表 RtoVR 的上行指令寄存器，存储内容为下面介绍的微指令。

5.3.18.2 RtoVR 相关的寄存器

1）SMC[*]/IN*_PIPE：SMC*中用于 IN*的 Pipeline Register，128bit。

2）SMC[*]/OUT*_PIPE：SMC*中用于 OUT*的 Pipeline Register，128bit。

5.3.18.3 RtoVR 相关的设计参数

表 5- 64：RtoVR 相关设计参数表

名称	含义	V1.0 支持
IN_PIPE_STAGE_0	输入编号 0 的信号的输入 pipeline 级数	0 或 1
IN_PIPE_STAGE_1	输入编号 1 的信号的输入 pipeline 级数	0 或 1
……		
IN_PIPE_STAGE_63	输入编号 63 的信号的输入 pipeline 级数	0 或 1
OUT_PIPE_STAGE_0	输出编号 0 的信号的输入 pipeline 级数	0 或 1
OUT_PIPE_STAGE_1	输出编号 1 的信号的输入 pipeline 级数	0 或 1
……		
OUT_PIPE_STAGE_63	输出编号 63 的信号的输入 pipeline 级数	0 或 1
OUT_FROM_IN_EN_0	64bit 参数，输出编号 0 的信号的对所有输入源使能信号	所有可能的取值
OUT_FROM_IN_EN_1	64bit 参数，输出编号 1 的信号的对所有输入源使能信号	所有可能的取值
……		
OUT_FROM_IN_EN_63	64bit 参数，输出编号 63 的信号的对所有输入源使能信号	所有可能的取值

参考《附录 2 RtoVR 连接关系》

5.3.18.4 RtoVR 相关的微指令

表 5- 65：RtoVR 相关上行微指令表

名称	Bit 数	Bit 位	含义
mv_out0_vld	1	0	‘b1：mv_out0 有效 ‘b0：mv_out0 无效，相关逻辑 IDLE，目标寄存器不更新
mv_out0	6	按顺序排列	在 mv_out0_vld 有效的情况下，将 mv_out0 编码对应的 input 数据 move 到 out0
mv_out1_vld	1	按顺序排列	
mv_out1	6	按顺序排列	
……		按顺序排列	
mv_out39_vld	1	按顺序排列	
mv_out39	6	按顺序排列	

5.3.18.5 RtoVR 功能说明

in*_dff 为 input*（如 input0=DVR_INTADD_S0）寄存一级的输出

in*_comb = IN_PIPE_STAGE_* ? in*_dff : in*;

out*_dff 为 out*_comb*寄存一级的输出

out* = OUT_PIPE_STAGE_* ? out*_dff : out*_comb;

out*_comb = !mv_out*_vld ? 0 :

(mv_out* == 0 & OUT_FROM_IN_EN_*[0]) ? in0_comb :

(mv_out* == 1 & OUT_FROM_IN_EN_*[1]) ? in1_comb :

.....

(mv_out* == 63 & OUT_FROM_IN_EN_*[63]) ? in63_comb :

0;

由于 RtoVR 不会进行计算，为了节省信号数量，所以 RtoVR 不使用 SMI_ID 的功能

5.3.19 VIN0（Vector Input Pipeline Register 0）

5.3.19.1 VIN0 的接口信号

表 5- 66：VIN0 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_VIN0_D	128	输出（寄存器）	见寄存器说明
CRU_VIN0_O	1	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_VIN0_S	128	输入	代表 VIN0 的计算输入数据
CRU_VIN0	1	输入	代表 VIN0 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.19.2 VIN0 相关的寄存器

1) SMC[*]/DR_VIN0_D：代表编号为*的 SMC 中的 VIN0 的输出寄存器，128bit。

5.3.19.3 VIN0 相关的设计参数

表 5- 67：VIN0 相关设计参数表

名称	含义	V1.0 支持

5.3.19.4 VIN0 相关的微指令

表 5- 68：VIN0 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.19.5 VIN0 功能说明

VIN0 为向量计算输入的第 0 组 Pipeline Register

注意 VIN0 为模块例化名，设计建议取名 PR（Pipeline Register）

if vld:

SMC[SMC_ID]/DR_VIN0_D = SMC[SMC_ID]/DR_VIN0_S

5.3.20 VIN1（Vector Input Pipeline Register 1）

5.3.20.1 VIN1 的接口信号

表 5- 69：VIN1 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_VIN1_D	128	输出（寄存器）	见寄存器说明
CRU_VIN1_O	1	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_VIN1_S	128	输入	代表 VIN1 的计算输入数据
CRU_VIN1	1	输入	代表 VIN1 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.20.2 VIN1 相关的寄存器

1) SMC[*]/DR_VIN1_D：代表编号为*的 SMC 中的 VIN1 的输出寄存器，128bit。

5.3.20.3 VIN1 相关的设计参数

表 5- 70：VIN1 相关设计参数表

名称	含义	V1.0 支持

5.3.20.4 VIN1 相关的微指令

表 5- 71：VIN1 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效

			‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.20.5 VIN1 功能说明

VIN1 为向量计算输入的第 1 组 Pipeline Register

注意 VIN1 为模块例化名，设计建议取名 PR（Pipeline Register）

if vld:

SMC[SMC_ID]/DR_VIN1_D = SMC[SMC_ID]/DR_VIN1_S

5.3.21 VOUT0（Vector Output Pipeline Register 0）

5.3.21.1 VOUT0 的接口信号

表 5- 72：VOUT0 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_VOUT0_D	128	输出（寄存器）	见寄存器说明
CRU_VOUT0_O	1	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_VOUT0_S	128	输入	代表 VOUT0 的计算输入数据
CRU_VOUT0	1	输入	代表 VOUT0 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.21.2 VOUT0 相关的寄存器

1) SMC[*]/DR_VOUT0_D：代表编号为*的 SMC 中的 VOUT0 的输出寄存器，128bit。

5.3.21.3 VOUT0 相关的设计参数

表 5- 73：VOUT0 相关设计参数表

名称	含义	V1.0 支持

5.3.21.4 VOUT0 相关的微指令

表 5- 74：VOUT0 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	'b1：指令有效 'b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.21.5 VOUT0 功能说明

VOUT0 为向量计算输出的第 0 组 Pipeline Register

注意 VOUT0 为模块例化名，设计建议取名 PR（Pipeline Register）

if vld:

SMC[SMC_ID]/DR_VOUT0_D = SMC[SMC_ID]/DR_VOUT0_S

~~5.3.22 MIN0 (Matrix put Pipeline Register 0)~~

~~5.3.22.1 MIN0 的接口信号~~

~~表 5- 75：MIN0 的接口信号表~~

~~名称~~	~~Bit 数~~	~~输入 / 输出（寄存器）~~	~~含义~~
~~CLK~~	~~1~~	~~输入~~	~~时钟~~
~~RST_N~~	~~1~~	~~输入~~	~~低电平有效的复位信号~~
~~DR_MIN0_D~~	~~128~~	~~输出（寄存器）~~	~~见寄存器说明~~
~~CRU_MIN0_O~~	~~1~~	~~输出（寄存器）~~	~~见 SMC 寄存器中 CRU 的说明~~
~~DVR_MIN0_S~~	~~128~~	~~输入~~	~~代表 MIN0 的计算输入数据~~
~~CRU_MIN0~~	~~1~~	~~输入~~	~~代表 MIN0 的上行指令寄存器，存储内容为下面介绍的微指令。~~
~~SMC_ID~~	~~5~~	~~输入~~	~~本模块所在的 SMC 的 ID 编码~~

5.3.22.2 MIN0 相关的寄存器

2) SMC[*]/DR_MIN0_D：代表编号为*的 SMC 中的 MIN0 的输出寄存器，128bit。

5.3.22.3 MIN0 相关的设计参数

表 5- 76：MIN0 相关设计参数表

名称	含义	V1.0 支持

5.3.22.4 MIN0 相关的微指令

表 5- 77：MIN0 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	'b1：指令有效 'b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.22.5 MIN0 功能说明

MIN0 为矩阵计算输入的第 0 组 Pipeline Register

注意 MIN0 为模块例化名，设计建议取名 PR（Pipeline Register）

if vld:

SMC[SMC_ID]/DR_MIN0_D = SMC[SMC_ID]/DR_MIN0_S

5.3.23 MIN1（Matrix Input Pipeline Register 1）

5.3.23.1 MIN1 的接口信号

表 5- 78：MIN1 的接口信号

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_MIN1_D	128	输出（寄存器）	见寄存器说明
CRU_MIN1_O	1	输出（寄存器）	见 SMC 寄存器中 CRU 的说明

DVR_MIN1_S	128	输入	代表 MIN1 的计算输入数据
CRU_MIN1	1	输入	代表 MIN1 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.23.2 MIN1 相关的寄存器

3) SMC[*]/DR_MIN1_D：代表编号为*的 SMC 中的 MIN1 的输出寄存器，128bit。

5.3.23.3 MIN1 相关的设计参数

表 5－79：MIN1 相关设计参数表

名称	含义	V1.0 支持

5.3.23.4 MIN1 相关的微指令

表 5－80：MIN1 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	'b1：指令有效 'b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.23.5 MIN1 功能说明

MIN1 为矩阵计算输入的第 1 组 Pipeline Register

注意 MIN1 为模块例化名，设计建议取名 PR（Pipeline Register）

if vld:

SMC[SMC_ID]/DR_MIN1_D = SMC[SMC_ID]/DR_MIN1_S

5.3.24 MOUT0（Matrix Output Pipeline Register 0）

5.3.24.1 MOUT0 的接口信号

表 5－81：MOUT0 的接口信号表

名称	Bit 数	输入/输出（寄存	含义

		器)	
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
DR_MOUT0_D	128	输出（寄存器）	见寄存器说明
CRU_MOUT0_O	1	输出（寄存器）	见 SMC 寄存器中 CRU 的说明
DVR_MOUT0_S	128	输入	代表 MOUT0 的计算输入数据
CRU_MOUT0	1	输入	代表 MOUT0 的上行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.24.2 MOUT0 相关的寄存器

1) SMC[*]/DR_MOUT0_D：代表编号为*的 SMC 中的 MOUT0 的输出寄存器，128bit。

5.3.24.3 MOUT0 相关的设计参数

表 5- 82：MOUT0 相关设计参数表

名称	**含义**	**V1.0 支持**

5.3.24.4 MOUT0 相关的微指令

表 5- 83：MOUT0 相关上行微指令表

名称	**Bit 数**	**Bit 位**	**含义**
vld	1	0	'b1：指令有效 'b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

5.3.24.5 MOUT0 功能说明

MOUT0 为矩阵计算输出的第 0 组 Pipeline Register

注意 MOUT0 为模块例化名，设计建议取名 PR（Pipeline Register）

if vld:

SMC[SMC_ID]/DR_MOUT0_D = SMC[SMC_ID]/DR_MOUT0_S

5.3.25 DS（Data Shuffle）

5.3.25.1 DS 的接口信号

表 5- 84：DS 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
AXISM_M*_STB			AXI Stream Master 接口信号，与 STB 模块的 AXISM_S*相连 数据位宽为 PARAM_UR_BYTE_CNT
AXISM_S*_STB			AXI Stream Slave 接口信号，与 STB 模块的 AXISM_M*相连 数据位宽为 PARAM_UR_BYTE_CNT
AXISM_M*_LDB			AXI Stream Master 接口信号，与 LDB 模块的 AXISM_S*相连 数据位宽为 PARAM_UR_BYTE_CNT
AXISM_S*_LDB			AXI Stream Slave 接口信号，与 LDB 模块的 AXISM_M*相连 数据位宽为 PARAM_UR_BYTE_CNT
CRU_DS		输入	代表 DS 的上行指令寄存器，存储内容为下面介绍的微指令。
CRU_DS_O		输出（寄存器）	代表 DS 的上行指令寄存器，存储内容为下面介绍的微指令。
CRD_DS		输入	代表 DS 的下行指令寄存器，存储内容为下面介绍的微指令。
CRD_DS_O		输出（寄存器）	代表 DS 的下行指令寄存器，存储内容为下面介绍的微指令。
SMC_ID	5	输入	本模块所在的 SMC 的 ID 编码

5.3.25.2 DS 相关的寄存器

5.3.25.3 DS 相关的设计参数

表 5- 85：DS 相关设计参数表

名称	含义	V1.0 支持
PARAM_TRANSPOSE_SIZE	支持的 transpose 的矩阵的 dim0 和	32

	dim1 尺寸，基于 4bit	
PARAM_TRANPOSE_SRC	transpose 操作的数据来源	STB 的 UR 读数据
PARAM_TRANPOSE_DST	transpose 操作的数据目的地	STB 的 UR 写数据
PARAM_OUTSTANDING_INST_NUM	SMI 最多可以发出的在执行中的指令	4

5.3.25.4 DS 相关的微指令

表 5- 86：DS 相关上行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
op_type	1	按顺序排列	‘b0：reshape ‘b1：transpose
data_type	2	按顺序排列	在 reshape 和 transpose 中的操作类型 ‘b00：4bit ‘b01：8bit ‘b10：16bit ‘b11：32bit
r_in_cnt	8	按顺序排列	reshape 操作的时候，输入数据一行的数据量
r_out_cnt	8	按顺序排列	reshape 操作的时候，输出数据一行的数据量
t_dim0_cnt	5	按顺序排列	在 transpose 中 0 维度的数据量 如当本字段为 0 的时候，代表 dim0 cnt 为 1 当本字段为’ b11111 的时候，代表 dim0 cnt 为 32
t_dim1_cnt	5	按顺序排列	在 transpose 中 1 维度的数据量 如当本字段为 0 的时候，代表 dim1 cnt 为 1 当本字段为’ b11111 的时候，代表 dim1 cnt 为 32
inst_id	4	按顺序排列	指令编号 SMI 最多可以发出 PARAM_OUTSTANDING_INST_NUM 个在执行指令，之后 SMI 只有从下行的微指令中获得指令完成的指示，才能发出新的指令。指令编号是循环使用的。
smc_id_en	5	按顺序排列	smc_id_en 如果大于等于接口信号中的 SMC_ID，则当前的 SMC 的本模块被该指令使能

表 5- 87：DS 相关下行微指令表

名称	Bit 数	Bit 位	含义
vld	1	0	‘b1：指令有效 ‘b0：指令无效，相关逻辑 IDLE，目标寄存器不更新
done	1	按顺序排列	指令完成
inst_id	4	按顺序排列	完成的指令编号

5.3.25.5 DS 功能说明

1）Reshape：数据不变、数据量不变、数据顺序不变、但是具体摆放变化。约定，来自 STB 的数据，在同一个 AXISM cycle 中的数据对于 reshape 之前的坐标，在高维度（dim1）坐标相同；发送到 LDB 的数据，在同一个 AXISM cycle 中的数据的坐标在变形后的高维度（dim1）坐标相同。

参考：

https://docs.pytorch.org/docs/stable/generated/torch.reshape.html#torch.reshape

2）Transpose：数据不变、数据量不变、数据顺序转置。PARAM_TRANSPOSE_SIZE 设置不大于 UR 一行的数据量，且当前设计仅支持 UR 到 UR 的 transpose。约定，来自 STB 的数据，在同一个 AXISM cycle 中的数据的坐标在高维度（dim1）坐标相同；发送到 LDB 的数据，在同一个 AXISM cycle 中的数据的坐标在转置后的高维度（dim1，原 dim0）坐标相同。

参考：

https://docs.pytorch.org/docs/stable/generated/torch.transpose.html

3）AXI Stream 数据的流向

针对 tid

‘b00：直接去往 LDB（通过 DS，但是只做 pipeline）

‘b01：通过 DS 的逻辑再去往 LDB

‘b10：直接去往 STBAXI（通过 DS，但是只做 pipeline）

‘b11：通过 DS 去往 STBAXI

4）注意 STB、LDB、DS 模块都具有 outstanding 指令功能，避免数据通道冲突应由软件而非硬件负责。

删除[SYT]: 4)

删除[SYT]: 有

5）对于下行微指令：

- 每一个 SMC 只有接到下游的 SMC 发来指令，并且下游指令对应的指令编号在本 SMC 也已经完成的情况下才会向上游发出指令。特例是最后一个 SMC（基于 SMC_ID 为全 1 判断）不需要考虑下游。
- 在发送指令的时候，发向上游的指令中的 err 是本地的 err 与来自下游的指令中的 err 的或。

5.4 SMI 架构

5.4.1 SMI 接口

5.4.1.1 SMI 接口信号说明

表 5- 88：SMI 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
AXIS*	*		一组 AXI Slave interface 数据位宽 128bit 地址位宽 32bit 注意：实现中采用 Vendor S 的 IP，AXI Slave 转为 GIF Slave interface，再做后续处理。原因是未来版本有可能直接采用 GIF 连接处理器做紧耦合实现。
INTR	1	输出（寄存器）	中断 上升沿代表一次中断，自动归零
CRU_O	*	输出（寄存器）	为 { CRU_INTADD_O, CRU_INT8toFP16_O, CRU_FP16MUL_S0123_O, CRU_FP16MUL_S4567_O, CRU_FP16MUL_O, CRU_FP32ADDTREE5TO1_O, CRU_FP32ADDTREE8TO1_O, CRU_FPMUL_O, CRU_FPADD_O, CRU_FPtoFP_O, CRU_INTtoINT_O, CRU_FPtoINT_O, CRU_INTtoFP_O, CRU_LOGICAL_O, CRU_SHIFTUP_O, CRU_SHIFTDN_O, CRU_STB_O, CRU_STBAXI_O, CRU_LDB_O, CRU_LDBAXI_O, CRU_UR_O,

			CRU_RtoVR_O, CRU_VIN0_O, CRU_VIN1_O, CRU_VOUT0_O, CRU_DS }

CRD	*	输入	为
			{
			CRD_SHIFTDN,
			CRD_STB,
			CRD_STBAXI,
			CRD_LDB,
			CRD_LDBAXI,
			CRD_DS
			}

5.4.1.2 SMI 接口与内部模块的连接说明

表 5- 89：SMI 的接口信号表

名称	含义	连接说明
CLK	时钟	时钟
RST_N	低电平有效的复位信号	低电平有效的复位信号
AXIS*	一组 AXI Slave interface 数据位宽 128bit 地址位宽 32bit 注意：实现中采用 Vendor S 的 IP，AXI Slave 转为 GIF Slave interface，再做后续处理。原因是未来版本有可能直接采用 GIF 连接处理器做紧耦合实现。	信号转为 GIF 接口后，一个分支是访问 SMI 控制寄存器；另一个分支是访问 SMI 指令寄存器。 SMI 指令寄存器由众多 SMI-IQ 组成。
INTR	中断 上升沿代表一次中断，自动归零	连接至 SMI 控制寄存器
CRU_O	为 { CRU_INTADD_O, CRU_INT8toFP16_O, CRU_FP16MUL_O, CRU_FP32ADDTREE5TO1_O, CRU_FP32ADDTREE8TO1_O, CRU_FPMUL_O, CRU_FPADD_O,	每一组子信号连接至一个 SMI-IQ 模块

	CRU_FPtoFP_O, CRU_INTtoINT_O, CRU_FPtoINT_O, CRU_INTtoFP_O, CRU_LOGICAL_O, CRU_SHIFTUP_O, CRU_SHIFTDN_O, CRU_STB_O, CRU_STBAXI_O, CRU_LDB_O, CRU_LDBAXI_O, CRU_UR_O, CRU_RtoVR_O, CRU_VIN0_O, CRU_VIN1_O, CRU_VOUT0_O, CRU_DS }	

CRD	为
	{
	CRD_SHIFTDN,
	CRD_STB,
	CRD_STBAXI,
	CRD_LDB,
	CRD_LDBAXI,
	CRD_DS
	}
		每一组子信号连接到一个 SMI-IS 模块

5.4.2 SMI 寄存器

5.4.2.1 SMI 控制寄存器

SMI 的控制寄存器的偏移地址为 0x0。

表 5- 90：SMI 控制寄存器表

偏移地址	寄存器	BIT 位	子寄存器	读写属性	默认值	含义
0x0	cfg	0	cfg_en	RW	0	模块使能
		1	cfg_rst	RWC	0	模块复位 写入 1 的时候模块复位清零，包含本寄存器
0x4	int	0	intr	RO	0	中断标志位，标志是否还有未处理的中断

						只有当 intid_hw=intid_sw 的时候本字段为 0，其他的时候为 1 目前的中断源有两个： 1）来自 SMC 中 STB 和 LDB 的 err 2）控制寄存器的 smi_iq_done
		1	int_err	RWC	0	如果出现中断队列溢出的情况，此字段置为 1，软件写 1 后清零 目前的错误源有一个： 1）来自 SMC 中 STB 和 LDB 的 err
		7	int_clr	RWU	0	软件向本字段写 1 后，所有的 int*寄存器清零，包含本字段
		15:8	intid_hw	RO	0	硬件根据内部中断更新此字段相关的硬件 ID 寄存器（intid_hw），每次中断硬件 ID 寄存器自加一，加到最大值后循环。
		23:16	intid_sw	RW	0	软件收到中断后，读取 intr 和 intid_hw，进行中断处理后将读取到的的最后一个 intid_hw 写回到 intid_sw。
0x8	smc	7:0	smc_id	W	0	AIACC 初始化的时候，软件向本字段连续写 PARAM_SMC_NUM 次，依次是从离 SMI 最远的 SMC 到离 SMI 最近的 SMC 的 smc_id。
0x10	smi_iq_en0	31:0	smi_iq_en0	RW	0	对应编号为 0-31 的 SMI_IQ 是否使能。
0x14	smi_iq_en1	31:0	smi_iq_en1	RW	0	对应编号为 32-63 的 SMI_IQ 是否使能。
0x18	smi_iq_en2	31:0	smi_iq_en2	RW	0	对应编号为 64-95 的 SMI_IQ 是否使能。
0x1C	smi_iq_en3	31:0	smi_iq_en3	RW	0	对应编号为 96-127 的 SMI_IQ 是否使能。
0x20	smi_iq_start	9:8	smi_iq_sid	RW	0	对应所有 SMI_IQ 的 RDI_IQSID，非直接驱动。
		0	smi_iq_start	WU	0	对应所有 SMI_IQ 的 RDI_START，非直接驱动。软件写入 1 之后，下一个周期硬件将其清为 0。
0x24	smi_iq_done	9:8	smi_iq_eid	RW	0	对应所有 SMI_IQ 的 RDI_IQEID，非直接驱动。
		0	smi_iq_done	RW	0	对应所有 SMI_IQ 的 RDI_DONE，非直接驱动。

5.4.2.2 SMI 指令寄存器

SMI 指令寄存器的偏移地址为 0x1000，即 4KB。

SMI_IQ 中存储指令的 SRAM 都映射为指令寄存器，具体信息由《AIACC 指令架构 5.4.2.2 SMI 指令寄存器》说明，包含起始地址和各个 SMI_IQ 队列的映射地址。

5.4.3 SMI 相关的设计参数

表 5- 91：SMI 相关设计参数表

名称	含义	V1.0 支持

5.4.4 SMI 功能说明

5.4.4.1 SMI 相关的功能说明

除非特殊说明，否则所有 SMI 寄存器复位后的默认值为 0。

5.4.4.2 SMI 控制寄存器补充说明

- 注意上电之后需要配置 cfg_en 为 1 以便使能 AIACC。
- 如果 AIACC 出现异常，可以通过 cfg_rst 进行复位。
- 参考 SMC 中 SMC_ID 的功能描述，各个 SMC 的 SMC_ID 寄存器形成了一串 shift 寄存器，AIACC 初始化的时候，软件需要向本节的 smc 寄存器写入 PARAM_SMC_NUM 个值，这些值会以 shift 的方式对各个 SMC 的 SMC_ID 寄存器进行初始化。

5.4.4.3 SMI 的 AXI Slave 接口说明

1) 写入协议

- 所有的控制存器都是真正的 register，支持 burst 写，但是要遵循各寄存器的要求进行操作。
- 所有的指令队列的地址空间都映射为指令寄存器，可以支持各种写入方式：乱序写入，写入后修改，SINGLE or BURST 写入，分多笔写入。

2) 启动和完成流程：

- 启动初始化之后，软件通过指令寄存器对 SMI_IQ 的指令队列进行写入。只要软件不配置控制寄存器中的 smi_iq_start，写入的指令就不会生效和执行。
- 写完指令之后，顺序写 smi_iq_en*和 smi_iq_sid，最后写入 smi_iq_start。
- SMI 的控制寄存器中存在部分胶水逻辑完成以下功能：
  - 维护着一个 RDI_START 管理队列，深度为 4。
  - 软件配置按顺序写完 smi_iq_en*、smi_iq_sid 和 smi_iq_start 之后，胶水逻辑会记录这个 smi_iq_sid 对应的 smi_iq_en*，在所有 SMI_IQ 空闲（之前发出的 RDI_START 返回 RDI_DONE 之后）的时候，向所有 SMI_IQ 发出 RDI_START 和 RDI_SID；如果 SMI_IQ 不空闲，则等待 SMI_IQ 空闲再发出。
  - 注意对于发出指令操作，仍然需要保存 smi_iq_en*和 smi_iq_sid 信息。
- 当 SMI_IQ 返回 RDI_DONE 和 RDI_EID 的时候（多个 SMI_IQ 未必同一时间返回，可能有先后），胶水逻辑执行以下功能：
  - 维护着一个 RDI_DONE 管理队列，深度为 4。
  - 将返回的 RDI_DONE 和 RDI_EID 保存到队列中（具体的存储的信息可以优化，但是要实现类似功能）
  - 胶水逻辑检查存储的 RDI_DONE 和 RDI_EID 与 RDI_START 管理队列中的 smi_iq_en*和 smi_iq_sid 是否相同。当 smi_iq_sid 中每一个 smi_iq_en 为 1 的 SMI_IQ 都返回了正确的 RDI_EID 和 RDI_DONE 的时候，意味着这个指令完成。此时将 smi_iq_done 置为 1，将 smi_iq_eid 置为存储的 smi_iq_sid。
  - smi_iq_done 会产生中断，因为设计支持中断队列，不需要担心前面被覆盖的中断相关信息丢失（smi_iq__done 和 smi_iq_eid）。

5.5 SMI 微架构和微指令

5.5.1 SMI_IQ（Instruction Queue）

5.5.1.1 SMI_IQ 的接口信号

表 5- 92：SMI_IQ 的接口信号表

删除[SYT]: 1

名称	Bit 数	输入 / 输出（寄存器）	含义	V1.0 是否支持

CLK	1	输入	时钟	是
RST_N	1	输入	低电平有效的复位信号	是
INST	*	输出（寄存器）	输出到 SMC 某个模块的指令，INST={CODE，ADDR，DATA}	是
INST_KEEP	1	输出（寄存器）	输出指令不变，当 INST_VLD 为 1 时，如果当前 INST 与上一个 INST_VLD 为 1 时候的 INST 完全相同的话，INST_KEEP 为 1，其他时候为 0	是
INST_VLD	1	输出（寄存器）	输出指令有效	是
INST_ID	6	输出（寄存器）	指令队列的编号，等于 PARA_IQ_ID	是
INST_RDY	1	输出（寄存器）	指令队列就绪。 对于没有附加地址或者附加数据的指令队列来说，收到 RDI_START 后，INST_RDY 就为高，直到发出 RDI_DONE；对于具有附加地址或者附加数据的指令队列，只有收到 RDI_START 后，且附加地址队列和附加数据队列不为空的时候，INST_RDY 才为高，直到发出 RDI_DONE。	是
INST_RDY_IN	*	输入	所有 SMI_IQ 的 INST_RDY 输出组成的 BUS，再输入给每一个 SMI_IQ，Bit 数为 SMI_IQ 的数量	是
INST_JUMP_IN	32	输入	用于 JUMP 指令时进行条件比较的输入数据。	是
RDI_START	1	输入	指示可以开始发射下一个子队列指令	是
RDI_IQSID	2	输入	指示可以开始发射的子队列编号	是
RDI_DONE	1	输出（寄存器）	反馈子队列指令发射结束	是
RDI_IQEID	2	输出	反馈发射结束的子队列指令编号	是
WRI_DATA	*	输入	写入指令队列的数据 该信号来自 AXI Slave 总线	是
WRI_VLD	1	输入	写入指令有效 该信号来自 AXI Slave 总线	是
WRI_ADDR	*	输入	写入指令队列的地址，地址高两位指示子队列编号，地址低位的位数取决于每个子队列的深度 该信号来自 AXI Slave 总线的译码	是
WRA_DATA	*	输入	写入地址队列的数据	否
WRA_VLD	1	输入	写入地址有效	否
WRA_START	1	输入	写入地址的开始标志位，当 WRA_VLD 为 1 且 WRA_START 为 1 的时候，将写入指针指向 WRD_QID 的地址 0，随后当每个 WRA_VLD 为 1 且 WRA_START 为 0 的时钟周期，写入指针自加 1。	否
WRA_END	1	输入	写入地址的结束位，WRA_VLD 为 1 且 WRA_END	否

			为 1 的时候，代表对当前子队列的写入结束。	
WRA_IQSID	2	输入	写入地址对应的地址子队列编号，每个地址队列模块中包含多个子队列，当前设计是四个子队列，编号为 0、1、2、3	否
WRA_FULL	1	输出	地址队列满信号	否
WRD_DATA	*	输入	写入数据队列的数据	是
WRD_VLD	1	输入	写入数据有效	是
WRD_START	1	输入	写入数据的开始标志位，当 WRD_VLD 为 1 且 WRD_START 为 1 的时候，将写入指针指向 WRD_QID 的地址 0，随后当每个 WRD_VLD 为 1 且 WRD_START 为 0 的时钟周期，写入指针自加 1。	是
WRD_END	1	输入	写入数据的结束位，WRD_VLD 为 1 且 WRD_END 为 1 的时候，代表对当前子队列的写入结束。	是
WRD_IQSID	2	输入	写入数据对应的数据子队列编号，每个地址队列模块中包含多个子队列，当前设计是四个子队列，编号为 0、1、2、3	是
WRD_FULL	1	输出	数据队列满信号	是

5.5.1.2 SMI_IQ 相关的设计参数

表 5- 93：SMI_IQ 相关设计参数表

删除[SYT]：RtoVR

名称	含义	V1.0 支持
PARA_IQ_ID	指令槽编号标识	
EN_EMBED	是否有 EMBED 字段	
BITS_EMBED	EMBED 字段的 BIT 数	’d3
EN_EMBED_END	是否有 EMBED_END 字段	
BITS_EMBED_END	EMBED_END 字段的 BIT 数	’d6
EN_LOOPS	是否有 LOOPS 字段	
BITS_LOOPS	LOOPS 字段的 BIT 数	’d10
EN_COND	是否有 COND 字段	
BITS_COND	COND 字段的 BIT 数	’d7
EN_DLY	是否有 DLY 字段	
BITS_DLY	DLY 字段的 BIT 数	‘d3
EN_INC	是否有 INC 字段	
BITS_INC	INC 字段的 BIT 数	LDB、LDBAXI、STB、STBAXI 对应的 SMI-IQ 为’d13，其他为‘d4
EN_INC_EMBED	是否有 INC_EMBED 字段	

带格式表格[孙浩南]

BITS_INC_EMBED	INC 字段的 BIT 数	'd6
MIN_HIGH	MIN 字段的高位，如果 EN_INC 就一定有 MIN_HIGH	
MIN_LOW	MIN 字段的地位，如果 EN_INC 就一定有 MIN_LOW	
BITS_CODE	CODE 字段的 BIT 数	
EXTRA_ADDR	是否有额外的地址队列，如果这个字段为无效，则没有 WRA*的接口	
EXTRA_DATA	是否有额外的数据队列，如果这个字段为无效，则没有 WRD*的接口	
INSTQ_DEPTH	指令队列深度	32
ADDRQ_DEPTH	地址队列深度	8
DATAQ_DEPTH	数据队列深度	8

5.5.1.3 SMI_IQ 微指令格式

这里的微指令格式为写入指令子队列（基于 SRAM 实现）中每一行的指令格式

表 5- 94：SMI_IQ 微指令格式表

字段	说明
VLD	是否是有效指令（有效指令是指非 NOP 指令）； 如果无效视为 NOP，NOP 指令不支持 DLY，但是需要考虑 LOOPS、COND，NOP 的时候 CODE 无意义，此时设置发送给 SMC 的微指令的 vld=0，其他的信号与前一拍保持不变。
JUMP	是否是跳转指令，跳转指令不支持 LOOPS、DLY，但是需要考虑 COND
EMBED	嵌套 LOOPS 层数标识， 如果是 0，代表是嵌套循环最内层； 如果是 1，代表第二层； 如果是 2，代表第三层； …… 如果是 6，代表第七层； 7 以及以上为无效编码。
EMBED_END	当 EMBED!=0 的时候无效； 当 EMBED=0 的时候，标志本条指令结束后，某一层嵌套循环是否结束。 EMBED_END[0]标识是否是 EMBED=1 的循环体的结束…… EMBED_END[5]标识是否是 EMBED=6 的循环体的结束。
LOOPS	如果 EN_LOOPS 有效则有本字段，其 BIT 数由 BITS_LOOPS 定义； 代表一条指令循环发射多次。 0 代表只执行 1 次，N 代表执行 N+1 次。
COND	如果 EN_COND 有效则有本字段，其 BIT 数由 BITS_COND 定义； 每次发射等待其数值所对应编号的指令槽（SMI_IQ 或者 SMI_IR）,即等待 INST_RDY_IN 中对

	应的 Bit 就绪。 软件需要合理配置所有 SMI_IQ 的 COND 字段，以保证不会发生死锁。
DLY	如果 EN_DLY 有效则有本字段，其 BIT 数由 BITS_DLY 定义； 每次发射所有条件就绪后发射到一个 DLY 数值对应的延迟线，等到移位到延迟线末端才真正发出。
INC	如果 EN_INC 有效则有本字段，其 BIT 数由 BITS_INC 定义； 每次循环的时候指令发射部分的某些位自加或者自减，只有涉及地址的指令或者寄存器选择的指令才能有本字段，INC 视为一个带符号整数，为自变量。 当使用嵌套循环的时候，INC 这个字段对应到 INC_i，i 来着 EMBED 字段
INC_EMBED	当 EN_INC 有效则有本字段，其 BIT 数为 6bit；从低到高代表 EMBED=1~6 层的 INC 是否要在 EMBED=0 的指令中进行叠加，仅当 EMBED=0 时，本字段有意义。 一般在 CODE 中有地址信息或者有源、目的寄存器的情况下使用。 发出的数值 =CODE[INC_HIGH:INC_LOW]+INC_0*$LOOP_CNT_0$+$\sum_{i=1}^{6}(INC_i * LOOP_CNT_i * INC_EMBED[i-1])$，其中： ● CODE[INC_HIGH:INC_LOW]是 BASE ● INC_i 是 EMBED=i 的嵌套循环定义的 INC ● $LOOP_CNT_i$ 是 EMBED=i 的嵌套循环的循环次数自变量（0<=LOOP_CNT<=LOOP） ● INC_EMBED[i-1]是 INC_EMBED 的第 i 层是否有效
CODE	可能有多种含义： 1) 如果不是以下的特殊情况，则是发给 SMC 的指令码；如果 EN_INC=1 且 EMBED=0 的时候，设计参数 INC_HIGH:INC_LOW 定义了 CODE 中哪些数据为基础值（记为 BASE）。 2) 如果 JUMP=1 的话， ● CODE[LSB+42]代表[LSB+41:LSB+40]是有符号数，还是无符号数，对于浮点数和整数通用 ● CODE[LSB+41:LSB+40]是用于判断跳转的条件： ■ 2’b00 表示当输入数据（INST_JUMP_IN）大于期望数据（CODE[LSB+39:LSB+8]）进行跳转，否则顺序执行下一条； ■ 2’b01 表示当输入数据（INST_JUMP_IN）等于期望数据（CODE[LSB+39:LSB+8]）进行跳转，否则顺序执行下一条； ■ 2’b10 表示当输入数据（INST_JUMP_IN）小于期望数据（CODE[LSB+39:LSB+8]）进行跳转，否则顺序执行下一条； ■ 2’b11 表示该指令是队列的最后一个可执行指令，注意由于有 JUMP 的存在队列可能存在多个最后一个可执行指令。一旦遇到一个该指令，就结束这一个 SMI_IQ 指令队列的执行。 ● CODE[LSB+39:LSB+8]是用于判断跳转条件期望数据； ● CODE[LSB+7:LSB]是跳转偏移量（视为一个有符号数）。 3) 如果 EMBED!=0 的话，BITS_CODE-1 位代表 CODE 是否被使用承载额外信息，1 代表是，0 代表否。 ● BITS_CODE-2:BITS_CODE-4=000，代表激活列嵌套循环指令。包含激活列相关的扩展信

删除[SYT]: （即 INST_RDY_IN 中对应的 Bit）

息；包含 LF_PAD（2D 图像的左 Padding 行数，4bit）、RT_PAD（2D 图像的右 Padding 行数，4bit），MIN（计算起始列号，12bit，从 0 开始编码），MAX（整图结束列号，12bit）。此时 INC 代表列 Stride，目前仅支持 2 的幂；LOOPS 达标对应产生的输出列数。

- BITS_CODE-2:BITS_CODE-4=001，代表激活行嵌套循环指令。包含激活行相关的扩展信息；包含额外信息包含 UP_PAD（2D 图像的上 Padding 行数，4bit）、DN_PAD（2D 图像的下 Padding 行数，4bit），MIN（计算起始行号，12bit，从 0 开始编码），MAX（整图结束列数，12bit）。此时 INC 字段代表行 Stride，目前仅支持 2 的幂；LOOPS 代表对应产生输出的行数。
- BITS_CODE-2:BITS_CODE-4=010，代表权重列循环指令，当前并未定义额外字段。此时 INC 代表列 Dilation（除最后一列外所有列插 INC_1 个“0”值列）；LOOPS 代表权重列数。
- BITS_CODE-2:BITS_CODE-4=011，代表权重行循环指令，当前并未定义额外字段。此时 INC 代表行 Dilation（除最后一行外所有列插 INC_1 个“0”值行）；LOOPS 代表权重行数。
- BITS_CODE-2:BITS_CODE-4=100，代表包含的数值信息，读取数据精度和 PADING 模型以及 PADDIN 数值。
  - BITS_CODE-5:BITS_CODE-6=00，代表读取数据为 2bit；
  - BITS_CODE-5:BITS_CODE-6=01，代表读取数据为 4bit；
  - BITS_CODE-5:BITS_CODE-6=10，代表读取数据为 8bit；
  - BITS_CODE-5:BITS_CODE-6=11，代表读取数据为 16bit。
  - BITS_CODE-7:BITS_CODE-8=00，代表 Padding 模式为 Constant，之后紧接着为 Padding 的具体数值，BITS 根据 BITS_CODE_5:BITS_CODE_6 定义；
  - BITS_CODE-7:BITS_CODE-8=01，代表 Padding 模式为 Reflection；
  - BITS_CODE-7:BITS_CODE-8=10，代表 Padding 模式为 Replication；
  - BITS_CODE-7:BITS_CODE-8=11，代表 Padding 模式为 Circular。

5.5.1.4 SMI_IQ 功能说明

1) 子队列的管理

- 系统复位后，认为所有的子队列为空；
- WRI 从编号 x 开始写子队列，写完之后发送对应编号的 RDI_START；
- WRI 写完编号 x 的子队列之后，可以写其他编号的子队列；
- WRI 不能写再编号 x 的子队列，直到收到对应编号的 RDI_DONE
- 本模块按照收到 RDI_START 的顺序来执行对指令队列的操作。
- 对于 V1.0 设计，仅支持基于 Queue 的工作模式，即写队列的顺序按照 0、1、2、3、0 循环，读队列的顺序

也是如此，仅通过前一个队列在执行的时候，预先写入下一个乃至多个队列的指令来隐藏指令配置的开销，从而提升性能。

- 对于 V1.0 设计，不会在前序 RDI_START 对应的 RDI_DONE 返回之前发出后续的 RDI_START。

2) SMI_IQ 对应控制的模块说明

- 一般情况下 SMI_IQ 控制 SMC 中的一个模块（如 FPADD）
- 但是对于 UR（包含 W0~W3、R0~R3），需要对应 8 个 SMI_IQ 模块
- RtoVR 需要 64 个 SMI_IQ 模块

3) 嵌套循环的使用

- 支持多层嵌套；
- 最内层嵌套 EMBED 字段为 0；只有 EMBED=0 时，其 CODE 为向 SMC 发射的指令；EMBED=0 的指令的 LOOPS 可以为 0。
- EMBED 值较高的循环体内可以包含多个 EMBED 值较低的循环体；存在 EMBED 值较高的指令紧跟着 EMBED=0 的指令的情况；
- EMBED_END 标识循环嵌套的各层循环体是否结束，某一层循环体一旦结束就需要重新有指令设置这一层循环才会再次使能该层嵌套循环。
- 各外层嵌套循环的附加值（基础值加上若干次 INC 值）按照 INC_EMBED 的指示加在最内层的基础值上。
- 当 IIC 读取 EMBED!=0 指令的时候只做内部控制寄存器的初始化工作，只有读到 EMBED=0 的指令的下一个周期才开始发射指令到 DLY 队列中。

举例如下面的情况：

- EMBED=3
  - EMBED=2
    - EMBED=0（此时实际为 3 层循环，但是使用 3、2、0 这三层）
    - EMBED=1

- EMBED=0（此时为 4 层循环）
- EMBED=0、EMBED_END=1（此时为 4 层循环、此条指令为 1、0 层循环结束）

- EMBED=1
  - EMBED=0（此时为 4 层循环）
  - EMBED=0、EMBED_END=111（此时为 4 层循环、此条指令为所有循环结束）

4) 对于 COND 和 DLY 的使用场景的说明

- DLY 用于调整不同的 SMI_IQ 之间短暂的时序差别。如一个向量加法计算涉及多个 SMI_IQ，数据在这些 SMI_IQ 对应的模块之间流动需要有严格的时序关系，此时可以考虑将生成的指令分为两个阶段：第一个阶段是不考虑时序关系，仅考虑每个模块的工作，写出对应的指令；第二个阶段加入 DLY 信息，就使得这些指令能够有序的启动并协作完成算子。
- DLY 需要设计对应的延迟线队列，当这个延迟很长或者延迟不确定的时候就很难用 DLY 的方式进行支持。此时会改为用 COND。延迟很长的例子是做矩阵乘法时从 UR 读取需要广播的数据，往往对应几十个时钟周期；延迟不确定的例子是广播的数据来自 DRAM。这个时候让需要读取广播数据的 SMI_IQ 先行执行，其他模块用 COND 等待广播数据取回、数据整理完毕、进入对应 SMI_IQ 的数据队列中，再开始同步执行。

5.5.2 SMI_IR（Instruction Recycle）

5.5.2.1 SMI_IR 的接口信号

表 5- 95：SMI_IR 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
CRD_I	5	输入	每一组 CRD_I 包含{done，inst_id}两个信号
CRU_I	*	输入	只关注每一个 CRU_I 中的{vld，inst_id}两个信号
IR_INST_RDY	1	输出	SMI_IR 指示指令就绪，所有的 SMI_IR 的 IR_INST_RDY 输出和所有的 SMI_IQ 的 INST_RDY 输出一起形成 INST_RDY 总线，返回给所有的 SMI_IQ 的 INST_RDY_IN。
ERR	1	输出	出错

删除[SYT]: IS_INST_RDY

5.5.2.2 SMI_IR 相关的设计参数

表 5- 96：SMI_IR 相关设计参数表

名称	含义	V1.0 支持
PARAM_OUTSTANDING_INST_NUM	SMI 最多可以发出的在执行中的指令	4

5.5.2.3 SMI_IR 微指令格式

本模块无微指令。

5.5.2.4 SMI_IR 功能说明

SMI_IR 模块的功能是接受一部分来自 SMC 的 CRD 指令.

CRD 指令分为两类：

- 一类是来自 SHIFTDN，主要是数据信号
- 一类是来自以下模块

CRD_STB,

CRD_STBAXI，

CRD_LDB，

CRD_LDBAXI，

CRD_DS

主要是用于回收 outstanding 指令的完成情况。

SMI_IR 仅用于连接第二类 CRD 信号。

在 SMI 中需要例化 5 个 SMI_IR，分别对应上述 5 组 CRD 信号。

SMI_IR 中维护 outstanding_inst_num，reset 之后初始值为 0

If(CRU_I.vld & CRD_I.done)

    outstanding_inst_num = outstanding_inst_num;

If(CRU_I.vld & !CRD_I.done)

```
    outstanding_inst_num = outstanding_inst_num+1;
If(!CRU_I.vld & CRD_I.done)
    outstanding_inst_num = outstanding_inst_num-1;

IR_INST_RDY = (outstanding_inst_num <= PARAM_OUTSTANDING_INST_NUM)?1:0;
```

删除[SYT]: IS_INST_RDY

报错情况：

- 每个 CRU 发出的 inst_id 将被 SMI_IR 记录下来，如果 CRD 返回的 inst_id 与目前保存的所有记录都不一致报错。
- 如果记录已经为空，仍然有反馈 CRD，报错。
- 如果出现 IR_INST_RDY 为 0，CRU_I 仍然 vld 的情况报错。

删除[SYT]: IS_INST_RDY

5.5.3 SMI_DS_T（Data Shuffle_Transpose）

5.5.3.1 SMI_DS_T 的接口信号

表 5- 97：SMI_DS_T 的接口信号表

名称	Bit 数	输入 / 输出（寄存器）	含义
CLK	1	输入	时钟
RST_N	1	输入	低电平有效的复位信号
CRD_SHIFTDN	129	输入	输入数据
DATA_OUT	128	输出（寄存器）	输出数据
DATA_OUT_VLD	1	输出（寄存器）	输出数据有效
CRI	*	输入	输入的指令
INST_DONE	1	输出（寄存器）	指示完成一个指令

5.5.3.2 SMI_DS_T 相关的设计参数

表 5- 98：SMI_DS_T 相关设计参数表

名称	含义	V1.0 支持
PARAM_IN_LINE_BYTE_CN	内部存储的一行的 BYTE CNT，DATA_IN 的数据会写	PARAM_UR_BYTE_CN

T	到一行中，填满一行再填第二行	T
PARAM_IN_LINE_NUM	内部存储的行数	PARAM_SMC_NUM
PARAM_INST_NUM	内部支持 PARAM_IN_LINE_BYTE_CNT*PARAM_IN_LINE_NUM 这样的存储块的块数，也意味着同时能处理的指令的数量	4

5.5.3.3 SMI_DS_T 微指令格式

表 5- 99：SMI_DS_T 相关微指令表

名称	Bit 数	Bit 位	含义
vld	1		‘b1：指令有效 ‘b0：指令无效 每个指令处理一个
intlv_en	1		指示当前指令是否开启 interleave 模式
intlv_grp	1		指示当前指令的 interleave 组编号， ‘b0：第一组 ‘b1：第二组 目前每个组为两个指令，具有相同的组编号的两个指令属于同一组，且 intlv_en 必定为 1，intlv_in 和 intlv_out 至少一个为 1，可以两个都为 1。
intlv_in	1		指示输入是否 interleave
intlv_out	1		指示输出是否 interleave

5.5.3.4 SMI_DS_T 功能说明

设计前提：

- 当前仅考虑将一个存储于多个 SMC 的数据读取回 SMI 的功能需求。
- 当前不考虑一个指令从多个 LANE 分 散读一个数据的方案，仅考虑一个指令从多个 LANE 分别读一个 UR 行数据的方案。

功能说明：

- 针对每一组数据获取指令获取的数据块，进行 Transpose 后发出；
- 从 SMC 读，写到 SMC 的情况

- 这里每一组数据获取指令是指对所有 SMC 进行 UR 读，读取之后通过 SHIFTDN 传回给 SMI；
- 每一组数据获取指令从每个 SMC 的 UR 读取一行数据，包含 PARAM_UR_BYTE_CNT 个字节，共 PARAM_SMC_NUM 行；
- 一个 SMI_DS_T 指令将接收上述数据 Tile 转置为 PARAM_UR_BYTE_CNT 行，PARAM_SMC_NUM 列的 Tile，再发出，发出的时候一行 PARAM_SMC_NUM 的数据分为多个 PARAM_UR_BYTE_CNT 发出。
- 本模块在数据接收的时候不反压，输出发出的时候不允许对方反压。
- 本模块可以同时支持四个在运行的 SMI_DS_T 指令，其中每两个指令前后两个时钟周期启动，数据间插读取；前两个指令完成数据读取后立即开始数据输出。
- 输出的时候也是是间插输出，第一个指令的一行（转置后）发出后就发第二个指令的一行，再发第一个指令的下一行；也可以按照指令顺序输出，发完第一个指令的所有数据再发第二个指令的数据。

- 从 SMC 读，写到 DRAM 情况（V1.0 不支持）
  - 待定，与 LSU 一起完善
- 从 DRAM 读，写到 SMC 情况（V1.0 不支持）
  - 待定，与 LSU 一起完善
- 从 DRAM 读，写到 DRAM 情况（V1.0 不支持）
  - 待定，与 LSU 一起完善

SMI_DS_T 的输出信号，送到用于 SHIFTUP 对应的 SMI_IQ，这个 SMI_IQ 具有 EXTRA_DATA。

通过 UR read —— SHIFTDN —— SMI_DS_T —— SHIFTUP —— UR write 形成了数据整理的完整 datapath。

5.5.4 SMI_DS_SG（Data Shuffle_Scatter Gather）（待完善）（V1.0 不支持）

5.5.5 SMI_DS_RD（Data Shuffle_Reduce）（SQZ、UNSQZ、None Zero、Masked Select）（待完善）（V1.0 不支持）

5.6 LSU 架构（V1.0 不支持）

5.7 LSU 微架构和微指令（V1.0 不支持）

5.8 MID 架构（V1.0 不支持）

5.9 MID 微架构和微指令（V1.0 不支持）

6 基于宏指令视角的编程模型

本章将以宏指令视角分别介绍 AIACC 编程模型，将从支持的数据存储方式、数据类型方面进行介绍，至于宏指令的具体实现则是在“基于微指令视角的编程指南和样例”章节中说明。

6.1 Tensor Storage Model

宏指令的输入、输出张量存储位置具有两种情况：

1）存储于 AIACC 内部的 UR 中；

2）存储于 AIACC 外部的地址空间，可以是其他 AIACC 的 UR 中，可以是片上 SRAM、也可以是 DRAM，甚至是其他芯片的地址空间。

6.1.1 User Register

多个 SMC 的存储空间按照编号从低到高形成一个连续的地址空间；每个 SMC 中包含多个 Sub UR，这些 Sub UR 组成一片连续地址的存储空间；每个 Sub UR 是一片连续地址的存储空间，可以由一个或者多个 SRAM 构成。所有的 SMC 中相同地址的 Sub UR 构成一个 UR，注意 UR 是由多个间插的地址空间构成的。

本节介绍宏指令的输入、输出张量存储于 AIACC 内部的 UR 中的规则。仅考虑一个四维张量 Tensor 存储于一个

UR 的方式，Tensor=numpy.empty((D3,D2,D1,D0))。对于维度大于 4 的张量，由软件负责如何将其切分为 4 维的 Tile（即 Python 中的 Slice）。Tensor 的存储方式是多样的，本文仅介绍常用的存储方式，不代表不支持其他的存储方式。

6.1.1.1 四维张量展开于多个 Sub UR

因为 SMC 由多个 Sub UR 构成，因此自然 4D Tensor 的一个维度会展开于多个 Sub UR，这里展开的意思是指将具有某一维度上具有不同坐标的 Tile 存储于不同的 Sub UR。约定 D3 是在 Sub_UR 上展开的维度。具体展开有两种方式：

1）连续展开：对于 Tensor[D3, D2, D1, D0]，

Slice_x = Tensor[x*ceil(D3/PARAM_SMC_NUM)：MIN((x+1)*ceil(D3/PARAM_SMC_NUM),D3), D2, D1, D0] 存储于 SMC_x（编号为 x 的 SMC），这里 PARAM_SMC_NUM 代表 SMC 的总数。

2）间插展开：对于 Tensor[D3, D2, D1, D0]，

Slice_x = Tensor[B*x+B*y*PARAM_SMC_NUM：(B*x+B-1)+B*y*PARAM_SMC_NUM, D2, D1, D0] 存储于 SMC_x（编号为 x 的 SMC），x=0~PARAM_SMC_NUM-1，y=0,1,2...ceil(D3 / (B * PARAM_SMC_NUM)) - 1，且需保证 B*x+B*y*PARAM_SMC_NUM 和(B*x+B-1)+B*y*PARAM_SMC_NUM 均小于 D3。

经过展开后，每个 Sub UR 中要存储一个四维张量 Tile，将这样的 Tile 记为 Tile[D3_sub, D2, D1, D0]。

B 称为间插因子的典型值有 1、2、2*PARAM_UR_BYTE_CNT/ELEMENT_BYTE_NUM，代表着每个 SMC 中连续存储多少个 D3 维度为 1 的切片。

6.1.1.2 四维张量的存储参数

对于 Tensor[D3, D2, D1, D0]，表示其存储方式的参数有，Start_addr，D3_stride，D2_stride，D1_stride，D0_stride，他们的定义是：

- Start_addr 为 Tensor 在 SMCs 中的存储起始地址，Start_addr=Sub_UR_addr+offset_addr，其中 Sub_UR_addr 为对应的 Sub_UR 在 SMC 中的偏移地址，offset_addr 为 Tile 在 Sub_UR 中的存储偏移地址。

当前仅考虑 Tensor 总是从 SMC_0 开始存储的情况。

- Dx_stride：对于 Tensor 展开后的 Tile，在同一个 SMC 中，在 Dx 维度相邻，其他维度相同的 element 存储的地址差值，x=0,1,2,3。对于 Dx_stride 需要支持正值、负值和零。
- 对于连续展开，在同一个 SMC 中，在 D3 维度相邻的两个 Tile，其 D3 维度坐标相差 1。对于间插因子为 B 的间插展开，在同一个 SMC 中，在 D3 维度相邻的两个 Tile，其 D3 维度坐标相差 1 或者 B*PARAM_SMC_NUM，这两种情况的 D3_stride 的数值要求一致。

6.1.1.3 在一个 Sub UR 内的 Tile 的典型存储方式

Tensor[D3, D2, D1, D0]典型的存储方式有两种：

1）D0 为存储最内围（D0inner）：D0_stride 为 1，且 D1_stride、D2_stride、D3_stride 是 PARAM_UR_BYTE_CNT 的整数倍。不特殊说明的情况下，UR 中的张量存储都满足本条件。

2）D3 为存储最内围（D3inner）：D3_stride 为 1，且 D0_stride、D1_stride、D2_stride 是 PARAM_UR_BYTE_CNT 的整数倍。

如果输入张量需要在某一个维度上进行广播，则该维度的 stride 可以为 0。

6.1.1.4 Transformer 中 1D 张量存储的典型实现

对于 1D Tensor[1,W]，主要存储方式为两种：

1) 非转置：将 1D Tensor 视为 Tensor[1,1,1,W]，此时所有数据存于一个 Sub UR。此存储格式记为记为 UR_Vec_Norm。

2) 转置连续：将 1D Tensor 视为 Tensor[W,1,1,1]，将原 Tensor 的多列（W 维度）在 Sub_SMC 上连续展开，并将 D3 作为存储最内围。此存储格式记为记为 UR_Vec_Tran_Succ_D3inner。

3) 转置间插：将 1D Tensor 视为 Tensor[W,1,1,1]，将原 Tensor 的多列（W 维度）在 Sub_SMC 上间插展开，并将 D3 作为存储最内围。此存储格式记为记为 UR_Vec_Tran_Inte_D3inner。典型 B 为 2*PARAM_UR_BYTE_CNT/ELEMENT_BYTE_NUM。

6.1.1.5 Transformer 中 2D 张量存储的典型实现

对于 2D Tensor[H,W]，主要存储方式有：

1) 非转置连续：将 2D Tensor 视为 Tensor[H,1,1,W]，将原 Tensor 的多行（H 维度）在 Sub_SMC 上连续展开。

此存储格式记为记为 UR_Mat_Norm。

2) 非转置间插：将 2D Tensor 视为 Tensor[H,1,1,W]，将原 Tensor 的多行（W 维度）在 Sub_SMC 上间插展开。

此存储格式记为记为 UR_Mat_Tran_Inte。典型 B=2。

3) 转置连续：将 2D Tensor 视为 Tensor[W,1,1,H]，将原 Tensor 的多列（W 维度）在 Sub_SMC 上连续展开。

此存储格式记为记为 UR_Mat_Tran_Succ。

4) 转置间插：将 2D Tensor 视为 Tensor[W,1,1,H]，将原 Tensor 的多列（W 维度）在 Sub_SMC 上间插展开。

此存储格式记为记为 UR_Mat_Tran_Inte。典型 B=2。

6.1.1.6 CNN 中 4D 张量存储的典型实现（V1.0 不支持）

对于 Activation Tensor[N, C, H, W]，典型的存储方式有：

1）Normal（W 为最内围）：将其按照 Tensor[C, N, H, W] 的方式进行存储，将 C 维度在 Sub UR 上连续展开或者间插展开。同时，D0_stride 为 1，且 D1_stride、D2_stride、D3_stride 是 PARAM_UR_BYTE_CNT 的整数倍。此存储格式记为记为 UR_Act_Normal。

2）Transpose（C 为最内围）：将其按照 Tensor[H*W, N, 1, C] 的方式进行存储，将 H*W 在 Sub UR 上连续展开或者间插展开。同时，D0_stride 为 1，且 D1_stride、D2_stride、D3_stride 是 PARAM_UR_BYTE_CNT 的整数倍。此存储格式记为记为 UR_Act_Transpose。

对于 Kernal Tensor[IC, OC, H, W]，典型的存储方式有：

1）Normal（IC 为最内围）：将其按照 Tensor[OC, H, W, IC] 的方式进行存储，将 OC 维度在 Sub UR 上连续展

开或者间插展开。同时，D0_stride 为 1，且 D1_stride、D2_stride、D3_stride 是 PARAM_UR_BYTE_CNT 的整数倍。此存储格式记为记为 UR_Ker_Normal。

6.1.2 张量存储于 SMC 外部地址空间

6.1.2.1 四维张量的存储参数及典型存储方式

对于存储于外部地址空间的 Tensor[Dd, Dc, Db, Da]，表示其存储方式的参数有：Start_addr，Dd_stride，Dc_stride，Db_stride，Da_stride，他们的定义是：

- Start_addr 为 Tensor 在外部地址空间中的存储起始地址。
- Dx_stride：Dx 维度相邻，其他维度相同的 element 存储的地址差值，x=a,b,c,d。对于 Dx_stride 需要支持正值、负值和零。

Tensor[Dd, Dc, Db, Da]典型的存储方式为：

Da_stride 为 1，且 D1_stride=Da，D2_stride=Db*Da，D3_stride=Dc*Db*Da。

如果输入张量需要在某一个维度上进行广播，则该维度的 stride 可以为 0。不特殊说明的情况下，外部地址空间中的张量存储都满足本条件。

6.1.2.2 Transformer 中 1D 张量存储的典型实现

对于 1D Tensor[1,W]，存储方式为：

将 1D Tensor 视为 Tensor[1, 1, 1, W]，Da_stride=1。

6.1.2.3 Transformer 中 2D 张量存储的典型实现

对于 2D Tensor[H,W]，支持两种存储方式：

1) 非转置：将 2D Tensor 视为 Tensor[H, 1, 1, W]，Da_stride=1，Dd_stride=W。此存储格式记为记为 GR_Mat_Norm。
2) 转置：将 2D Tensor 视为 Tensor[W, 1, 1, H]，Da_stride=1，Dd_stride=W。此存储格式记为记为

GR_Mat_Tran。

6.1.2.4 CNN 中 4D 张量存储的典型实现（V1.0 不支持）

对于 Activation Tensor[N, C, H, W]，典型的存储方式有：

1）Normal（W 为最内围）：将其按照 Tensor[C, N, H, W] 的方式进行存储，Da_stride=1，Db_stride=W，Dc_stride=H*W，Dd_stride=N*H*W。此存储格式记为记为 GR_Act_Normal。

2）Transpose（C 为最内围）：将其按照 Tensor[H*W, N, 1, C] 的方式进行存储，Da_stride 为 1，Db_stride=C，Dc_stride=C、Dd_stride=N*C。此存储格式记为记为 GR_Act_Transpose。

对于 Kernal Tensor[IC, OC, H, W]，典型的存储方式有：

1）Normal（IC 为最内围）：将其按照 Tensor[OC, H, W, IC] 的方式进行存储，Da_stride 为 1，Db_stride=IC，Dc_stride=IC*W，Dd_stride=H*W*IC。此存储格式记为记为 GR_Ker_Normal。

6.2 支持的数据类型

表 6- 1：支持的数据类型表

dtype	description	V1.0 设计是否支持
torch.float32	32_bit floating point, as defined in https://en.wikipedia.org/wiki/IEEE_754	Y
torch.float64	64_bit floating point, as defined in https://en.wikipedia.org/wiki/IEEE_754	N
torch.float16	16_bit floating point, as defined in https://en.wikipedia.org/wiki/IEEE_754, S_E_M 1_5_10	Y
torch.bfloat16	16_bit floating point, sometimes referred to as Brain floating point, S_E_M 1_8_7	Y
torch.complex32	32_bit complex with two float16 components	N
torch.complex64	64_bit complex with two float32 components	N
torch.complex128	128_bit complex with two float64 components	N
torch.float8_e4m3fn	8_bit floating point, S_E_M 1_4_3, from https://arxiv.org/abs/2209.05433	N

torch.float8_e5m2	8_bit floating point, S_E_M 1_5_2, from https://arxiv.org/abs/2209.05433	N
torch.float8_e4m3fnuz	8_bit floating point, S_E_M 1_4_3, from https://arxiv.org/pdf/2206.02915	N
torch.float8_e5m2fnuz	8_bit floating point, S_E_M 1_5_2, from https://arxiv.org/pdf/2206.02915	N
torch.float8_e8m0fnu	8_bit floating point, S_E_M 0_8_0, from https://www.opencompute.org/documents/ocp_microscaling_formats_mx_v1_0_spec_final_pdf	N
torch.float4_e2m1fn_x2	packed 4_bit floating point, S_E_M 1_2_1, from https://www.opencompute.org/documents/ocp_microscaling_formats_mx_v1_0_spec_final_pdf	N
uint4	4_bit integer (unsigned)	Y
int4	4_bit integer (signed)	Y
torch.uint8	8_bit integer (unsigned)	Y
torch.int8	8_bit integer (signed)	Y
torch.uint16	16_bit integer (unsigned)	Y
torch.int16	16_bit integer (signed)	Y
torch.uint32	32_bit integer (unsigned)	Y
torch.int32	32_bit integer (signed)	Y
torch.uint64	64_bit integer (unsigned)	N
torch.int64	64_bit integer (signed)	N
torch.bool	Boolean	Y

备注：下一版设计会扩展到支持 FP8，FP4 待考虑

7 基于微指令视角的编程指南和样例

7.1 基于微指令视角的编程指南

7.2 基于微指令视角的编程样例

7.2.1 向量浮点加法

本节将浮点加法为例，介绍单一计算指令编程的方法。首先这里例子（而非所有向量计算或者浮点加法向量计算）的前提条件：

1）输入 0 的数据量为：N*128bit，且存储于 UR 的 1 个 Bank 中的连续 N 行。

2）输入 1 和输出也满足上述条件，且输入 0、输入 1 和输出存于不同的 Bank。

3）本例子中假设 N=16，输入 0 存储于 Bank0，输入 1 存储于 Bank1，输出存储于 Bank2。

在此例子中，涉及 3 个数据流，他们的流向分别是：

1）输入 0：从 UR Bank0 读出到 UR 的输出数据端口 UR_RDATA_0、从 UR 的输出数据端口 UR_RDATA_0 流到 FPADD 的输入接口 DVR_FPADD_S0

2）输入 1：从 UR Bank1 读出到 UR 的输出数据端口 UR_RDATA_1、从 UR 的输出数据端口 UR_RDATA_1 流到 FPADD 的输入接口 DVR_FPADD_S1

3）输出：FPADD 的计算结果输出到 FPADD 的输出端口 DR_FPADD_D、从 FPADD 的输出端口 DR_FPADD_D 流到 UR 的输入数据端口 UVR_WDATA_0

在此例子中，涉及的 SMC 模块有三个：FPADD、UR、RtoVR，因此涉及的 SMI_IQ 就是这三个对应的 SMI_IQ。他们分别的微指令为

表 7- 1：UR_R0 SMI_IQ 微指令格式表

IQ 地址	VLD	JUMP	LOOPS	COND	DLY	INC	CODE	说明
0	1	0	16	0	0	0	{ur_rbank_0, ur_raddr_0, ur_re_0}	从 UR 中读取第一个数据
1	1	1	0	0	0	0	0	结束执行

表 7- 2：UR_R1 SMI_IQ 微指令格式表

IQ 地址	VLD	JUMP	LOOPS	COND	DLY	INC	CODE	说明
0	1	0	16	0	0	1	{ur_rbank_1, ur_raddr_1, ur_re_1}	从 UR 中读取第二个数据
1	1	1	0	0	0	0	0	结束执行

表 7- 3：RtoVR_? SMI_IQ 微指令格式表

IQ 地址	VLD	JUMP	LOOPS	COND	DLY	INC	CODE	说明

0	1	0	16	0	1	0	{mv_out?_vld, mv_out?}	将第一个数据从 UR 读出端口传送到 FPADD 的输入端口
1	1	1	0	0	1	0	0	结束执行

表 7- 4：RtoVR_? SMI_IQ 微指令格式表

IQ 地址	VLD	JUMP	LOOPS	COND	DLY	INC	CODE	说明
0	1	0	16	0	1	0	{mv_out?_vld, mv_out?}	将第二个数据从 UR 读出端口传送到 FPADD 的输入端口
1	1	1	0	0	1	0	0	结束执行

表 7- 5：FPADD SMI_IQ 微指令格式表

IQ 地址	VLD	JUMP	LOOPS	COND	DLY	INC	CODE	说明
0	1	0	16	0	2	0	{1,1,1,0}	FPADD 执行加法； DLY 为 2 的原因是从控制 UR 读数据、读出数据、转发到 FPADD 接口，操作延迟了 2 拍
1	1	1	0	0	2	0	0	结束执行

表 7- 6：RtoVR_？ SMI_IQ 微指令格式表

IQ 地址	VLD	JUMP	LOOPS	COND	DLY	INC	CODE	说明
0	1	0	16	0	3	0	{mv_out?_vld, mv_out?}	将加法结果传送回 UR
1	1	1	0	0	3	0	0	结束执行

表 7- 7：UR_W0 SMI_IQ 微指令格式表

IQ 地址	VLD	JUMP	LOOPS	COND	DLY	INC	CODE	说明
0	1	0	16	0	4	0	{ur_wbank_0, ur_waddr_0, ur_we_0, ur_wbe_0}	将加法结果写回 UR
1	1	1	0	0	4	0	0	结束执行

7.2.2 基于加法树的向量与矩阵乘法

基于现有的架构，对于采用加法树的向量与矩阵乘法，其向量、矩阵与结果向量的所有可选的存储形式参考下表。

备注：

1）基于将多个 SMC 的算力充分利用的考虑，只能广播向量。

2）结果矩阵的存储方式一定是和非广播的矩阵存储方式一致的。

表 7- 8：向量和矩阵乘法可选存储格式表

向量、矩阵存储方式	广播	结果向量存储方式	是否可行	实现方式概述
非转置、非转置连续	左	非转置连续	No	广播向量，则矩阵必须转置
非转置、非转置间插	左	非转置间插	No	广播向量，则矩阵必须转置
非转置、转置连续	左	转置连续	Yes（结果与输入存储不一致，不能用该方式实现连乘，不推荐）	从一个 SMC 中读取向量，广播，与各个 SMC 中的列向量点积 如果向量是读取自 DRAM，那么读取的带宽受限于单个 SMC，但是向量读取占比重低
非转置、转置间插	左	转置间插	Yes（结果与输入存储不一致，不能用该方式实现连乘，不	从一个 SMC 中读取向量，广播，与各个 SMC 中的列向量点积 如果向量是读取自 DRAM，那么读取的带宽受限于单个 SMC，但是向量读取占比重低

			推荐)	
转置连续、非转置连续	左	非转置连续	No	广播向量，则矩阵必须转置
转置连续、非转置间插	左	非转置间插	No	广播向量，则矩阵必须转置
转置连续、转置连续	左	转置连续	Yes (输入向量较小的时候适用)	依次从一个 SMC 中读取向量，广播，与各个 SMC 中的列向量点积
转置连续、转置间插	左	转置间插	Yes (输入向量较小的时候适用)	依次从一个 SMC 中读取向量，广播，与各个 SMC 中的列向量点积
转置间插、非转置连续	左	非转置	No	广播向量，则矩阵必须转置
转置间插、非转置间插	左	非转置间插	No	广播向量，则矩阵必须转置
转置间插、转置连续	左	转置连续	Yes (输入向量较大的时候适用)	从多个 SMC 中读取向量，广播，与各个 SMC 中的列向量点积 向量是读取自 DRAM，要求选取较大的间插因子 B，以提高 DRAM 读取效率，但是太大会导致读取逻辑复杂，一般选取 2*PARAM_UR_BYTE_CNT/ELEMENT_BYTE_NUM
转置间插、转置间插	左	转置间插	Yes (输入向量较大的时候适用)	从多个 SMC 中读取向量，广播，与各个 SMC 中的列向量点积 如果向量是读取自 DRAM，要求选取较大的间插因子 B，以提高 DRAM 读取效率，但是太大会导致读取逻辑复杂，一般选取 2*PARAM_UR_BYTE_CNT/ELEMENT_BYTE_NUM

总结：

- 向量与矩阵相乘的时候，比较推荐的是向量用转置连续或者转置间插存储，矩阵用转置连续或者转置间插存储
- 对于向量来说，当数据量比较小，小于 PARAM_UR_BYTE_CNT/ELEMENT_BYTE_NUM*PARAM_SMC_NUM 的时候，用转置连续；当数据量大于

这个门限之后推荐采用转置间插。

- 对于矩阵来说，一般来说，列数比较小，小于 PARAM_UR_BYTE_CNT/ELEMENT_BYTE_NUM*PARAM_SMC_NUM 的时候，用转置连续；当数据量大于这个门限之后推荐采用转置间插。这是由于向量与矩阵乘法的结果是向量，结果向量期望的存储方式，决定了矩阵的存储方式。

基于应用的考量：

- 对于 Transformer 中的 MHA 来说，向量经常是需要分组的，这就造成总的数据量比较大，但是分组后，向量比较小，此时注意要保证分组后的数据是转置连续展开的。这要求权重矩阵应该先分为 32 组，然后每组用转置连续的方式进行存储。

7.2.2.1 向量、权重矩阵和结果矩阵均为转置间差展开存储时候的向量与矩阵乘法过程

考虑输入向量、输入矩阵和结果向量都按照转置间差展开存储方式进行存储，间插因子 B 为 2。

向量乘以矩阵的一个基础计算粒度是 B*(PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_NUM)*PARAM_SMC_NUM 的向量被读取并且广播，在 PARAM_SMC_NUM 个 SMC 中分别于一个列的部分数据（数据个数为 B*(PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_NUM)*PARAM_SMC_NUM ）进行点积，于 PARAM_SMC_NUM 个 SMC 分别产生 1 个部分和。

下一步有三种选择：

1）复用 ACTIVATION Tile，读取更多的 WEIGHT Tile，计算更多 RESULT 列的部分和，此时需要将之前计算的部分和写回 UR。

2）复用 WEIGHT Tile，读取更多的 ACTIVATION Tile，计算更多 RESULT 行的部分和，此时需要将之前计算的部分和写回 UR。

3）复用 RESULT Tile，按照从左到右的顺序读取更多的 ACTIVATION Tile，按照从上到下的顺序读取更多的

WEIGHT Tile，进行点积，结果累加到 RESULT 上。

三种方式都可实现功能，亦可间插进行复用，具体可以按照功耗来评估具体数据参数下的孰优孰劣以进行选择。

从性能的角度，矩阵规模足够大的情况下，三种选择都是一样的。

7.2.2.1.1 激活向量读取微指令

对于一次广播给所有 SMC 的激活向量读取，将消耗 PARAM_SMC_NUM*2 的时间反馈数据（其中一半为气泡），反馈的数据为 PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_CNT*PARAM_SMC_NUM 的向量，其中 ACTIVATION_BYTE_CNT 为激活向量一个元素包含的字节数。

连续发出两个读指令，由于读取流水线的构造，当 B=2 的时候，读回的数据按照顺序恰好是激活向量中的连续的 B*(PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_NUM)*PARAM_SMC_NUM 个数据，不需要进行转置即可发出广播。这些数据以线速进行接收和广播，每个时钟周期接收和广播 PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_CNT 个激活向量数据。

7.2.2.1.2 权重矩阵读取微指令

广播出去的数据将在每个 SMC 中与 SMC 内部读取的权重 Tile 做点积，每个时钟周期读取（PARAM_UR_BYTE_CNT/WEIGHT_BYTE_CNT）个权重，对应 ACTIVATION Tile 一行的 WEIGHT 也是 2*PARAM_SMC_NUM 个，需要 2*PARAM_SMC_NUM/（PARAM_UR_BYTE_CNT/WEIGHT_BYTE_CNT）个时钟周期将对应 ACTIVATION 一行的 WEIGHT 读取到对应的计算单元。所有的 SMC 合并起来读取了一个 2*PARAM_SMC_NUM 行、PARAM_SMC_NUM 列的 Tile。

7.2.2.1.3 点积计算微指令

读取的 ACTIVATION 一行和 WEIGHT Tile 在计算单元做点积计算，每个 SMC 中产生 1 个 RESULT。

7.2.2.1.4 结果向量写回微指令

这些 RESULT 是 RESULT 矩阵一列中 1 个结果的部分和。所有 SMC 合并起来，产生了 1 行 PARAM_SMC_NUM 列的 RESULT Tile 的部分和。

7.2.2.2 向量、权重矩阵和结果矩阵均为转置间差展开存储时候的向量与矩阵乘法过程举例

表 7- 9：向量与矩阵乘法举例 LLAMA2 7B Decode 阶段

	向量输入			权重矩阵输入			输出			说明
	G	H	W	G	H	W	G	H	W	
Matmul_Q	1	1	4096	32	4096	128	32	1	128	分 32 组进行计算。 输入 1：1*4096 个数据要分散到 32 个 SMC 中，每个中 128 个数据。转置间差展开。 输入 2：128 列分散到 32 个 SMC 中，每个 4 列。转置连续展开。 输出：128 个数据分散到 32 个 SMC，每个 4 个。转置连续展开。
Matmul_K	1	1	4096	1	4096	4096	1	1	4096	输入 1：1*4096 个数据要分散到 32 个 SMC 中，每个中 128 个数据。转置间差展开。 输入 2：4096 列分散到 32 个 SMC 中，每个 128 列。转置间插展开。 输出：1*4096 个数据要分散到 32 个 SMC 中，每个中 128 个数据。转置间差展开。
Matmul_V	1	1	4096	32	4096	128	32	1	128	同 Matmul_Q
Matmul_QK	32	1	128	32	128	4096	32	1	4096	分 32 组进行计算。 输入 1：来自 Matmul_Q 的输出，不需要移动。转置连续展开。 输入 2：4096 列分散到 32 个 SMC 中，每个 128 列。转置间插展开。其中最后一列来自 Matmul_K 的计算结果，需要把 Matmul_K 的结果收集存储到最后一个 SMC，再进行拼接。

									输出：1*4096 个数据要分散到 32 个 SMC 中，每个 128 个数据。转置间插展开。
Matmul_QKV	32	1	4096	32	4096	128	32	1	128	分 32 组进行计算。 输入 1：1*4096 个数据要分散到 32 个 SMC 中，每个中 128 个数据。转置连续展开。 输入 2：128 列分散到 32 个 SMC 中，每个 4 列。转置连续展开。其中最后一行来自 Matmul_V 的计算结果，需要在 SMC 内部进行数据拼接。 输出：1*128 个数据要分散到 32 个 SMC 中，每个 4 个数据。转置连续展开。
Matmul_FC	1	1	4096	1	4096	4096	1	1	4096	输入 1：1*4096 个数据，分散到 32 个 SMC 中，每个中 128 个数据。来自 Matumul_QKV，但是源数据是分为 32 组的，每组是转置连续展开的。 输入 2：4096 列分散到 32 个 SMC 中，每个 128 列。转置间插展开。 输出：1*4096 个数据要分散到 32 个 SMC 中，每个 128 个数据。转置间差展开。

这里用到

- 转置间差*转置连续=转置连续
- 转置间差*转置间差=转置间差
- 转置连续*转置间差=转置间差

三种计算方式。

7.2.3 基于加法树的矩阵与矩阵乘法

基于现有的架构，对于采用加法树的矩阵与矩阵乘法，其左矩阵、右矩阵与结果矩阵的所有可选的存储形式参考下表。

备注：

1）左右矩阵原则上都可以广播，但是因为这里是用加法树实现，所以，广播左矩阵一定是广播一行中的一个片段，广播右矩阵一定是广播一列中的一个片段。

2）结果矩阵的存储方式一定是和非广播的矩阵存储方式一致的。

表 7- 10：可选存储格式表

左矩阵、右矩阵存储方式	广播矩阵	结果矩阵存储方式	是否可行	实现方式概述
非转置连续、非转置连续	左	非转置连续	No	广播左矩阵，则右矩阵必须转置
非转置连续、非转置间插	左	非转置间插	No	广播左矩阵，则右矩阵必须转置
非转置连续、转置连续	左	转置连续	Yes（读取行向量的指令较为复杂，不推荐）	从一个 SMC 中读取行向量，广播，与各个 SMC 中的列向量点积 读完一个 SMC 中所有行向量之后转到第二个 SMC
非转置连续、转置间插	左	转置间插	Yes（读取行向量的指令较为复杂，不推荐）	从一个 SMC 中读取行向量，广播，与各个 SMC 中的列向量点积 读完一个 SMC 中所有行向量之后转到第二个 SMC
非转置间插、非转置连续	左	非转置连续	No	广播左矩阵，则右矩阵必须转置
非转置间插、非转置间插	左	非转置间插	No	广播左矩阵，则右矩阵必须转置
非转置间插、转置连续	左	转置连续	Yes（连续存储仅适用于向量，不推荐）	依次从多个 SMC 中读取行向量，广播，与各个 SMC 中的列向量点积
非转置间插、转置间插	左	转置间插	Yes（结果矩阵列数较多的	依次从多个 SMC 中读取行向量，广播，与各个 SMC 中的列向量点积

			时候适用)	
转置连续、非转置连续	左	非转置连续	No	广播左矩阵，则右矩阵必须转置
转置连续、非转置间插	左	非转置间插	No	广播左矩阵，则右矩阵必须转置
转置连续、转置连续	左	转置连续	Yes (想读取连续的列复杂，不推荐)	从多个 SMC 中读取列向量，转置，广播，与各个 SMC 中的列向量点积
转置连续、转置间插	左	转置间插	Yes (想读取连续的列负责，不推荐)	从多个 SMC 中读取列向量，转置，广播，与各个 SMC 中的列向量点积
转置间插、非转置连续	左	非转置连续	No	广播左矩阵，则右矩阵必须转置
转置间插、非转置间插	左	非转置间插	No	广播左矩阵，则右矩阵必须转置
转置间插、转置连续	左	转置连续	Yes (连续存储仅适用于向量，不推荐)	从多个 SMC 中读取列向量，转置，广播，与各个 SMC 中的列向量点积
转置间插、转置间插	左	转置间插	Yes (结果矩阵列数较多的时候适用)	从多个 SMC 中读取列向量，转置，广播，与各个 SMC 中的列向量点积
非转置连续、非转置连续	右	非转置连续	Yes (连续存储仅适用于向量，不推荐)	从多个 SMC 读取右矩阵的行向量，转置，广播，与各个 SMC 中的行向量点积
非转置连续、非转置间插	右	非转置连续	Yes (连续存储仅适用于向量，不推荐)	从多个 SMC 读取右矩阵的行向量，转置，广播，与各个 SMC 中的行向量点积
非转置连续、转置连续	右	非转置连续	Yes (连续存储仅适用于向	依次从一个 SMC 中读取列向量，广播，与各个 SMC 中的列向量点积

			量，不推荐)	
非转置连续、转置间插	右	非转置连续	Yes (连续存储仅适用于向量，不推荐)	依次从一个 SMC 中读取列向量，广播，与各个 SMC 中的列向量点积
非转置间插、非转置连续	右	非转置间插	Yes (指令复杂，不推荐)	从多个 SMC 读取右矩阵的行向量，转置，广播，与各个 SMC 中的行向量点积
非转置间插、非转置间插	右	非转置间插	Yes	从多个 SMC 读取右矩阵的行向量，转置，广播，与各个 SMC 中的行向量点积
非转置间插、转置连续	右	非转置间插	Yes (指令复杂，不推荐)	依次从一个 SMC 中读取列向量，广播，与各个 SMC 中的列向量点积
非转置间插、转置间插	右	非转置间插	Yes	依次从一个 SMC 中读取列向量，广播，与各个 SMC 中的列向量点积
转置连续、非转置连续	右	转置连续	No	广播右矩阵，则左矩阵必须非转置
转置连续、非转置间插	右	转置连续	No	广播右矩阵，则左矩阵必须非转置
转置连续、转置连续	右	转置连续	No	广播右矩阵，则左矩阵必须非转置
转置连续、转置间插	右	转置连续	No	广播右矩阵，则左矩阵必须非转置
转置间插、非转置连续	右	转置间插	No	广播右矩阵，则左矩阵必须非转置
转置间插、非转置间插	右	转置间插	No	广播右矩阵，则左矩阵必须非转置
转置间插、转置连续	右	转置间插	No	广播右矩阵，则左矩阵必须非转置
转置间插、转置间插	右	转置间插	No	广播右矩阵，则左矩阵必须非转置
转置间插、非转置连续	右	转置间插	No	广播右矩阵，则左矩阵必须非转置
转置间插、非转置间插	右	转置间插	No	广播右矩阵，则左矩阵必须非转置

转置间插、转置连续	右	转置间插	No	广播右矩阵，则左矩阵必须非转置
转置间插、转置间插	右	转置间插	No	广播右矩阵，则左矩阵必须非转置

总结：

- 对于矩阵来说，因为数据量大，间插存储方式最友好，一般可以选取间插因子为 2，读取向量逻辑最简化，读取延迟最低。
- 如果广播左矩阵，那么左矩阵的存储方式任意，但是右矩阵要求转置存储。
- 如果广播右矩阵，那么右矩阵的存储方式任意，但是左矩阵要求非转置存储。

基于应用的考量：

- 矩阵乘法如果不需要量化的时候，广播左矩阵或者右矩阵均可；如果涉及量化的时候，广播非量化的矩阵对于指令来说更简单。
- 当涉及矩阵连乘的时候，结果矩阵要么是下一次计算的左矩阵，要么是下一次计算的右矩阵。如果不涉及转置的话，那么所有矩阵都选择一致的存储方式更简便，比如都选择非转置间插或者转置间插。如果涉及转置的话，就要求结果矩阵可以更灵活的选择存储格式，非转置间插经过转置（实际不需要进行操作）就变成了转置后的转置间插了，而转置间插经过转置（实际不需要进行操作）就变成了转置后的非转置间插了。这样通过安排好前面矩阵乘计算的数据存储格式，可以规避数据搬运。
- 但是对于数据量特别大，远超片上 SRAM 存储的时候，结果矩阵一般需要写回 DRAM，此时就不需要考虑如果规避转置操作，而是如果高效率的写回 DRAM 和读取 DRAM，此时可以利用转置算子先对数据进行整理，再写回。

7.2.3.1 左右结果矩阵均为转置间差展开存储时候的矩阵乘法过程

考虑左矩阵、右矩阵和结果矩阵都按照转置间差展开存储方式进行存储。

矩阵乘以矩阵的一个基础计算粒度是

B*(PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_NUM)*PARAM_SMC_NUM 的向量被读取并且广播，在 PARAM_SMC_NUM 个 SMC 中分别于一个列的部分数据（数据个数为 B*(PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_NUM)*PARAM_SMC_NUM ）进行点积，于 PARAM_SMC_NUM 个 SMC 分别产生 1 个部分和。

下一步有三种选择：

1）复用 ACTIVATION Tile，读取更多的 WEIGHT Tile，计算更多 RESULT 列的部分和，此时需要将之前计算的部分和写回 UR。

2）复用 WEIGHT Tile，读取更多的 ACTIVATION Tile，计算更多 RESULT 行的部分和，此时需要将之前计算的部分和写回 UR。

3）复用 RESULT Tile，按照从左到右的顺序读取更多的 ACTIVATION Tile，按照从上到下的顺序读取更多的 WEIGHT Tile，进行点积，结果累加到 RESULT 上。

三种方式都可实现功能，亦可间插进行复用，具体可以按照功耗来评估具体数据参数下的孰优孰劣以进行选择。从性能的角度，矩阵规模足够大的情况下，三种选择都是一样的。

7.2.3.1.1 激活矩阵读取微指令

对于一次广播给所有 SMC 的激活向量读取，将消耗 PARAM_SMC_NUM*2 的时间反馈数据（其中一半为气泡），反馈的数据为 PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_CNT 行、PARAM_SMC_NUM 列的 Tile，其中 ACTIVATION_BYTE_CNT 为激活向量一个元素包含的字节数。

连续发出两个读指令，由于读取流水线的构造，当 B=2 的时候，读回的数据按照顺序恰好是激活矩阵中的连续的 PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_NUM 行和 2*PARAM_SMC_NUM 列，读回来的数据是按照列对齐的（每个时钟周期内的数据是一列的一部分数据）。

7.2.3.1.2 激活矩阵排序微指令

将读回的 Tile 转置为行对齐（每个时钟周期内的数据为一行的一部分数据）。每一行的数据分 2*PARAM_SMC_NUM/(PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_CNT) 个时钟周期将其广播出去，每次广

播 (PARAM_UR_BYTE_CNT/ACTIVATION_BYTE_CNT) 个元素。

7.2.3.1.3 权重矩阵读取微指令

广播出去的数据将在每个 SMC 中与 SMC 内部读取的权重做点积，每个时钟周期读取（PARAM_UR_BYTE_CNT/WEIGHT_BYTE_CNT）个权重，对应 ACTIVATION 一行的 WEIGHT 也是 2*PARAM_SMC_NUM 个，需要 2*PARAM_SMC_NUM/（PARAM_UR_BYTE_CNT/WEIGHT_BYTE_CNT）个时钟周期将对应 ACTIVATION 一行的 WEIGHT 读取到对应的计算单元。所有的 SMC 合并起来读取了一个 2*PARAM_SMC_NUM 行、PARAM_SMC_NUM 列的 Tile。

7.2.3.1.4 点积计算微指令

读取的 ACTIVATION 一行和 WEIGHT Tile 在计算单元做点积计算，每个 SMC 中产生 1 个 RESULT。

7.2.3.1.5 结果矩阵写回微指令

这些 RESULT 是 RESULT 矩阵一列中 1 个结果的部分和。所有 SMC 合并起来，产生了 1 行 PARAM_SMC_NUM 列的 RESULT Tile 的部分和。

7.2.3.2 左右结果矩阵均为转置间差展开存储时候的矩阵乘法过程举例

表 7- 11：向量与矩阵乘法举例 LLAMA2 7B Decode 阶段

	向量输入			权重矩阵输入			输出			说明
	G	H	W	G	H	W	G	H	W	
Matmul_Q	1	iLen	4096	32	4096	128	32	iLen	128	分 32 组进行计算。 输入 1：iLen*4096 个数据要分散到 32 个 SMC 中，每个中 iLen/32*4096 个数据。非转置间插展开。 输入 2：128 列分散到 32 个 SMC 中，每个 4 列。转置间插展开。 广播左矩阵。 输出：iLen*128 个数据分散

										到 32 个 SMC，每个 iLen*4 个。转置间插展开。
Matmul_K	1	iLen	4096	32	4096	128	32	iLen	128	分 32 组进行计算。 输入 1：iLen*4096 个数据要分散到 32 个 SMC 中，每个中 iLen/32*4096 个数据。非转置间插展开。 输入 2：128 列分散到 32 个 SMC 中，每个 4 列。转置间插展开。 广播右矩阵。 输出：iLen*128 个数据分散到 32 个 SMC，每个 iLen/32*128 个。非转置间插展开。
Matmul_V	1	iLen	4096	32	4096	128	32	iLen	128	同 Matmul_Q
Matmul_QK	32	iLen	128	32	128	iLen	32	iLen	iLen	分 32 组进行计算。 输入 1：来自 Matmul_Q 的输出，不需要移动。转置间插展开。 输入 2：来自 Matmul_K 的输出，不需要移动。因为需要转置，所以原来的非转置间插展开，变成了转置间插展开。 输出：iLen*iLen 个数据要分散到 32 个 SMC 中，每个 iLen/32 列数据。转置间插展开。
Matmul_QKV	32	iLen	iLen	32	iLen	128	32	iLen	128	分 32 组进行计算。 输入 1：来自 Matmul_QK 的输出，不需要移动。转置间插展开。 输入 2：128 列分散到 32 个 SMC 中，每个 4 列。转置间插展开。 输出：1*128 个数据要分散到 32 个 SMC 中，每个 4 个数据。转置间插展开。
Matmul_FC	1	iLen	4096	1	4096	4096	1	iLen	4096	输入 1：iLen*4096 个数据，分散到 32 个 SMC 中，

										每个中 iLen*128 个数据。来自 Matumul_QKV，转置间插展开。 输入 2：4096 列分散到 32 个 SMC 中，每个 128 列。转置间插展开。 输出：1*4096 个数据要分散到 32 个 SMC 中，每个 128 个数据。转置间插展开。

如果考虑尽量避免转置操作的话，需要用到三种计算方式：

- 非转置间插*转置间插=转置间插
- 非转置间插*转置间插=非转置间插
- 转置间插*转置间插=转置间插

如果考虑到数据量大，必须写回 DRAM，那么建议统一采用一种计算方式：

- 转置间插*转置间插=转置间插

因为此时广播的是左矩阵，指令简单。

7.2.4 基于乘累加器的矩阵乘法（V1.0 不支持）

7.2.5 卷积（V1.0 不支持）

1）当 IW>8 且 OW>8 的时候，卷积计算的伪代码如下：

```
for on in range (0 , ON , 1)
    for oc in range (0 , OC , PARAM_SMC_NUM)
        for oh in range (OH_BEGIN , OH_END , 1)
            for ow in range (OW_BEGIN , OW_END, 16) //基于 INT8 精度，同时计算 16 个结果
                for kh in range (0 , KH , 1)
                    for kw in range (0 , KW , 1)
                        for ic (0 , OC , PARAM_SMC_NUM)
```

```
ih = F_ih ( oh , kh , IH_BEGIN , IH_END , T_PADDING , B_PADDING ,
H_STRIDE , KH , KH_DILATION ) ;
iw[15:0] = F_iw ( ow [15:0] , kw , IW_BEGIN , IW_END , L_PADDING ,
R_PADDING , W_STRIDE , KW , KW_DILATION ) ;
for i in range (0 , PARAM_SMC_NUM)
    O[on, oc + i, oh, ow] = I [ on , ic : ic + PARAM_SMC_NUM-1 , ih , iw[0]]
* K [ ic : ic + 32 , oc + i , kh , kw ] + O [ on , oc + i , oh , ow];
    ......
    O[on, oc + i, oh, ow+15] = I [ on , ic : ic + PARAM_SMC_NUM-1 , ih ,
iw[15]] * K [ ic : ic + 32 , oc + i , kh , kw ] + O [ on , oc + i , oh , ow+15];
```

这里：

- 对于 F_ih 和 F_iw 函数的说明，因为卷积核为奇数，所以当不带有卷积 stride 的时候[oh,ow]对应的 $[ih,iw]_{after_padding}$ 以及对应的 $[ih,iw]_{before_padding}$ 满足如下关系：

$ih_{after_padding} = oh * S_h + kh * D_h$

$iw_{after_padding} = ow * S_w + kw * D_w$

$ih_{before_padding} = oh * S_h + kh * D_h - P_{top}$，当结果小于 0 或者结果大于 MAX 的时候，用 Padding 值取代读取值。

$iw_{before_padding} = ow * S_w + kw * D_w - P_{left}$，当结果小于 0 或者结果大于 MAX 的时候，用 Padding 值取代读取值。

- 每个 SMC 同时计算 16 个点，他们具有相同的 oh，连续的 ow。
  - 由于规定输入和输出的每一行第一个点要对齐到 SMC 中 UR 的一行
  - 有 $iw[0] = ow[0] * S_w + kw * D_w - P_{left}$，只有 $kw * D_w = P_{left}$ 的时候 iw[0]是在行对齐的位置
  - 而 $iw[15] = ow[15] * S_w + kw * D_w - P_{left}$
  - 则 $iw[15]-iw[0] = 15 * S_w$，当 $kw * D_w = P_{left}$ 的时候，需要读取 S_w 行以获取计算 16 个输出点所需的输入点；当!($kw * D_w = P_{left}$)的时候，需要读取 S_w+1 行以获取计算 16 个输出点所需的输入点。这里也具有足够的信息使得数据读回之后可以精确地获取需要的数据。

2）当 IW>8 且 OW<=8 的时候，按照上述方式计算则带来较大的算力浪费。这时候输出为每个 UR 含存储 2 行，这两行具有相同的 N、H、W，具有不同的 C。此时伪代码修正为：

3）当 IW<=8 且 OW<=8 的时候，按照上述方式计算则带来较大的算力浪费。这时候输入和输出为每个 UR 含存储 2 行，这两行具有相同的 N、H、W，具有不同的 C。此时伪代码修正为：

4）不考虑 IW<=8 且 OW<=8 的情况。

7.2.5.1 卷积激活读取

7.2.5.2 卷积激活排序

7.2.5.3 卷积权重读取和数据写回

7.2.6 池化和深度分离卷积（V1.0 不支持）

8 宏指令集（后端接口 IR）

AIACC 的宏指令集在两个点体现：

- 一是在编译器的角度是体现为编译器后端接口的 IR
- 二是在芯片的角度是体现为一组微指令。

备注：宏指令列表在单独的文档，待定稿后整理到本文档

9 支持的量化方案（待完善）

9.1 Transformer 量化方案

9.1.1 W4IA16F 量化方案

V1 主要实现 W4IA16F 量化方案，也就是权重用 4bit 量化，激活用 FP16 量化。

9.1.1.1 激活量化

激活 FP16 和 FP32 之间的转换遵循 IEEE 标准，不赘述。

特殊的，所有的特殊函数都需要用 FP32 数据精度进行计算。

9.1.1.2 权重量化

权重的量化方案遵循 AWQ 的方案，参考 vLLM 框架中的实现。

存储用 uint4，Zero_point（Z）用 uint4，Scalar（S）用 FP16。当权重作为右矩阵时，每一列的相邻 64 个权重使用同一个 Scalar。

$$W_{dq} = S_{fp16} * FP16(W_{uint4} - Z_{uint4})$$

9.1.2 W16FA16F 量化方案

9.1.3 W8IA8I 量化方案

9.1.4 W8FA8F 量化方案